



BANCO DE DADOS SQL E NOSQL

ALEXANDRE SAVARIS
CARLOS DANILO LUZ
EDSON YANAGA
JEFERSON KAISER
VICTOR DE MARQUI PEDROSO
WILLIAM ROBERTO PELISSARI

ORGANIZADOR
ANDRÉ OLÍMPIO



**ACESSE AQUI ESTE
MATERIAL DIGITAL!**

EXPEDIENTE

Coordenador(a) de Conteúdo

Nader Ghoddosi

Projeto Gráfico e Capa

Arthur Cantareli Silva

Editoração

Alexandre Donzelli

Design Educacional

Rossana Giani

Revisão Textual

Cindy Luca

Ilustração

André Azevedo

Wellington Vainer S. Oliveira

Geison Odlevati Ferreira

Fotos

Shutterstock e Envato

FICHA CATALOGRÁFICA

N964 Núcleo de Educação a Distância. **SAVARIS**, Alexandre; **LUZ**, Carlos Danilo; **YANAGA**, Edson; **KAISER**, Jeferson; **PEDROSO**, Victor de Marqui; **PELISSARI**, William Roberto.

Banco De Dados Sql e NoSQL / Alexandre Savaris, Carlos Danilo Luz, Edson Yanaga, Jeferson Kaiser, Victor De Marqui Pedroso, William Roberto Pelissari; organizador: André Olímpio. - Florianópolis, SC: Arquê, 2024.

227 p.

ISBN papel 978-65-6137-743-0

ISBN digital 978-65-6137-742-3

1. Banco e 2. Dados 3. NoSQL 4. EaD. I. Título.

CDD - 005.1

Bibliotecária: Leila Regina do Nascimento - CRB- 9/1722.

Ficha catalográfica elaborada de acordo com os dados fornecidos pelo(a) autor(a).

Impresso por:

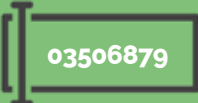
AVALIE ESTE LIVRO!



CRIAR MOMENTOS DE APRENDIZAGENS INESQUECÍVEIS É O NOSSO OBJETIVO E POR ISSO, **GOSTARIAMOS DE SABER COMO FOI SUA EXPERIÊNCIA.**

Conta para nós! leva menos de 2 minutos. Vamos lá?!

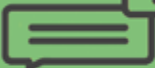
DIGITE O CÓDIGO

 03506879

 Aa

RESPONDA A PESQUISA









RECURSOS DE IMERSÃO



PENSANDO JUNTOS

Este item corresponde a uma proposta de reflexão que pode ser apresentada por meio de uma frase, um trecho breve ou uma pergunta.



APROFUNDANDO

Utilizado para temas, assuntos ou conceitos avançados, levando ao aprofundamento do que está sendo trabalhado naquele momento do texto.



EU INDICO

Utilizado para agregar um conteúdo externo.



ZOOM NO CONHECIMENTO

Utilizado para desmistificar pontos que possam gerar confusão sobre o tema. Após o texto trazer a explicação, essa interlocução pode trazer pontos adicionais que contribuam para que o estudante não fique com dúvidas sobre o tema.

PRODUTOS AUDIOVISUAIS

Os elementos abaixo possuem recursos audiovisuais. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.



PLAY NO CONHECIMENTO

Professores especialistas e convidados, ampliando as discussões sobre os temas por meio de fantásticos podcasts.



EM FOCO

Utilizado para aprofundar o conhecimento em conteúdos relevantes utilizando uma linguagem audiovisual.



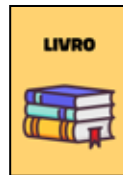
INDICAÇÃO DE FILME

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de filmes que se conectam com o tema do conteúdo.



INDICAÇÃO DE LIVRO

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de livros que agregarão muito na sua vida profissional.



SUMÁRIO

7

UNIDADE 1

INTRODUÇÃO AO BANCO DE DADOS	8
MODELAGEM DE DADOS.	32
LINGUAGEM SQL.	58

83

UNIDADE 2

CRIANDO BANCOS DE DADOS SQL.	84
CRIANDO UM BANCO DE DADOS E A LINGUAGEM SQL	108
INTRODUÇÃO AOS BANCOS DE DADOS NOSQL	130

155

UNIDADE 3

BANCOS DE DADOS PARA SÉRIES TEMPORAIS ORIENTADOS A CHAVE-VALOR	156
CONEXÕES E RELACIONAMENTOS EM BANCOS DE DADOS ORIENTADOS A GRAFOS	180
FLEXIBILIDADE DE ARMAZENAMENTO EM BANCOS DE DADOS ORIENTADO A COLUNAS.	204

unidade





TEMA DE APRENDIZAGEM 1

INTRODUÇÃO AO BANCO DE DADOS

MINHAS METAS

- Estabelecer uma compreensão sólida dos fundamentos dos bancos de dados.
- Desenvolver habilidades de modelagem de dados.
- Ser capaz de formular consultas, inserir, atualizar e excluir dados em um banco de dados relacional.
- Distinguir diferentes modelos de banco de dados.
- Aplicar os conhecimentos adquiridos em projetos simples.
- Definir e contextualizar os conceitos de modelagem, gerenciamento de banco de dados, SQL e transação.
- Identificar os pontos positivos sobre a utilização do sistema de gerenciamento de banco de dados.

INICIE SUA JORNADA

Nesta jornada, nós mergulharemos nos fundamentos da estrutura de armazenamento e gerenciamento de bancos de dados. Neste tema de aprendizagem, desvendaremos os alicerces que sustentam a organização inteligente e eficiente dos dados existentes no ambiente digital. Assim, prepare-se, pois exploraremos o universo das tabelas, dos registros e dos campos, entendendo como esses elementos formam a espinha dorsal dos sistemas de informação modernos existentes no mercado.

Ao embarcar nesta jornada, você descobrirá como um Sistema de Gerenciamento de Banco de Dados (SGBD) atua como o maestro por trás da harmonia das informações, fazendo uma varredura que vai desde a compreensão estrutural que é conhecida como modelagem até a aplicação prática da Linguagem de Consulta Estruturada, a famosa SQL.

A nossa missão é descomplicar a complexidade do tema, fornecendo as ferramentas essenciais para que você possa navegar com confiança pelo vasto oceano da gestão de dados. Vamos começar essa emocionante jornada pela essência dos bancos de dados!



PLAY NO CONHECIMENTO

Mergulharemos nas águas profundas dos bancos de dados. Em nosso podcast intitulado *Desbravando dados: uma jornada na introdução aos mistérios dos bancos de dados*, iniciaremos essa expedição emocionante. Neste episódio, a nossa viagem começa entendendo a arquitetura essencial dos bancos de dados, explorando o significado de alguns conceitos, como tabelas, registros e campos, que são fundamentais para a formação da estrutura dessa vasta reserva de informações digitais. À medida que avançamos, o papel crítico do Sistema de Gerenciamento de Banco de Dados (SGBD) aumenta gradualmente e direciona o fluxo de informações em sistemas complexos. Prepare-se para decifrar os códigos da Linguagem de Consulta Estruturada (SQL) e desbloquear os segredos por trás da manipulação eficaz de dados. Essa é a sua chance de dar um play no conhecimento e iniciar uma jornada emocionante pelos bastidores dos bancos de dados. Vamos explorar juntos as maravilhas e os desafios que aguardam aqueles que desejam desbravar o vasto mundo dos dados digitais!

VAMOS RECORDAR?

Ao longo das décadas, testemunhamos uma notável evolução nos métodos de armazenamento de dados, que moldaram o cenário da tecnologia da informação. Nas fases iniciais, os sistemas de armazenamento eram dominados por fitas magnéticas, oferecendo capacidade limitada e acesso sequencial. Contudo, a ascensão dos discos rígidos na década de 1960 revolucionou o armazenamento, proporcionando acesso aleatório e maior velocidade. O advento dos dispositivos de armazenamento flash nas últimas duas décadas representou outro marco, introduzindo a agilidade dos SSDs e a diminuição do tamanho físico, além de impulsionar a popularidade de dispositivos de armazenamento portáteis.

Mais recentemente, a computação em nuvem emergiu como uma revolução na maneira como concebemos o armazenamento de dados. Ao oferecer escalabilidade, flexibilidade e acessibilidade, a computação em nuvem permitiu às organizações libertar-se das limitações físicas, armazenando e acessando dados de forma remota. Costumamos dizer que essa evolução contínua reflete não apenas avanços tecnológicos, mas também a crescente demanda por soluções de armazenamento mais eficientes e adaptáveis em um mundo digital em constante transformação.

Para saber mais sobre a evolução dos tipos de armazenamento de dados, assista ao vídeo a seguir. link: <https://www.youtube.com/watch?v=lkBg4RVR3Zo>

DESENVOLVA SEU POTENCIAL

O QUE É UM BANCO DE DADOS?

Esse conceito pode ser descrito como uma estrutura organizada que abriga conjuntos de dados relacionados entre si e é projetada para armazenar, gerenciar e recuperar informações de forma eficiente. Essa ferramenta centralizada é essencial em ambientes onde a coleta, a organização e o acesso rápido a dados são fundamentais.

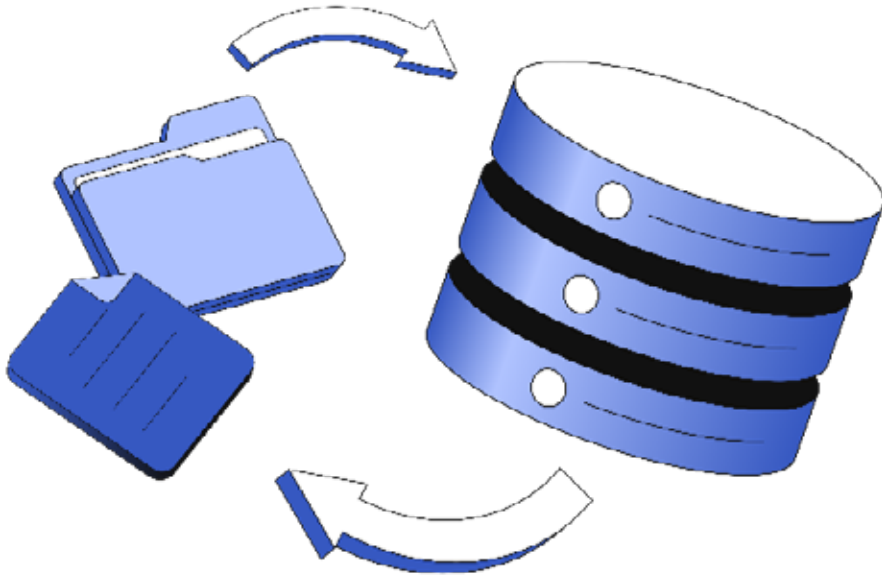


Figura 1 - Carregamento de arquivos para um banco de dados

Descrição da Imagem: é demonstrado como é realizado o carregamento de dados e arquivos em uma estrutura de armazenamento (banco de dados). Do lado esquerdo, encontram-se os arquivos que estão fora da estrutura de armazenamento, enquanto, à direita, há a representação do banco de dados em azul. Trata-se de um objeto cilíndrico que representa o espaço físico no qual os dados serão guardados no sistema computacional. Fim da descrição.

Atualmente, segundo Barboza e Freitas (2018), os bancos de dados desempenham um papel fundamental na gestão eficiente das informações, sendo uma peça vital na infraestrutura de sistemas modernos. Essa tecnologia não apenas facilita a organização e a recuperação de dados, mas também sustenta o processo de tomada de decisões e o funcionamento suave de uma ampla variedade de aplicações em setores diversos existentes no mercado.

Desempenham um papel fundamental na gestão eficiente das informações

A gestão dessa estrutura é realizada por meio de Sistemas de Gerenciamento de Banco de Dados (SGBD), como MySQL, PostgreSQL ou Oracle. Esses sistemas fornecem uma interface para criar, modificar e consultar um banco de dados, garantindo a integridade, a segurança e a consistência de todo o conteúdo nele armazenado.

O armazenamento de dados em servidores de grande porte é uma prática fundamental na era da computação em larga escala em que vivemos atualmente. Esses servidores são estruturas robustas que foram projetadas para hospedar volumes massivos de dados e atender às crescentes demandas de empresas e organizações. A infraestrutura desses servidores é caracterizada por discos rígidos de alta capacidade, unidades de estado sólido (SSDs) e sistemas RAID (*Redundant Array of Independent Disks*), proporcionando redundância e desempenho otimizado. Nesse cenário, os dados são frequentemente distribuídos em *clusters*, permitindo o acesso e a recuperação eficientes em ambientes de alta concorrência. Os sistemas de gerenciamento de armazenamento oferecem conectividade avançada e compartilhamento de dados entre servidores. Além disso, a virtualização desempenha um papel vital, ao permitir a criação de ambientes virtuais independentes, a fim de otimizar recursos de armazenamento.

A segurança é uma preocupação central nesse cenário, com servidores de grande porte implementando medidas eficazes, como criptografia, *backups* regulares e controle de acesso rigoroso, com o intuito de proteger a integridade e a confidencialidade dos dados. A escalabilidade é outra característica-chave, permitindo, assim, que os sistemas cresçam conforme a necessidade, evitando, dentre outras coisas, os gargalos de desempenho, que afetam diretamente a performance.

Assim, de acordo com Silberschatz (2020), podemos afirmar que o armazenamento de dados em servidores de grande porte representa uma abordagem avançada e estratégica para lidar com grandes volumes de informações, oferecendo desempenho robusto, segurança aprimorada e adaptabilidade às demandas dinâmicas do mundo digital.

PRINCIPAIS VANTAGENS DE SE UTILIZAR UM BANCO DE DADOS

A implementação de sistemas de banco de dados traz várias vantagens importantes para as organizações. Duas delas são a centralização e a organização dos dados, que são armazenados de forma estruturada e lógica. Isso facilita a recuperação eficiente das informações relevantes, garantindo que estejam prontamente

disponíveis quando necessárias. Além disso, proporciona uma base sólida para a gestão eficiente de dados, com organização, integridade, segurança e facilidade na manipulação das informações. Esses aspectos são essenciais para o sucesso e a inovação contínua em diversos setores. A integridade também é uma vantagem crucial, pois os sistemas de banco de dados são projetados para garantir a precisão e a consistência das informações, evitando valores duplicados indesejados ou conflitantes, o que contribui para a confiabilidade dos dados armazenados e para a tomada de decisões.

A segurança é uma preocupação central. Assim os SGBDs oferecem recursos robustos de controle de acesso, autenticação e autorização, protegendo os dados contra acessos não autorizados e/ou ameaças cibernéticas. Nesse contexto, a execução de backups regulares assegura a recuperação de dados em caso de eventos adversos.

Por fim, a facilidade de recuperação e manipulação de dados é uma característica distintiva, indicada mediante linguagens próprias, como o SQL, que permitem a realização de consultas complexas, filtragem e ordenação de dados, simplificando a obtenção das informações desejadas. Isso é especialmente valioso em ambientes empresariais, em que a agilidade na manipulação de dados é crucial. Além disso, um sistema de banco de dados proporciona a escalabilidade e a adaptação, podendo lidar com volumes crescentes de dados à medida que uma organização/instituição expande as operações. A computação em nuvem aprimorou ainda mais essa escalabilidade, permitindo o crescimento flexível dos recursos de armazenamento.

A expressão "banco de dados" diz respeito a um conjunto de dados organizado e estruturado para que possa ser facilmente acessado, gerenciado e atualizado. Já uma "base de dados" é um termo mais geral que pode se referir a qualquer coleção de dados, independentemente de como ela é organizada ou acessada. Não é errado dizer que ambos são iguais, ou seja, são sinônimos.

Contudo, nem tudo são flores. Assim como tudo na vida, existem desvantagens ao se utilizar um sistema de banco de dados. Dentre elas, podemos destacar a complexidade e o custo de implementação. Estabelecer e manter esse sistema de forma robusta, muitas vezes, são ações que requerem investimentos significativos em hardware, software,

treinamento e manutenção contínua. Já a complexidade de configuração pode vir a tornar a implementação inicial um desafio de certa forma considerável.

Há, também, a dependência de tecnologia. Uma operação eficaz de um sistema de banco de dados, muitas vezes, exige uma especialização técnica para que possa ser devidamente realizada. Isso pode resultar em uma dependência contínua de profissionais altamente qualificados, aumentando, assim, os custos operacionais e tornando as organizações mais vulneráveis à escassez de habilidades específicas.

A possibilidade de falhas e/ou interrupções de serviços também é uma preocupação constante, tendo em vista que todo e qualquer sistema computacional está sujeito a falhas técnicas, *bugs* ou até mesmo ataques cibernéticos externos, o que pode resultar em interrupções significativas nas operações. A recuperação dessas falhas pode exigir uma grande demanda de tempo e recursos substanciais.

A rigidez e a dificuldade de adaptação são desafios comumente enfrentados por algumas organizações, pois um sistema de banco de dados é projetado com estruturas específicas, devendo respeitar as regras de negócio. Logo, a necessidade de se realizar alterações significativas pode ser uma tarefa complexa. Dentre essas tarefas, é possível destacar a migração, um processo que é costumeiramente muito oneroso e custoso de ser executado. Tudo isso pode representar problemas em ambientes computacionais dinâmicos, que exigem flexibilidade e adaptação rápidas.

Outras desvantagens são a privacidade e a segurança de dados, especialmente em um cenário no qual os ataques cibernéticos são uma ameaça constante. Apesar dos consideráveis avanços em relação às medidas de segurança, não existe, atualmente, nenhum sistema que esteja completamente imune a violações de dados, o que pode resultar em sérias consequências para a reputação e a confiança de uma organização.

Embora um sistema de banco de dados proporcione inúmeras vantagens, é de suma importância que qualquer profissional da área considere essas desvantagens ao decidir implementá-lo.

Segundo Machado (2020), uma abordagem equilibrada envolve avaliar cuidadosamente as necessidades específicas da organização e ponderar os benefícios em relação às possíveis limitações.

SISTEMA DE GERENCIAMENTO DE BANCO DE DADOS (SGBD)

Também conhecida como SGBD, essa expressão diz respeito a uma infraestrutura usada para o armazenamento, a organização, a recuperação e o gerenciamento eficiente de grandes volumes de dados em ambientes computacionais. A principal função dela é fornecer uma interface entre o usuário, os aplicativos e o banco de dados, garantindo, dentre outras coisas, a integridade, a segurança e a acessibilidade dos dados armazenados.

Um SGBD opera a partir de um conjunto de softwares e programas que facilitam a criação, a atualização e a consulta ao banco de dados. Dentre as principais funcionalidades, podemos destacar as capacidades de definir a estrutura do banco, gerenciar transações, assegurar a consistência dos dados, implementar políticas de segurança e fornecer interfaces para a interação com usuários e outros aplicativos.

A evolução dos SGBDs nos remete a uma variedade de modelos, como o relacional, o orientado a objetos e o NoSQL. Cada um desses modelos atenderá a situações específicas de aplicabilidade, permitindo a flexibilidade na escolha do SGBD mais adequado para diferentes cenários e que melhor se adeque às necessidades do usuário.

Nesse contexto, podemos afirmar que um SGBD desempenha um papel fundamental na gestão eficiente de dados, ao oferecer uma plataforma unificada para a manipulação e a organização de informações, independentemente da complexidade ou do volume do banco de dados em questão.

TIPOS DE MODELOS DE DADOS

Os modelos de dados são estruturas que definem como as informações são organizadas, armazenadas e manipuladas em sistemas de banco de dados. Cada modelo apresenta características distintas, adequadas para diferentes tipos de dados e/ou requisitos de aplicação.

Vamos explorar alguns dos principais tipos de modelos de dados:

MODELO RELACIONAL

É um dos mais utilizados do mercado atual. Mesmo sendo idealizado nos anos de 1970, é extremamente eficaz e estabelece relações entre tabelas bidimensionais. Cada tabela possui linhas (registros) e colunas (atributos), e apresenta as relações que são estabelecidas por meio de chaves primárias e estrangeiras. Linguagens, como SQL, são comumente empregadas para consultas.

MODELO HIERÁRQUICO

Os dados são organizados em uma estrutura de árvore, com um nó raiz e nós secundários. Cada nó pode ter vários filhos, criando, assim, uma hierarquia. Esse modelo foi popularizado por sistemas de banco de dados mais antigos, porém a complexidade hierárquica acaba limitando a aplicabilidade dele.

MODELO DE REDE

Similar ao hierárquico, este modelo adiciona a capacidade de um nó ter múltiplos pais. Isso permite a organização dos dados de forma mais flexível, superando algumas limitações do modelo hierárquico. Apesar disso, a complexidade dele pode tornar o desenvolvimento e a manutenção mais desafiadores.

MODELO ORIENTADO A DOCUMENTOS

Comumente associado a banco de dados NoSQL, este modelo armazena os dados em documentos semelhantes ao JSON ou BSON. Cada documento pode conter dados complexos e hierárquicos, sendo eficaz para lidar com estruturas dinâmicas, cujo volume de informações se altera frequentemente.

MODELO DE FAMÍLIA DE COLUNAS

É um modelo NoSQL que organiza os dados em colunas, ao contrário de linhas, sendo muito eficiente para consultas que envolvem conjuntos de grandes massas de dados, sendo normalmente utilizado em sistemas distribuídos.

MODELO DE GRAFOS

Os dados são representados a partir de nós e arestas, permitindo uma visualização direta das relações existentes, sendo extremamente útil quando houver a necessidade de representar redes complexas.

MODELO DE OBJETOS

Representa os dados como objetos, semelhantes aos utilizados diretamente na programação orientada a objetos. Segundo Silberschatz (2020), essa abordagem permite uma representação mais fiel do mundo real e é comumente utilizada em sistemas orientados a objetos e mapeamento objeto-relacional.

Cada modelo de dados possui vantagens e limitações. Por isso, é importante escolher o que melhor se adapta às necessidades específicas do seu projeto. Segundo Pichetti, Vida e Cortes (2021), os avanços tecnológicos contínuos estão impulsionando a inovação em modelos de dados e fornecendo capacidades cada vez mais especializadas para atender aos diversos requisitos atuais.

Modelo Relacional

Trata-se de uma abordagem estruturada largamente representada para organizar e representar informações em bancos de dados, tendo sido desenvolvida por Edgar Codd na década de 1970 e se tornou a base para o modelo relacional utilizado por SGBDs, como MySQL, PostgreSQL, Oracle Database e Microsoft SQL Server.

O Sistema de Gerenciamento de Banco de Dados Relacional (SGBD-R) é um modelo que segue os princípios propostos por Codd: os dados são representados em tabelas bidimensionais, devidamente relacionadas entre si, sendo compostas por linhas (registros) e colunas (atributos). Cada uma das tabelas existentes no banco de dados possui uma chave primária única para identificação exclusiva dos registros, e as relações são estabelecidas por meio de chaves estrangeiras.

**APROFUNDANDO**

Edgar Frank Codd ou simplesmente Ted Codd foi um matemático e cientista da computação inglês que, enquanto trabalhava para a International Business Machines (IBM), criou, em meados da década de 1960, o modelo relacional para o gerenciamento de banco de dados que é largamente utilizado até os dias atuais. Ele também realizou outras contribuições valiosas para a computação, como o processamento analítico de dados, que é fundamental para conceitos, como Data Mining e Data Warehouse.

A principal vantagem de um SGBD-R é a capacidade dele de garantir a integridade referencial dos dados, mantendo a consistência e a precisão das informações armazenadas.

Um exemplo é o MySQL, um sistema de código aberto (*open source*) amplamente utilizado no mercado e conhecido pela confiabilidade e desempenho em ambientes web. O SQL Server é outra solução popular integrada ao ecossistema Microsoft, oferecendo recursos avançados para ambientes corporativos.

De acordo com Elmasri e Navathe (2018), a linguagem SQL (Structured Query Language) é a base de interação dos SGBDs relacionais, permitindo a execução de consultas complexas, atualizações e manipulações de dados em tempo real. Um de seus recursos mais difundidos é a capacidade de realizar operações JOIN, combinando dados de diferentes tabelas do banco, otimizando o resultado das consultas.

Além disso, o modelo relacional proporciona flexibilidade na expansão do banco de dados sem a necessidade de reestruturação significativa. No entanto, a rigidez em situações em que os dados não estruturados precisam ser armazenados, normalmente, torna-se uma limitação, levando ao surgimento de modelos, como o NoSQL, para atender a esse tipo de demanda.

Outro conceito extremamente relevante no modelo relacional é a normalização. Trata-se de um processo que visa aprimorar a estrutura e a eficiência do armazenamento de dados, tendo como objetivo central a redução da redundância e da duplicidade de informações, promovendo, assim, a coesão e a integridade do banco de dados. Esse processo é alcançado por meio da organização das tabelas, a fim de minimizar as dependências funcionais e eliminar as anomalias estruturais. O resultado é uma estrutura mais compacta, fácil de se manter e menos suscetível a inconsistências, contribuindo para um sistema mais robusto e eficiente.

A normalização, segundo Silberschatz (2020), segue uma série de formas, sendo que cada uma aborda aspectos específicos das relações entre as entidades e os atributos, garantindo que a estrutura do banco de dados seja livre de ambiguidades e redundâncias desnecessárias. Ao aplicar esse processo, os projetistas do banco buscam otimizar a organização das informações, contribuindo para um sistema mais robusto, eficiente e resistente a anomalias durante as operações de manipulação de dados.

Outros exemplos de SGBD-Rs são o PostgreSQL, que é uma opção de código aberto reconhecida por apresentar uma conformidade excelente com os padrões SQL, e o Oracle Database, uma escolha robusta normalmente voltada para organizações que exigem alta escalabilidade e recursos avançados de gerenciamento de dados.

Por fim, um SGBD relacional oferece uma estrutura sólida para a gestão de dados, sendo uma escolha altamente confiável para administrar uma variedade de aplicações, desde soluções voltadas para ambiente web até sistemas corporativos mais complexos e robustos.



EU INDICO

Estudante, eu te convido a assistir ao vídeo *Ted Codd, o pai dos bancos de dados relacionais*, que aborda, com riqueza de detalhes, a biografia e a história desse grande profissional e a imensa contribuição dele para a computação, que é possível ser identificada até os dias de hoje. Não perca essa oportunidade de conhecer uma das grandes celebridades da Tecnologia da Informação!

link: <https://www.youtube.com/watch?v=Bso55VPsLUc>

Modelo Não-Relacional

Um Sistema de Gerenciamento de Banco de Dados NoSQL (Not Only SQL) representa uma abordagem alternativa ao modelo relacional tradicional, oferecendo flexibilidade para lidar com grandes volumes de dados não estruturados ou semiestruturados.

Segundo Alves (2021), a principal característica desse modelo é a capacidade dele de lidar com esquemas dinâmicos, permitindo inserções de dados que não tenham uma estrutura predefinida, o que é particularmente útil em ambientes em que os requisitos são fluidos, como as redes sociais.

**ZOOM NO CONHECIMENTO**

MongoDB é um tipo de SGBD orientado a documentos que armazena dados em formato BSON ou Binary JSON, o que permite a representação de informações complexas em documentos específicos, facilitando a manipulação e a consulta de dados hierárquicos.

Outra opção bastante popular é o Apache Cassandra, um SGBD distribuído altamente escalável projetado para administrar volumes massivos de dados armazenados em clusters de servidores.

A escalabilidade horizontal é uma característica marcante dos SGBDs NoSQL, permitindo a distribuição de dados em vários servidores para suportar cargas de trabalho intensas e em grande número.

No entanto, segundo Pichetti, Vida e Cortes (2021), essa flexibilidade pode resultar em uma consistência eventual, ao contrário de ser imediata, uma vez que essa consistência pode ser difícil de ser garantida em sistemas distribuídos de grande escala.

Embora os SGBDs NoSQL sejam ideais situações específicas, como análise de Big Data, gerenciamento de conteúdo web e aplicações em tempo real, é importante avaliar cuidadosamente as necessidades do projeto antes de optar por esse modelo, considerando alguns aspectos, como a complexidade das consultas e a consistência requerida.

Diferenças entre Oracle Database e MySQL

O Oracle Database é um SGBD-R amplamente utilizado em ambientes corporativos devido à escalabilidade, à confiabilidade e a uma ampla gama de recursos avançados.

Dentre as principais características, podemos destacar:

ESCALABILIDADE

capacidade de realizar escalabilidade vertical e horizontal, permitindo gerenciar grandes volumes de dados e ambientes de missão crítica.

SEGURANÇA AVANÇADA

há recursos robustos no quesito segurança, incluindo algoritmos de criptografia, auditoria de recursos e controle de acesso granular.

RECUPERAÇÃO DE DESASTRES

inclui soluções de recuperação, como o Oracle Data Guard, que permite a replicação de dados para garantir a continuidade dos negócios.

O MySQL é um SGBD-R de código aberto amplamente utilizado em soluções para ambientes web, mas é facilmente aplicável em outros tipos de sistemas.

Dentre as principais características, podemos destacar:

CÓDIGO ABERTO

esse recurso torna o MySQL mais acessível e popular em comunidades de desenvolvimento de software, já que garante que seja executado em diferentes plataformas e aplicações, facilitando a integração com tecnologias diversas.

DESEMPENHO RÁPIDO

otimizado para operações de leitura e gravação rápidas, sendo uma escolha comum para aplicativos web e sistemas de gerenciamento de conteúdo.

COMUNIDADE ATIVA

a comunidade de desenvolvedores MySQL é ativa, proporcionando suporte e recursos adicionais.

Considerando as diferenças entre o Oracle Database e o MySQL, podemos destacar:

- **Licenciamento:** o Oracle Database é um produto comercial com licenciamento associado, ou seja, é pago. Já o MySQL, por ser de código aberto, é gratuito para uso comunitário, mas também oferece versões comerciais com suporte pago.
- **Escalabilidade e complexidade:** o Oracle Database é mais robusto em termos de escalabilidade e suporta ambientes complexos de empresas. O MySQL, por sua vez, é mais leve e frequentemente escolhido para ambientes menores e aplicações web.
- **Recursos avançados:** o Oracle Database oferece uma ampla gama de recursos avançados, como particionamento de disco, indexação de alto nível e suporte a objetos. Já o MySQL, embora apresente recursos poderosos, é menos robusto para esses tipos de recurso.
- **Suporte e comunidade:** o Oracle Database apresenta um suporte comercial extensivo, oferecido pela empresa que o administra. O MySQL, em contrapartida, por ser *open source*, possui uma comunidade bastante ativa de desenvolvedores, com suporte gratuito a partir de fóruns e documentação on-line.
- **Propósito e casos de uso:** o Oracle Database é frequentemente escolhido para administrar projetos voltados para ambientes empresariais complexos e de missão crítica. Já o MySQL é bastante popular em aplicativos web, sistemas de gerenciamento de conteúdo e ambientes de pequenas e médias empresas.

Em uma análise de ambos SGBD-Rs, fica evidente que cada um apresenta méritos e são escolhidos com base nas necessidades específicas do projeto, no orçamento e nos requisitos de desempenho.

Transações

São elementos cruciais em um banco de dados do tipo relacional, garantindo a consistência, a integridade e a confiabilidade das operações realizadas sobre os dados. Uma transação representa uma unidade lógica de trabalho que pode envolver uma ou várias operações do banco de dados.

Uma transação deve respeitar ao princípio ACID. Nele, quatro conceitos básicos são fundamentais para o funcionamento do SGBD-R em sua essência. Esse princípio é composto por:

ATOMICIDADE

garante que a transação seja tratada como uma operação única e indivisível. Todas as partes são executadas com sucesso ou nenhuma delas é executada. Se ocorrer uma falha em qualquer ponto da transação, todas as alterações realizadas até esse momento são revertidas.

CONSISTÊNCIA

garante que a execução da transação não possa violar as regras de integridade do banco de dados.

ISOLAMENTO

garante que os efeitos da transação não sejam visíveis para outras transações até que a primeira esteja concluída. Isso evita conflitos e garante que sejam independentes umas das outras durante a execução.

DURABILIDADE

garante que, uma vez que a transação seja concluída com sucesso, as alterações dela permaneçam permanentes, mesmo em caso de falhas do sistema. Isso é geralmente alcançado por meio de técnicas, como logs de transações e backups.

Um SGBD fornece mecanismos para controlar transações, incluindo instruções em linguagem SQL, como COMMIT (para confirmar uma execução bem-sucedida) e ROLLBACK (para desfazer uma execução em caso de falha). Além disso, o sistema de banco de dados mantém registros de *logs* para rastrear as operações realizadas durante uma transação.

Segundo Silberschatz (2020), é correto afirmar que uma transação é um elemento fundamental em aplicações empresariais em que a consistência dos dados é crítica. Em sistemas bancários, sistemas de reservas e outros ambientes de transação intensiva, a correta implementação do princípio ACID é essencial para garantir a confiabilidade e a precisão dos dados.

CONSIDERE UMA TRANSAÇÃO SIMPLES DE TRANSFERÊNCIA DE FUNDOS ENTRE DUAS CONTAS BANCÁRIAS, O QUE PERMITE A INCLUSÃO DE ETAPAS DISTINTAS QUE PRECISAM SER TRATADAS ADEQUADAMENTE PARA GARANTIR O DEVIDO FUNCIONAMENTO:

Dedução do valor da conta de origem.

Adição do mesmo valor à conta de destino.

As transações desempenham um papel vital na gestão de dados em um SGBD-R, proporcionando uma estrutura robusta para garantir a integridade e a confiabilidade das operações existentes.

As vantagens de se utilizar um SGBD

Um SGBD-R oferece diversas vantagens fundamentais no gerenciamento eficiente e na organização estruturada da informação. Segundo Elmasri e Navathe (2018), ao implementar um modelo relacional, os dados são organizados em tabelas para fornecer uma estrutura clara e compreensível. A principal vantagem reside no fato de que as chaves podem ser utilizadas para estabelecer relacionamentos entre diferentes conjuntos de dados, garantindo integridade e consistência. Além disso, o modelo relacional facilita consultas complexas e permite extrair os dados necessários de maneira direcionada. Modularidade e extensibilidade são recursos essenciais que permitem expandir e adaptar-se às mudanças nos requisitos.

Dentre as vantagens existentes nesse modelo, podemos destacar:

- **Organização estruturada:** o SGBD oferece uma estrutura organizada para armazenar e recuperar dados. A informação é categorizada em tabelas e relacionamentos, proporcionando uma abordagem lógica e estruturada.
- **Consistência dos dados:** o SGBD garante que as regras de integridade referencial sejam devidamente seguidas. Isso evita redundâncias e inconsistências, proporcionando um ambiente confiável.
- **Acesso rápido e eficiente:** o acesso aos dados é otimizado a partir de índices, consultas e técnicas de otimização. Isso resulta em tempos de resposta mais rápidos, o que é fundamental para aplicações que exigem recuperação eficiente de informações.
- **Segurança avançada:** o SGBD oferece mecanismos robustos de segurança, permitindo o controle de acesso granular aos dados. Senhas, permissões e criptografia são implementados para proteger informações sensíveis.
- **Recuperação de desastres:** funcionalidades, como *backups* regulares e técnicas de replicação, garantem a recuperação de dados, em caso de falhas no sistema, erro humano ou eventos catastróficos.
- **Integridade Referencial:** o SGBD suporta o princípio da integridade referencial, garantindo que as relações entre as tabelas sejam mantidas. Isso impede a existência de registros órfãos e garante uma visão coerente dos dados.
- **Escalabilidade e desempenho:** um SGBD é projetado para administrar grandes volumes de dados e escalabilidade, oferecendo recursos, como particionamento e indexação, para otimizar o desempenho, independentemente do tamanho do conjunto de dados.
- **Suporte a transações:** o SGBD implementa transações seguindo o princípio ACID (Atomicidade, Consistência, Isolamento e Durabilidade), garantindo, assim, que operações sejam concluídas com sucesso ou revertidas em caso de falha.
- **Manutenção facilitada:** a manutenção de dados é simplificada com recursos, como atualizações em lote, exclusões em cascata e a capacidade de modificar a estrutura do banco de dados sem afetar drasticamente as aplicações.
- **Suporte a consultas complexas:** um SGBD suporta consultas complexas a partir de instruções realizadas em linguagem SQL, permitindo a extração de informações específicas e a realização de análises detalhadas sobre os dados armazenados.
- **Compartilhamento de Dados:** diversas aplicações podem acessar e compartilhar dados de um único repositório, facilitando a colaboração e a consistência de informações voltadas para ambientes corporativos.

A utilização de um SGBD em um projeto computacional proporciona uma série de benefícios essenciais para garantir a eficiência, a segurança e a confiabilidade na gestão de dados em aplicações diversas. Essas vantagens contribuem para um ambiente de desenvolvimento e operação mais robusto e eficaz.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Os profissionais de tecnologia da informação enfrentam uma era dinâmica que é caracterizada por constantes desafios referentes ao gerenciamento de banco de dados.

Nesse ritmo acelerado da inovação tecnológica, a aplicação prática dos conceitos aprendidos nas instituições de ensino se tornou crítica para enfrentar esses desafios.

1. Evolução Tecnológica:

A constante evolução das tecnologias de banco de dados, como NoSQL, Big Data e soluções em nuvem, exige que os profissionais estejam atualizados. A aplicação prática envolve a adaptação a novos paradigmas e a escolha de soluções que atendam a demandas específicas.

2. Segurança da Informação:

O aumento das ameaças cibernéticas tornou a segurança da informação uma prioridade. Profissionais de administração de banco de dados precisam aplicar práticas robustas de segurança, implementando criptografia, auditorias e mecanismos avançados de controle de acesso para proteger dados sensíveis de alunos e funcionários.

3. Integração de Dados:

A integração de dados provenientes de diversas fontes, como sistemas acadêmicos, financeiros e de recursos humanos, é um desafio constante. A aplicação prática envolve a implementação de estratégias eficientes de integração para garantir consistência e disponibilidade dos dados.

4. Escalabilidade e Desempenho:

O crescimento exponencial da quantidade de dados em instituições educacionais requer sistemas de banco de dados escaláveis e de alto desempenho. Profissionais precisam aplicar técnicas, como particionamento, indexação adequada e ajustes de consultas, para garantir que o banco atenda às demandas crescentes de usuários e aplicações.

5. Governança de Dados:

A governança eficaz de dados é essencial para garantir a qualidade e a conformidade das informações. A aplicação prática envolve a criação e a implementação de políticas, procedimentos e processos que assegurem a integridade desses dados, além do cumprimento de regulamentações e padrões educacionais.

Nesse cenário, os profissionais de TI especializados em administração de banco de dados enfrentam desafios dinâmicos que exigem uma aplicação prática e adaptabilidade contínua. Isso se traduz em soluções inovadoras e eficientes que garantem a gestão adequada dos dados, contribuindo para o sucesso e a excelência educacional.

AUTOATIVIDADE

1. Qual é a principal função do Dicionário de Dados em um Sistema de Gerenciamento de Banco de Dados (SGBD)?
 - a) Armazenar dados temporários.
 - b) Catalogar e descrever os metadados do banco de dados.
 - c) Realizar consultas complexas.
 - d) Gerar relatórios automáticos.
 - e) Fornecer interfaces gráficas para usuários

2. O que é a normalização em um contexto de banco de dados relacional?
 - a) Um processo de otimização de consultas.
 - b) A adição de redundâncias aos dados.
 - c) Um método para criar backups regulares.
 - d) A organização de dados para reduzir a redundância e melhorar a integridade.
 - e) Uma técnica para criar partições em grandes conjuntos de dados.

3. O que representa a cardinalidade em um modelo Entidade-Relacionamento (ER)?
 - a) O número total de entidades em um banco de dados.
 - b) A quantidade de registros em uma tabela. A relação entre chaves primárias e estrangeiras.
 - c) O grau de complexidade das consultas SQL.
 - d) A quantidade máxima de atributos em uma entidade.
 - e) A formatação do banco de dados para consulta em linguagem SQL.

REFERÊNCIAS

ALVES, W. P. **Banco de dados**: teoria e desenvolvimento. São Paulo: Érica, 2021.

BARBOZA, F. F. M.; FREITAS, P. H. C. **Modelagem e desenvolvimento de banco de dados**. Porto Alegre: SAGAH, 2018.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de bancos de dados**. São Paulo: Pearson Education do Brasil, 2018.

MACHADO, F. N. R. **Banco de dados**: projeto e implementação. São Paulo: Érica, 2020.

PICHETTI, R. F.; VIDA, E. da S.; CORTES, V. S. M. P. **Banco de dados**. Porto Alegre: SAGAH, 2021.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

GABARITO

1. Opção B.

Um dicionário de dados armazena informações sobre a estrutura e a definição dos dados no banco de dados, ou seja, os metadados, descrevendo as tabelas, os atributos, os relacionamentos, entre outros.

2. Opção D.

A normalização é o processo de organização de dados em um banco de dados relacional para minimizar a redundância e melhorar a integridade referencial.

3. Opção C.

A cardinalidade em um modelo Entidade-Relacionamento indica a relação entre as chaves primárias e estrangeiras nas entidades, ou seja, quantos registros de uma entidade podem se relacionar com outra entidade.



TEMA DE APRENDIZAGEM 2

MODELAGEM DE DADOS

MINHAS METAS

- Conhecer os conceitos básicos dos tipos de modelagem.
- Entender o que é Modelo Entidade-Relacionamento (MER).
- Apresentar os elementos que compõem um Diagrama Entidade-Relacionamento (DER).
- Conceituar a normalização de dados.
- Otimizar a estrutura de um banco de dados para garantir desempenho e performance eficientes.
- Criar esquemas de banco de dados para situações do mundo real.
- Criar modelos que representem eficientemente a realidade que o banco de dados está destinado a retratar.

INICIE SUA JORNADA

Bem-vindo à fascinante jornada pela modelagem de banco de dados! Prepare-se para embarcar em um universo de dados estruturados, em que a lógica relacional é a chave para desvendar a complexidade das informações. Nessa jornada, você terá a oportunidade de explorar as entranhas dos relacionamentos, das tabelas e das chaves que formam a espinha dorsal de sistemas de informação eficientes.

Durante o nosso caminho, vamos desvendar os mistérios da normalização, aprendendo a organizar e a otimizar as informações para alcançar um design robusto. Você será guiado por conceitos práticos e desafiadores, desenvolvendo habilidades cruciais para esculpir bancos de dados eficazes. Lembre-se: cada linha de código que você escreve é uma peça do quebra-cabeça que molda o futuro da gestão de dados. Sinta a empolgação e prepare-se para criar estruturas que serão a base sólida para inovações e *insights* valiosos.

Então, pronto para começar? Aperte os cintos, mergulhe no mundo da modelagem de banco de dados relacional e descubra como transformar dados em conhecimento! A sua jornada começa agora.



PLAY NO CONHECIMENTO

Seja bem-vindo(a) ao Desbravando Dados, o podcast que mergulha na complexa e fascinante arte da modelagem de dados na era da informação. Exploraremos os fundamentos e as estratégias que moldam a criação de estruturas de dados sólidas, essenciais para o coração de qualquer sistema de informação.

Navegaremos entre os conceitos de modelagem, revelando os segredos da arquitetura inteligente que impulsiona as mais modernas tecnologias do mercado. A cada episódio, mergulharemos em temas envolventes, desde as técnicas clássicas de estruturação até as inovações mais recentes, proporcionando uma jornada educativa e inspiradora para todos os entusiastas da tecnologia.

Prepare-se para decifrar as técnicas da modelagem de dados, desbravando as nuances e descobrindo como ela se torna a espinha dorsal de sistemas inteligentes que moldam o mundo à nossa volta. Este é o Desbravando Dados, onde a arte encontra a ciência na criação da estrutura que sustenta a revolução da informação.

VAMOS RECORDAR?

O conceito de banco de dados relacional diz respeito a uma abordagem revolucionária de armazenamento que proporciona uma estrutura organizada e eficiente. Nela, a informação é cuidadosamente modelada por meio de tabelas interconectadas entre si, gerando uma arquitetura plena que reflete a lógica relacional que dá coesão e integridade aos dados nela armazenados.

Ao explorarmos os princípios do banco de dados relacional, lembraremos como as relações entre tabelas, chaves primárias e estrangeiras e a normalização desempenham papéis cruciais na construção de sistemas robustos. Trata-se de uma jornada pelos alicerces que sustentam muitas aplicações que usamos diariamente, proporcionando uma visão clara de como a estruturação inteligente dos dados é essencial para o sucesso de qualquer sistema de informação.

Para enfatizar o conceito de banco de dados relacional, indicamos o vídeo a seguir.
link: <https://www.youtube.com/watch?v=tRK4-wkTt9I>

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À MODELAGEM

A modelagem de banco de dados desempenha um papel fundamental no desenvolvimento de software, sendo o alicerce sobre o qual sistemas robustos e eficientes são construídos. Essa prática envolve a representação organizada e estruturada dos dados que serão manipulados pelo software, proporcionando uma visão clara e compreensível do ambiente de armazenamento. Ao criarem um modelo de banco de dados, os desenvolvedores podem visualizar como os dados estão inter-relacionados, o que é essencial para garantir a integridade e a consistência das informações ao longo do tempo.

Guia para o desenvolvimento de sistemas de software sólidos e adaptáveis

A importância da modelagem de banco de dados é ainda mais evidente quando se considera a necessidade de adaptação e evolução dos sistemas ao longo do ciclo de vida do software.

Segundo Elmasri e Navathe (2018), um modelo bem projetado oferece flexibilidade para incorporar mudanças nos requisitos do sistema, proporcionando uma base sólida para a implementação eficiente de funcionalidades adicionais ou modificações. Além disso, a modelagem contribui para a otimização do desempenho, permitindo que os desenvolvedores criem consultas e operações mais eficazes, resultando em sistemas mais rápidos e eficientes.

Outro aspecto crucial da modelagem de banco de dados é a garantia da segurança e da integridade dos dados. Ao definirem relações, chaves primárias e estrangeiras, restrições e outros elementos, os modelos asseguram que apenas informações válidas e consistentes sejam armazenadas. Essa abordagem não apenas evita a corrupção de dados, mas também contribui para a confiabilidade e a confidencialidade das informações.

Segundo Silberschatz (2020), a modelagem de banco de dados é uma prática essencial que serve como guia para o desenvolvimento de sistemas de software sólidos e adaptáveis. A aplicação diligente dela não apenas facilita a criação de soluções eficientes, mas também contribui para a construção de sistemas que podem evoluir com as demandas em constante mudança no mundo da tecnologia.

Por que modelar?

É uma etapa fundamental que não deve ser negligenciada pelos profissionais de Tecnologia da Informação (TI). Essa prática envolve a criação de representações abstratas e estruturadas dos dados que serão armazenados e manipulados por um sistema. Esses modelos oferecem uma visão organizada e compreensível das relações e das interações entre os diferentes elementos de dados, permitindo que os desenvolvedores compreendam e planejem a estrutura do banco de dados.

A importância da modelagem de banco de dados reside na capacidade dela de proporcionar uma base sólida para o desenvolvimento de sistemas eficientes e sustentáveis. Ao criarem um modelo preciso, os desenvolvedores podem identificar requisitos, relacionamentos e restrições essenciais, resultando em uma implementação mais eficaz.

Segundo Pichetti, Vida e Cortes (2021), isso é particularmente vital para sistemas complexos, nos quais a compreensão clara dos dados é crucial para o funcionamento adequado do software.

Além disso, a modelagem de banco de dados contribui para a **flexibilidade e a escalabilidade do sistema**. Um modelo bem elaborado permite a adaptação e a expansão do sistema à medida que os requisitos evoluem, garantindo que o software seja capaz de crescer e se ajustar às mudanças nas demandas do usuário ou do mercado. Isso não apenas aumenta a longevidade do sistema, mas também reduz a necessidade de reestruturação significativa no futuro.

Outro aspecto crucial é a **segurança dos dados**. A partir de uma modelagem adequada, é possível estabelecer restrições e chaves que garantem a integridade e a confidencialidade dos dados armazenados. Isso protege contra a inserção de dados inconsistentes ou inválidos e previne vulnerabilidades no sistema, proporcionando uma base sólida para a proteção das informações.

MODELO ENTIDADE-RELACIONAMENTO (MER)

O Modelo Entidade-Relacionamento (MER) é uma abordagem visual que representa as entidades (objetos ou conceitos) e os relacionamentos entre elas em um sistema de banco de dados relacional, fornecendo uma representação gráfica clara e concisa da estrutura e da organização do banco de dados.

A essência desse modelo é capturar a realidade do domínio de um sistema, destacando as entidades significativas e os vínculos existentes entre elas.

Nesse contexto, entidades, como "clientes" ou "produtos", são identificadas por atributos e podem ser inter-relacionadas por meio de diferentes tipos de relacionamentos, como "possui" ou "realiza". Cada entidade tem uma chave primária única, fundamental para a identificação exclusiva dos registros.

A normalização do MER é crucial para garantir **uma estrutura eficiente e sem redundâncias**. A partir desse processo, o modelo procura eliminar dependências e duplicações, resultando em uma representação organizada de dados. Ao traduzir o MER para esquemas de tabelas em um banco de dados do tipo relacional,

garante-se que a estrutura seja eficaz, flexível e capaz de acomodar alterações futuras sem perda de integridade ou desempenho.

O Modelo Entidade-Relacionamento pode ser definido, segundo Silberschatz (2020), como uma ferramenta valiosa para os profissionais de TI, oferecendo uma representação visual intuitiva dos elementos e das conexões fundamentais de um sistema, facilitando a compreensão e a implementação eficiente em um ambiente para o gerenciamento de banco de dados.

Modelo Conceitual

O MER representa uma abordagem conceitual essencial em um banco de dados do tipo relacional, oferecendo uma visão clara e concisa dos elementos fundamentais e das interações dentro de um sistema, concentrando-se nas entidades, nos relacionamentos e nos atributos, proporcionando uma compreensão robusta do domínio do problema.

As entidades, que podem ser objetos tangíveis ou conceitos abstratos, são representadas por retângulos no MER. Cada entidade possui atributos que definem as características individuais.

Em um sistema de gerenciamento de biblioteca, a entidade "livros" contém atributos, como "título", "autor" e "ano de publicação". Essas entidades são fundamentais para compreender o que está sendo modelado, sendo a base sobre a qual as informações são organizadas.

Já os relacionamentos, expressos por linhas que conectam as entidades, destacam as conexões e as dependências entre diferentes elementos do sistema. Assim, relacionamentos, como "possui", «*é amigo de*» ou «*participa de*», elucidam a maneira como as entidades interagem e se associam entre si na estrutura do banco de dados. A precisão na identificação dessas relações é crucial para uma modelagem conceitual eficaz.

No âmbito conceitual, segundo Elmasri e Navathe (2018), o MER estabelece uma ponte entre o entendimento do problema e a posterior implementação em um banco de dados, servindo como uma representação valiosa para discutir e validar requisitos com partes interessadas, garantindo uma base sólida para o desenvolvimento desse banco de dados.

Consideremos o cenário de uma aplicação de gerenciamento de escola, em que desejamos modelar as principais entidades e os respectivos relacionamentos (Tabela 1).

ENTIDADES	RELACIONAMENTOS
Alunos: atributos "Nome", "Data de Nascimento" e "Número de Matrícula". Professores: atributos "Nome", "Especialidade" e "Número de Identificação". Disciplinas: atributos "Nome" e "Código".	Matrícula: um aluno pode estar matriculado em várias disciplinas. Esse relacionamento é representado por uma linha que conecta "Alunos" e "Disciplinas" à descrição "Matriculado em". Leciona: um professor pode lecionar várias disciplinas. Esse relacionamento é representado por uma linha que conecta "Professores" e "Disciplinas" à descrição "Leciona".

Tabela 1 - Principais entidades e os respectivos relacionamentos / Fonte: os autores.

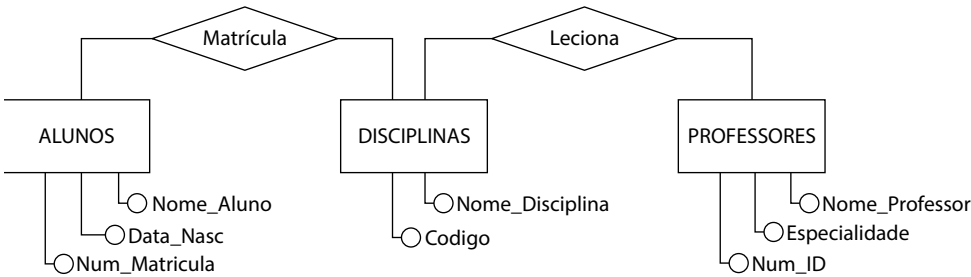


Figura 1 - Exemplo de diagrama conceitual do MER / Fonte: os autores.

Descrição da Imagem: são demonstradas as entidades "ALUNOS", "DISCIPLINAS" e "PROFESSORES", representadas em retângulos alinhados entre si. Há dois relacionamentos: "MATRÍCULA" e "LECIONA", representados por losangos alinhados acima dos retângulos. Cada entidade apresenta atributos representados a partir de pequenos círculos interligados entre si mediante linhas. Os retângulos e os losangos também estão interligados da mesma forma. Fim da descrição.

Nesse exemplo, o MER destaca visualmente como as entidades (Alunos, Professores e Disciplinas) estão interligadas por meio de relacionamentos, fornecendo uma compreensão clara das dinâmicas da aplicação. A modelagem conceitual com MER é crucial para alinhar a visão do sistema entre os desenvolvedores e as partes interessadas antes da implementação em um banco de dados relacional.

Modelo Lógico

O modelo lógico do MER é a fase que transforma as abstrações conceituais em estruturas lógicas que serão a base principal para a criação física de um banco de dados do tipo relacional. Nessa etapa, as entidades, os relacionamentos e os atributos são convertidos em tabelas, campos, chaves (primárias e estrangeiras) e cardinalidade, todos conceitos representando a arquitetura fundamental desse banco de dados.

Passos para construir o modelo lógico:

IDENTIFICAÇÃO DE ENTIDADES E ATRIBUTOS

cada entidade se torna uma tabela, com os respectivos atributos como colunas ou campos.

RELACIONAMENTOS

a interligação entre entidades é implementada por meio da relação delas a partir de campos-chave.

CHAVES PRIMÁRIAS

cada entidade deve ter um atributo que seja uma chave primária única, a qual permite a identificação de um registro como único nessa estrutura.

NORMALIZAÇÃO

este processo pode ser aplicado para garantir que o banco de dados esteja livre de redundâncias e anomalias. Isso envolve a divisão de tabelas grandes em tabelas menores e mais relacionadas.

TIPOS DE DADOS E RESTRIÇÕES

cada atributo deve ser atribuído a um tipo de dado apropriado (como texto, número e data) e podem ser impostas restrições para garantir a integridade dos dados.

O resultado é um modelo lógico que traduz as relações conceituais do MER em uma estrutura mais tangível e efetiva para a implementação de um banco de dados.

Segundo Barboza e Freitas (2018), esse modelo servirá de embasamento para a criação de *scripts* SQL que definem a estrutura desse banco em um ambiente real.



ZOOM NO CONHECIMENTO

O tipo de dados a ser utilizado na modelagem deve ser definido de acordo com o SGBD no qual o banco será implementado. Esses tipos variam entre os softwares, pois cada um apresenta particularidades.

Tipos de Chaves

Em um banco de dados relacional, a estrutura é fundamentada em conceitos, como os três tipos de chaves existentes, que são: primária, estrangeira e candidata. Além disso, há o embasamento em elementos cruciais para garantir a integridade, a consistência e o relacionamento entre as tabelas. Conheça-os a seguir.

CHAVE PRIMÁRIA

Trata-se de um campo (ou conjunto de campos em casos específicos) que identifica, de maneira única, cada registro em uma tabela, garantindo a unicidade e a integridade referenciais, sendo essencial para estabelecer relações com outras tabelas.

CHAVE ESTRANGEIRA

Trata-se de um campo que estabelece uma relação entre duas tabelas, apontando para a chave primária de outra tabela, criando um vínculo entre os registros. Essa relação é vital para manter a consistência e a integridade dos dados.

CHAVE CANDIDATA

Trata-se de um ou mais campos de uma tabela que poderiam ser escolhidos como chave primária, pois possuem a propriedade de identificação única.

A interação eficiente entre esses elementos é essencial para o sucesso de um banco de dados. A chave primária é como a identidade única de cada registro. Já a chave estrangeira estabelece relações, enquanto a chave candidata oferece opções flexíveis para garantir a integridade dos dados. Esses conceitos formam a base sólida para a estruturação eficiente de bancos de dados relacionais.

TIPOS DE RELACIONAMENTO ENTRE TABELAS

Os relacionamentos entre tabelas desempenham um papel crucial na organização e na integridade dos dados, estabelecendo conexões lógicas entre diferentes conjuntos de dados. Eles são classificados em três principais tipos: *um para um*, *um para muitos* e *muitos para muitos*.

Relacionamento Um para Um é um registro em uma tabela A e está diretamente associado a um único registro em outra tabela B. Esse tipo de relacionamento é menos comum, mas é útil em casos específicos, como quando os dados são naturalmente agrupados.

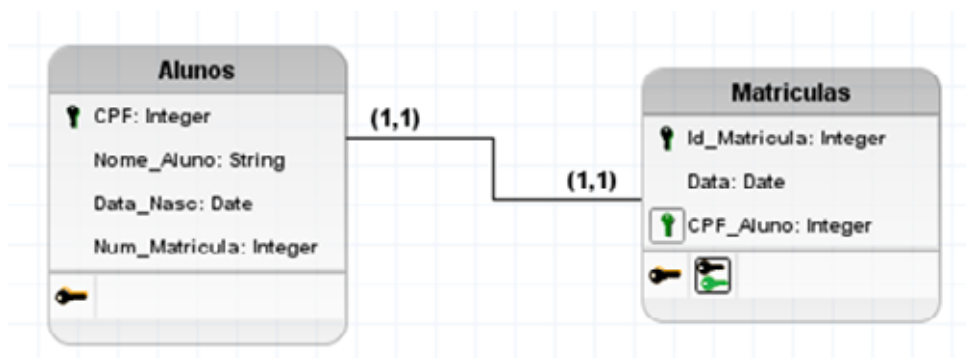


Figura 2 - Exemplo de diagrama lógico do MER para relacionamento 1 para 1 / Fonte: os autores.

Descrição da Imagem: são demonstradas as tabelas "ALUNOS" e "MATRÍCULAS", representadas em quadrados com cantos arredondados. A tabela "ALUNOS" possui quatro campos (atributos), sendo o primeiro deles a chave primária (CPF). Já a tabela "MATRÍCULAS" tem três campos (atributos), sendo o primeiro deles a chave primária (ID_MATRICULA), e o CPF_ALUNO como chave estrangeira. As duas tabelas estão relacionadas entre si a partir de uma linha que exibe as cardinalidades referentes ao relacionamento (1,1). Fim da descrição.

Relacionamento Um para Muitos é o tipo mais comum. Nesse caso, um registro em uma tabela A está associado a um ou a vários registros em outra tabela B. Entretanto, cada registro na tabela B está vinculado a apenas um registro da tabela A.

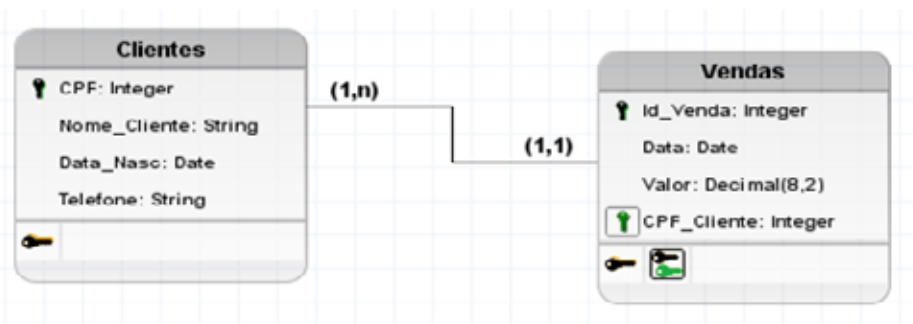


Figura 3 - Exemplo de diagrama lógico do MER para relacionamento 1 para N / Fonte: os autores.

Descrição da Imagem: são demonstradas as tabelas "CLIENTES" e "VENDAS", representadas em quadrados com cantos arredondados. A tabela "CLIENTES" tem quatro campos (atributos), sendo o primeiro deles a chave primária (CPF). Já a tabela "VENDAS" também possui quatro campos (atributos), sendo o primeiro deles a chave primária (ID_VENDA), e o CPF_CLIENTE como chave estrangeira. As duas tabelas estão relacionadas entre si a partir de uma linha que exibe apenas as cardinalidades referentes aos relacionamentos (1,1) e (1,N). Fim da descrição.

Relacionamento Muitos para Muitos ocorre quando vários registros em uma tabela A estão associados a vários registros em outra tabela B. Esse tipo de relacionamento é implementado por meio de uma tabela de junção (tabela intermediária). Por exemplo, em um banco de dados que registra alunos e cursos, a tabela de junção pode representar a relação entre alunos e cursos matriculados, em que um aluno pode estar matriculado em vários cursos, e um curso pode ter vários alunos.

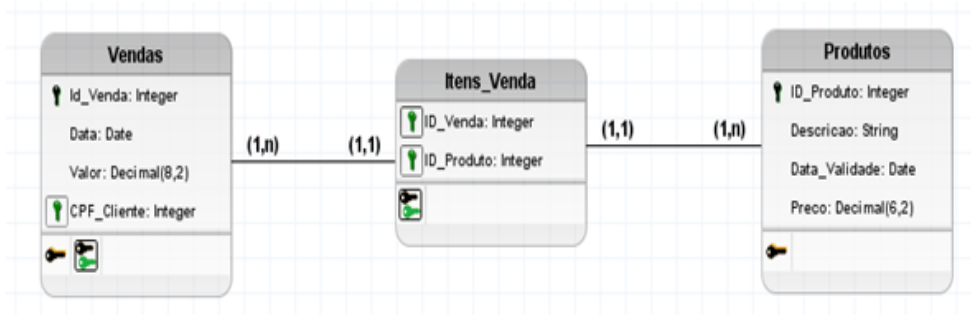


Figura 4 – Exemplo de diagrama lógico do MER para relacionamento N para N / Fonte: os autores.

Descrição da Imagem: são demonstradas as tabelas “VENDAS”, “PRODUTOS” e “ITENS_VENDA”, representadas por quadrados com cantos arredondados. A tabela “PRODUTOS” possui quatro campos (atributos), sendo o primeiro deles a chave primária (ID_PRODUTO). Já a tabela “VENDAS” também possui quatro campos (atributos), sendo o primeiro deles a chave primária (ID_VENDA), e o CPF_CLIENTE como chave estrangeira. Já a tabela “ITENS_VENDA” possui dois campos (atributos), sendo que ambos são chaves estrangeiras (ID_VENDA e ID_PRODUTO). As tabelas estão relacionadas entre si a partir de uma linha que apenas exibe as cardinalidades referentes aos relacionamentos (1,1) e (1,N). Fim da descrição.

Entender e definir adequadamente esses relacionamentos são atitudes cruciais para garantir a integridade referencial e a consistência em um banco de dados relacional. A escolha do tipo de relacionamento depende da natureza dos dados armazenados no banco e dos requisitos específicos do sistema.

Cardinalidade

Trata-se de um conceito que se refere à maneira como as tabelas estão interligadas por meio de relacionamentos. Três tipos principais de cardinalidade são comumente utilizados: *um para um*, *um para muitos* e *muitos para muitos*.

Entender e definir a cardinalidade são atitudes cruciais para o design eficiente de bancos de dados relacionais. Elas orientam como as informações estão interconectadas, garantindo a integridade referencial e contribuindo para sistemas de gerenciamento de dados robustos e eficazes.

Segundo Barboza e Freitas (2018), ao determinarem a cardinalidade, os projetistas de banco de dados podem criar estruturas que refletem com precisão os relacionamentos entre entidades, proporcionando uma base sólida para a manipulação e a recuperação de dados.

No contexto de bancos de dados relacionais, as cardinalidades mínima e máxima são conceitos que complementam a definição de relacionamentos entre tabelas. A **cardinalidade máxima** representa o número máximo de instâncias (registros) de uma entidade (tabela) que poderão se relacionar com os dados de outra tabela existente no banco. Já a **cardinalidade mínima** define o número mínimo de instâncias que devem participar de um relacionamento.

Assim, nesse contexto, os projetistas de bancos de dados garantem a integridade referencial e especificam as restrições necessárias para manter a consistência dos dados. Esses conceitos são fundamentais para o design eficiente de bancos de dados, permitindo uma representação precisa dos relacionamentos entre entidades e orientando a manipulação confiável dos dados ao longo do tempo. O entendimento detalhado desses aspectos contribui para a construção de sistemas de gerenciamento mais robustos e adaptáveis.

Considere um cenário em que está sendo projetado um banco de dados para uma aplicação de gerenciamento de biblioteca que apresenta duas entidades principais: **livros** e **autores**. Assim, será estabelecido um relacionamento entre essas duas entidades para refletir uma relação do tipo “*muitos para muitos*”, pois um livro pode ter vários autores, e um autor pode estar associado a vários livros.

**ZOOM NO CONHECIMENTO**

- Tipo de Cardinalidade (LIVROS):

Máxima: como um livro pode ter vários autores, a cardinalidade máxima seria muitos (M).

Mínima: caso todos os livros tenham, pelo menos, um autor, a cardinalidade mínima seria um (1).

- Tipo de Cardinalidade (AUTORES):

Máxima: como um autor pode estar associado a vários livros, a cardinalidade máxima seria muitos (M).

Mínima: caso for permitido que autores estejam cadastrados sem que tenham livros a eles associados, a cardinalidade mínima seria zero (0).

Dessa forma, o relacionamento entre livros e autores é definido com muito mais precisão, indicando que cada livro deve ter, pelo menos, um autor, mas também pode ter diversos autores. Simultaneamente, um autor pode estar associado a muitos livros ou a nenhum, proporcionando, assim, uma maior flexibilidade ao sistema. Essas especificações de cardinalidade são cruciais para garantir que o banco de dados represente fielmente as relações entre as entidades e para preservar a integridade referencial ao longo do tempo.

Cardinalidade Unária

Esse conceito se refere a uma relação entre elementos da mesma entidade. Por exemplo, consideremos a existência de uma entidade chamada **funcionários** em um sistema de gerenciamento de recursos humanos. Para representar a relação “**supervisor - subordinado**”, sendo que os dados estão armazenados na mesma entidade em questão, cada funcionário pode ter de zero a um supervisor. Além disso, esse supervisor também é um funcionário.

Assim, segundo Alves (2021), a cardinalidade unária permite modelar relações hierárquicas dentro da mesma entidade.

Funcionário 1: João (sem supervisor)
Funcionário 2: Maria (supervisor: João)
Funcionário 3: Pedro (supervisor: João)

Cardinalidade Ternária

Trata-se de um tipo especial de relação que envolve dados provenientes de três entidades. Por exemplo, consideremos as entidades ESTUDANTES, CURSOS e PROFESSORES em um sistema acadêmico. Para representar a relação “ESTUDANTE - INSCRITO - EM - CURSO - LECIONADO - POR - PROFESSOR”, cada associação entre estudante, curso e professor representa uma instância única dessa relação, permitindo uma modelagem mais complexa de interações entre entidades distintas.

Estudante 1: Ana
Curso 1: Matemática
Professor 1: Carlos
Ana está inscrita em Matemática, lecionada por Carlos.

Esses exemplos ilustram como os tipos de cardinalidade **unária** e **ternária** podem ser aplicadas em modelos de bancos de dados para representar relações específicas entre entidades.

Além delas, podemos destacar outros dois tipos de cardinalidade: **nula** e **binária**.

A **cardinalidade nula** se refere à possibilidade de uma entidade não possuir relacionamentos com outras entidades, indicando que uma instância específica pode existir independentemente de associações com outras. Isso permite flexibilidade na modelagem, em que certas entidades podem não ter conexões obrigatórias.

Por outro lado, a **cardinalidade binária** caracteriza um relacionamento entre duas entidades, indicando quantas instâncias de uma entidade podem estar associadas a outras entidades. Esse tipo estabelece uma relação mais simples, em que cada instância pode ser conectada a, no máximo, uma correspondente na tabela relacionada, promovendo uma estrutura mais direta e fácil de compreender no banco de dados relacional.

NORMALIZAÇÃO

É um processo fundamental para organizar eficientemente os dados e minimizar redundâncias, garantindo integridade e consistência. Esse método, proposto por Edgar F. Codd, objetiva reduzir a duplicidade e a dependência funcional nas tabelas, melhorando o desempenho e a eficácia do banco de dados a ser implementado.

Segundo Silberschatz (2020), o processo de normalização é composto por diferentes formas normais (FN), cada uma endereçando diferentes tipos de dependências e níveis diferentes de complexidades. As mais utilizadas são as denominadas Primeira Forma Normal (1FN), Segunda Forma Normal (2FN) e Terceira Forma Normal (3FN).

Para realizar a normalização, é necessário identificar chaves primárias e estrangeiras, analisar dependências funcionais e redistribuir as informações em várias tabelas. Isso resulta em tabelas mais coesas e diminui a necessidade de atualizações redundantes, evitando problemas, como a anomalia de inserção, alteração e exclusão.

Esse processo, embora essencial, requer equilíbrio, pois, se feito de maneira excessiva, pode levar a consultas complexas e desempenho reduzido. Diante disso, a decisão sobre qual forma normal alcançar depende da natureza específica dos dados e dos requisitos do sistema.

Em resumo, a normalização é uma prática essencial para garantir a eficiência e a integridade do banco de dados, sendo uma técnica valiosa para projetar esquemas mais robustos e eficientes.

Primeira Forma Normal (1FN)

A Primeira Forma Normal (1FN) é um dos princípios fundamentais da normalização em bancos de dados relacionais introduzido por Edgar F. Codd. Essa forma normal estabelece que os valores armazenados em uma tabela devem ser atômicos, ou seja, indivisíveis, eliminando a presença de valores multivalorados ou conjuntos.

Cada célula da tabela deve conter apenas um valor simples e único

Em termos práticos, isso significa que cada célula da tabela deve conter apenas um valor simples e único. Para entender melhor, consideremos uma tabela que armazena informações de alunos, em que a coluna “DISCIPLINAS” têm valores, como “**matemática, física, química**”. Em 1FN, seria realizado um ajuste para ter uma nova tabela que armazenará os dados das disciplinas e se relacionará diretamente com ALUNOS, mantendo a atomicidade dos valores de cada uma.

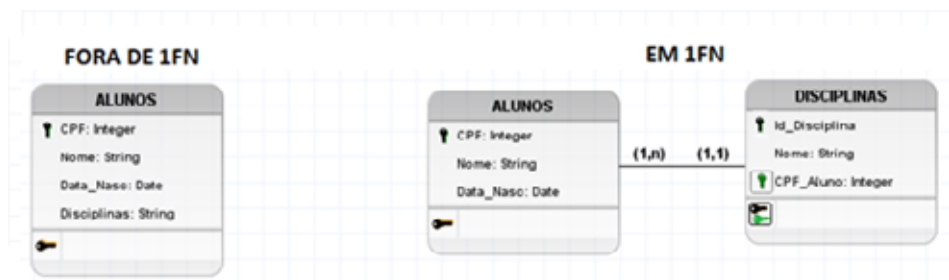


Figura 5 - Exemplo de normalização 1FN / Fonte: os autores.

Descrição da Imagem: são demonstradas duas representações: uma à esquerda, na qual a estrutura não se exhibe adequada ao princípio da primeira forma normal (1FN), e a outra à direita, que está devidamente ajustada a esse conceito. À esquerda, há uma tabela “ALUNOS” com quatro campos, sendo que o atributo “DISCIPLINA” pode apresentar situações de multivalores. Assim, ao se adequar a 1FN, esse campo foi retirado de “ALUNOS” e foi criada uma tabela chamada “DISCIPLINAS” com três atributos (ID_DISCIPLINA, NOME e CPF_ALUNO). Fim da descrição.

Vejamos um exemplo mais concreto. Suponha uma tabela chamada “Pedidos”, em que uma linha contém informações sobre um pedido e os itens associados. Em 1FN, essa tabela seria decomposta em duas: uma tabela “Pedidos” e outra “Itens do Pedido”. Cada linha da tabela de pedidos, agora, teria uma identificação única, e os itens do pedido seriam armazenados em outra tabela, referenciando a identificação do pedido original.

Em resumo, a 1FN é crucial para garantir a integridade dos dados e simplificar consultas, evitando redundâncias e ambiguidades. Ao adotar a atomicidade dos valores, cria-se uma base sólida para avançar nas formas normais subsequentes, promovendo a eficiência e a organização nos bancos de dados relacionais.

Segunda Forma Normal (2FN)

Trata-se de uma extensão do conceito de normalização em bancos de dados relacionais que busca eliminar redundâncias de dados, ao garantir cada campo em uma tabela que depende completamente da chave primária, promovendo uma estrutura mais coesa e eficiente.

Para entender a 2FN, é essencial compreender a diferença entre os conceitos de dependências totais e parciais. Uma tabela está na 2FN quando não há nenhum tipo de dependência parcial na chave primária, ou seja, cada campo depende de forma completa da totalidade da chave primária, sem depender de apenas uma parte dela.

Para entender melhor esse conceito, vamos considerar um exemplo no qual uma tabela “PEDIDOS” apresente os campos “ID_PEDIDO”, “ID_PRODUTO”, “QUANTIDADE” e “PRECO_UNITARIO”. Com a chave primária sendo “ID_PEDIDO” e “ID_PRODUTO”, o campo “QUANTIDADE” depende apenas do “ID_PEDIDO”, enquanto “PRECO_UNITARIO” dependerá somente do “ID_PRODUTO”. Assim, a estrutura não está adequada à 2FN. Para isso, seria necessário dividir essa tabela em duas, sendo: “PEDIDOS” e “PRODUTOS”.

Agora, ambas as tabelas estão na 2FN, pois cada campo depende completamente da chave primária correspondente.

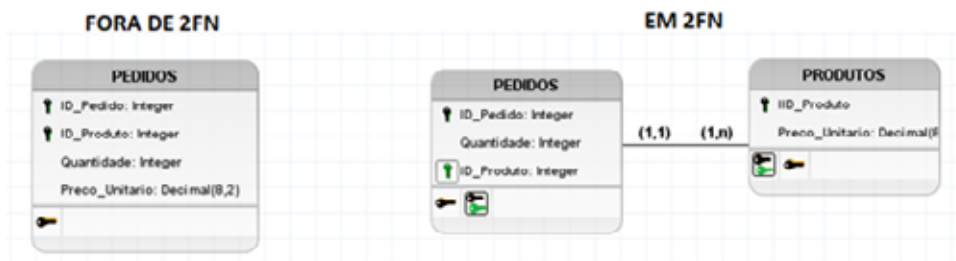


Figura 6 - Exemplo de normalização 2FN / Fonte: os autores.

Descrição da Imagem: são demonstradas duas representações em três tabelas mostradas em quadrados com cantos arredondados: uma à esquerda, na qual a estrutura não se encontra adequada ao princípio da segunda forma normal (2FN), e outra à direita, que está devidamente ajustada a esse conceito. À esquerda, há uma tabela “PEDIDOS” com quatro campos, sendo duas chaves primárias (ID_PRODUTO e ID_PEDIDO) e outros dois (QUANTIDADE e PRECO_UNITARIO). À direita, há duas tabelas (PEDIDOS e PRODUTOS), sendo a primeira delas com três campos (ID_PEDIDO, QUANTIDADE e ID_PRODUTO), já a outra com dois campos (ID_PRODUTO e PRECO_UNITARIO). Fim da descrição.

Assim, para que uma estrutura esteja em 2FN, primeiramente, ela precisa estar em 1FN e não apresentar anomalias que possam comprometer futuras consultas e/ou alterações no banco de dados após a implementação. Assim, aprimora-se a estrutura, eliminam-se dependências parciais e são reduzidas redundâncias, o que contribui para uma organização mais eficaz dos dados a serem armazenados.

Terceira Forma Normal (3FN)

Trata-se de um conceito da normalização que busca eliminar as dependências transitivas que possam existir na estrutura do banco de dados.

Em termos simples, uma **dependência transitiva** ocorre quando um campo de uma tabela depende de outro que não faz parte da chave primária, mas de um campo intermediário. Nesse contexto, a 3FN busca eliminar essas dependências, promovendo uma estrutura mais limpa e evitando ao máximo redundâncias, praticamente as eliminando.

Para entender melhor, vamos considerar o exemplo de uma tabela “FUNCIONARIOS”, com os atributos “ID_FUNCIONARIO”, “NOME”, “DEPARTAMENTO” e “LOCALIZACAO_DEPTO”. Nesse caso, fica evidenciado que o campo “LOCALIZACAO_DEPTO” depende, ao mesmo tempo, tanto de “ID_FUNCIONARIO” quanto de “DEPARTAMENTO”. Para atender ao conceito de 3FN, é necessário dividir a tabela em duas: “FUNCIONARIOS” e “DEPARTAMENTOS”, eliminando, assim, essa dependência transitiva.

Assim, em “FUNCIONARIOS”, serão mantidos os atributos “ID_FUNCIONARIO” e “NOME”. Já a nova tabela criada, chamada “DEPARTAMENTOS”, terá “NOME_DEPTO” e “LOCALIZACAO”. Agora, a estrutura do banco de dados encontra-se em 3FN, pois não há mais nenhum tipo de dependência transitiva, aumentando a integridade dos dados e a eficiência na manipulação das informações.



Figura 7 - Exemplo de normalização 3FN / Fonte: os autores.

Descrição da Imagem: são demonstradas duas representações em três tabelas dispostas em quadrados com cantos arredondados: uma à esquerda, na qual a estrutura não se encontra adequada ao princípio da terceira forma normal (3FN), e a outra à direita, que está devidamente ajustada a esse conceito. À esquerda, há uma tabela “FUNCIONARIOS” com quatro campos, sendo uma chave primária (ID_FUNCIONARIO) e outros três (NOME, DEPARTAMENTO e LOCALIZACAO_DEPTO). À direita, há duas tabelas (FUNCIONARIOS e DEPARTAMENTOS), sendo a primeira delas com três campos (ID_FUNCIONARIO, NOME e ID_DEPTO), já a outra com três campos (ID_DEPTO, NOME e LOCALIZACAO). Fim da descrição.

Assim, para que uma estrutura esteja em 2FN, primeiramente, ela precisa estar em 1FN e não apresentar dependências entre os campos das tabelas que compõem o banco de dados, visando garantir que todas as interligações estejam claras, promovendo uma estrutura livre de redundâncias e/ou dependências indesejadas.



EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Em um cenário profissional dinâmico e repleto de desafios, a habilidade de implementar banco de dados de maneira consciente e estruturada se torna um diferencial significativo para os profissionais de Tecnologia da Informação. Ao criarem um ambiente de dados robusto, eficiente e seguro, os especialistas em banco de dados desempenham um papel vital na gestão da informação, contribuindo diretamente para o sucesso e a competitividade das organizações.

A jornada começa com uma compreensão profunda dos conceitos fundamentais de modelagem de banco de dados. Os profissionais de TI devem dominar não apenas a teoria, mas também as práticas de normalização, relacionamentos e estruturação eficiente das informações. A aplicação desses conhecimentos em projetos práticos durante a formação acadêmica cria a base sólida necessária para enfrentar os desafios do mundo real.

Em um ambiente profissional, a conscientização sobre as necessidades específicas da organização é crucial. Os profissionais de TI devem ser capazes de traduzir as demandas e as expectativas dos usuários finais em modelos de dados eficientes. Antecipar crescimento, garantir a integridade e a segurança dos dados e adaptar-se a mudanças nas demandas comerciais são habilidades essenciais que definem o sucesso na implementação de bancos de dados.

A implementação consciente também envolve escolhas tecnológicas informadas. Os profissionais devem avaliar cuidadosamente as opções disponíveis, levando em consideração alguns fatores, como escalabilidade, segurança e interoperabilidade. Além disso, estar atualizado com as tendências do mercado, como a ascensão de bancos de dados do tipo NoSQL em determinados contextos, é vital para tomar decisões informadas e alinhar as estratégias de banco de dados com os objetivos organizacionais.

Em um mercado de TI que evolui muito rapidamente, os profissionais que conseguem implementar bancos de dados de maneira consciente não apenas aplicam conhecimentos técnicos, mas também incorporam uma mentalidade proativa e estratégica. Esses especialistas tornam-se arquitetos da informação, moldando o futuro digital das organizações por meio de decisões fundamentadas e da aplicação de melhores práticas em modelagem de banco de dados. O cenário profissional aguarda aqueles que estão preparados para transformar conhecimento em impacto positivo.

1. Embora vivamos em uma era de tecnologias diversas, a modelagem de banco de dados relacional ainda é uma prática relevante e necessária, contribuindo para a organização, a manutenção e a compreensão eficiente dos dados. Um dos benefícios da modelagem relacional é a garantia da integridade referencial, assegurando que as chaves estrangeiras em uma tabela estejam corretamente relacionadas às chaves primárias em outra, promovendo consistência nos dados.

Considerando a importância da modelagem de banco de dados relacional, analise as afirmativas a seguir:

- I - A modelagem de banco de dados relacional auxilia na organização dos dados, definindo a estrutura e os relacionamentos entre as tabelas.
- II - Esse processo é crucial para garantir a integridade referencial, evitando inconsistências e redundâncias nos dados armazenados.
- III - A modelagem de banco de dados relacional é uma prática obsoleta, sem relevância nos ambientes modernos de desenvolvimento de software.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

2. Em um banco de dados relacional, a normalização é um processo essencial para garantir a eficiência, a consistência e a integridade das informações, tendo, como objetivo, organizar as tabelas, de modo a reduzir redundâncias e dependências, contribuindo para a manutenção da qualidade dos dados.

No contexto da normalização de um banco de dados do tipo relacional, considere a seguinte situação: em um banco de dados, uma tabela foi projetada com os seguintes atributos: "Código do Cliente", "Nome do Cliente", "Endereço", "Telefone" e "Produtos Comprados".

No cenário apresentado, é possível identificar que essa tabela apresenta uma violação em relação à qual forma normal?

- a) Primeira Forma Normal (1FN).
- b) Segunda Forma Normal (2FN).
- c) Terceira Forma Normal (3FN).
- d) O BD já está normalizado.
- e) Não é possível normalizar nessa situação.

3. A cardinalidade em bancos de dados relacionais é um conceito essencial que define a quantidade de instâncias de uma entidade que podem estar associadas a instâncias correspondentes de outra entidade por meio de um relacionamento. Esse conceito determina a natureza e a extensão das conexões entre tabelas, indicando a quantidade de instâncias que pode se relacionar diretamente entre si dentro de um banco de dados mesmo estando fisicamente em entidades diferentes.

A respeito dos tipos de cardinalidades em bancos de dados relacionais, analise as afirmativas a seguir:

- I - Cardinalidade Nula: refere-se a uma situação em que uma entidade não tem relacionamentos com outras entidades em um banco de dados.
- II - Cardinalidade Unária: representa um relacionamento em que cada instância de uma entidade em uma tabela está associada a uma única instância da mesma entidade em outra tabela.
- III - Cardinalidade Ternária: indica um relacionamento entre três entidades. Cada instância de uma entidade em uma tabela está associada a instâncias correspondentes em outras duas tabelas.
- IV - Cardinalidade Binária: caracteriza um relacionamento entre duas entidades, em que cada instância de uma entidade em uma tabela pode estar associada a, no máximo, duas instâncias da outra entidade em outra tabela.

É correto o que se afirma em:

- a) I e IV, apenas.
- b) II e III, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) II, III e IV, apenas.

REFERÊNCIAS

ALVES, W. P. **Banco de dados**: teoria e desenvolvimento. São Paulo: Érica, 2021.

BARBOZA, F. F. M.; FREITAS, P. H. C. **Modelagem e desenvolvimento de banco de dados**. Porto Alegre: SAGAH, 2018.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

PICHETTI, R. F.; VIDA, E. da S.; CORTES, V. S. M. P. **Banco de dados**. Porto Alegre: SAGAH, 2021.

SILBERSCHATZ, A. sistema de banco de dados. Rio de Janeiro: GEN; LTC, 2020.

GABARITO

1. Opção C.

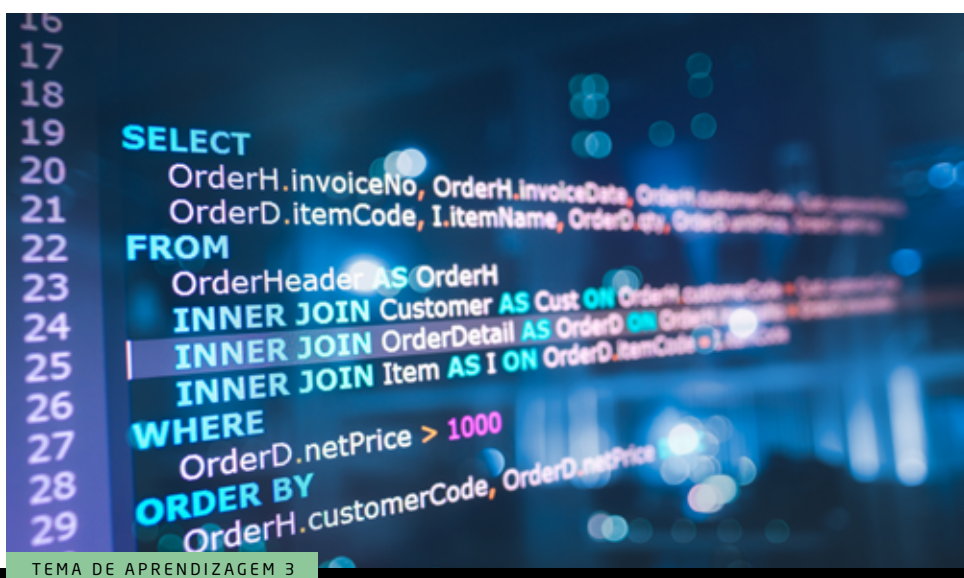
A afirmativa I está correta, pois a modelagem de banco de dados relacional é fundamental para organizar os dados, determinando a estrutura das tabelas e os relacionamentos entre elas. A afirmativa II também está correta, pois a modelagem é crucial para garantir a integridade referencial, evitando inconsistências e redundâncias nos dados. A afirmativa III está incorreta, já que a modelagem de banco de dados relacional é uma prática relevante nos dias de hoje.

2. Opção C.

Essa escolha se justifica pela presença da dependência transitiva na tabela original, indicando que existe uma relação indireta entre as colunas. A Terceira Forma Normal (3FN) visa eliminar dependências transitivas, garantindo maior eficiência e integridade na estrutura do banco de dados.

3. Opção B.

A cardinalidade unária representa um relacionamento em que cada instância de uma entidade em uma tabela está associada a uma única instância da mesma entidade em outra tabela. É um tipo de relacionamento intrínseco à própria entidade. A cardinalidade ternária indica um relacionamento entre três entidades, em que cada instância de uma entidade em uma tabela está associada a instâncias correspondentes em outras duas tabelas. Esse tipo de relacionamento envolve três entidades. A cardinalidade nula não é um tipo de cardinalidade. Refere-se à possibilidade de uma entidade não ter relacionamentos com outras entidades. Não é considerada um tipo específico de cardinalidade. A definição de cardinalidade binária está incorreta. Ela caracteriza um relacionamento entre duas entidades, em que cada instância de uma entidade em uma tabela pode estar associada a, no máximo, uma instância da outra entidade em outra tabela, não duas.



TEMA DE APRENDIZAGEM 3

LINGUAGEM SQL

MINHAS METAS

- Definir o que é SQL.
- Apresentar os comandos de definição e uso da SQL.
- Desenvolver habilidades para formular consultas SQL complexas.
- Adquirir uma compreensão sólida dos fundamentos da linguagem SQL.
- Aprender a criar e a manter bancos de dados utilizando SQL.
- Desenvolver competências em otimização de desempenho.
- Adquirir conhecimentos sobre segurança dos dados, incluindo permissões de usuário.

INICIE SUA JORNADA

Bem-vindo à jornada emocionante pelo universo da Tecnologia da Informação (TI), em que a linguagem SQL (Structured Query Language) se revela uma ferramenta essencial para moldar o futuro dos profissionais de TI. Exploraremos os fundamentos que abrirão as portas para a criação, a manipulação e a otimização de bancos de dados relacionais, a fim de te capacitar a enfrentar desafios do mundo real.

Na base de tudo, está a compreensão profunda dos fundamentos dessa linguagem. Desde as consultas básicas até as operações mais avançadas, você aprenderá a construir e a manter bancos de dados relacionais. Essa habilidade não somente permite que você recupere dados de maneira eficiente, mas também oferece uma base sólida para o desenvolvimento de sistemas de software mais robustos e abrangentes.

Ao longo desta jornada, você terá a oportunidade de desenvolver habilidades práticas essenciais. A construção de consultas complexas, a manipulação de dados de forma eficaz e a compreensão do papel crítico da linguagem SQL em alguns aspectos, como segurança e otimização de bancos de dados, serão aspectos fundamentais do seu aprendizado.

À medida que mergulhamos na linguagem SQL, é importante se ater que cada conceito dominado é um passo em direção ao enfrentamento de desafios do mundo real. Os conhecimentos adquiridos não apenas fortalecerão as suas habilidades técnicas, mas também te posicionarão como um(a) profissional de TI capaz de moldar e inovar em ambientes cada vez mais complexos. Estamos apenas começando: prepare-se para uma jornada incrível, na qual o SQL se tornará uma ferramenta indispensável em seu arsenal tecnológico.

**PLAY NO CONHECIMENTO**

Bem-vindo(a) a mais uma jornada de aprendizado e descoberta no podcast Desbravando Dados. Nesta edição especial, mergulharemos fundo na linguagem SQL, além de explorarmos o papel dela no contexto dos bancos de dados relacionais. Prepare-se para uma viagem pelo mundo das consultas, comandos e estruturas que transformam dados em informações valiosas.

A linguagem SQL é muito mais do que uma série de comandos: é uma linguagem de comunicação eficaz com os bancos de dados relacionais. Ao sintonizar este episódio, você descobrirá como expressões simples podem desencadear operações complexas, facilitando a extração de informações cruciais. Este é o momento de desmistificar a linguagem SQL e perceber como ela se torna a ponte entre você e os dados que moldam o mundo digital.

Se você está apenas começando ou já possui alguma experiência com SQL, este episódio oferecerá insights valiosos. Vamos explorar desde conceitos fundamentais até técnicas avançadas, proporcionando uma compreensão abrangente da linguagem e das aplicações práticas dela. Ao final, você estará mais preparado(a) para enfrentar os desafios e as oportunidades que a manipulação eficiente de bancos de dados relacionais oferece.

Está pronto(a) para decifrar os códigos e desvendar os segredos da linguagem SQL? Então, aperte o play e embarque conosco nesta jornada pelo universo dos bancos de dados relacionais. Aprender nunca foi tão envolvente, fascinante e prático. Sintonize-se, desbrave dados e amplie as suas habilidades no mundo da Tecnologia da Informação.

VAMOS RECORDAR?

A manipulação de um sistema de bancos de dados do tipo relacional é um conjunto de operações que permite extrair, inserir, atualizar e excluir dados em um software gerenciador conhecido como SGBD. Essa prática é realizada por meio da linguagem SQL, a qual oferece uma abordagem padronizada para interagir com as tabelas e os relacionamentos dentro dessa estrutura.

Ao manipular um banco de dados relacional, os profissionais de TI podem realizar consultas complexas para obter informações específicas, inserir novos registros, atualizar dados existentes ou excluir informações que não são mais necessárias. Essa manipulação é uma parte integral do ciclo de vida desse tipo de banco de dados, proporcionando flexibilidade e controle sobre as informações armazenadas, essenciais para atender às demandas dinâmicas das aplicações modernas.

<https://www.youtube.com/watch?v=XfO3TRvESBo>

DESENVOLVA SEU POTENCIAL

DATABASE ADMINISTRATOR (DBA)

O **Administrador de Banco de Dados (DBA)** desempenha um papel crucial no desenvolvimento, na implementação e na manutenção de sistemas de bancos de dados. A responsabilidade principal dele é garantir que o banco funcione de maneira eficiente, segura e sem interrupções. O DBA trabalha na criação e na configuração inicial do banco de dados, decidindo a estrutura, os índices e os relacionamentos entre as tabelas para atender às necessidades específicas do sistema. Essa fase é vital para a performance futura deste banco.

Além disso, segundo Alves (2021), o DBA é o responsável pela administração diária do banco de dados, monitorando o desempenho, otimizando consultas, aplicando *patches* e atualizações e gerenciando a segurança dos dados. A implementação de *backups* regulares e a recuperação em caso de falhas também estão entre as tarefas críticas do DBA para garantir a integridade e a disponibilidade contínua do sistema.

A importância do DBA no desenvolvimento de sistemas de bancos de dados é evidente na capacidade dele de otimizar o desempenho, garantir a segurança e permitir a escalabilidade à medida que o sistema evolui. Além disso, a expertise dele é fundamental na resolução de problemas, na implementação de melhores práticas e na colaboração com os desenvolvedores para garantir a integração eficiente do banco com a aplicação.



Figura 1 - O trabalho de um DBA

Descrição da Imagem: é exibido um profissional que veste uma camisa branca. Ele opera um notebook, ao fazer um trabalho que engloba telas de recursos humanos e gestão, preenchendo um formulário de pesquisa on-line, respondendo às perguntas do teste e fazendo o checklist em uma prancheta virtual. Fim da descrição.

Em resumo, o papel do DBA, conforme Machado (2020), é essencial para o sucesso de qualquer sistema de banco de dados, contribuindo diretamente para a eficácia, a estabilidade e a segurança do ambiente de dados.

A LINGUAGEM SQL

SQL é uma sigla em inglês que significa “Structured Query Language” ou, em português, “Linguagem de Busca Estruturada”. Ela teve origem nos laboratórios da International Business Machines (IBM) nos anos de 1970, como um esforço conjunto de Donald D. Chamberlin e Raymond F. Boyce.

Inicialmente chamada de SEQUEL (Structured English Query Language), foi projetada para proporcionar uma forma de manipulação eficiente e intuitiva de dados em sistemas de gerenciamento de um banco de dados do tipo relacional. Rapidamente, tornou-se uma linguagem padrão no segmento e a evolução contínua dela tem contribuído significativamente para a eficiência e a consistência no tratamento de dados.

Essa linguagem desempenha um papel central em um banco de dados relacional, atuando como uma ponte crucial entre os usuários e as informações armazenadas. O principal propósito é permitir que os usuários executem operações, como consultas, inserções, atualizações e exclusões de dados em tabelas relacionadas.

Mediante alguns comandos, como SELECT, INSERT, UPDATE e DELETE, os usuários podem interagir de maneira eficaz com o banco de dados, garantindo a integridade e a consistência de todas as informações armazenadas.

Além da função básica de manipulação de dados, segundo Silberschatz (2020), o SQL oferece recursos avançados, como a capacidade de criar, modificar e gerenciar a estrutura do banco de dados. A linguagem também suporta a definição de restrições de integridade, transações e controle de acesso, proporcionando aos desenvolvedores e administradores de banco de dados um conjunto abrangente de ferramentas para otimização e manutenção do sistema. A sintaxe clara e poderosa faz do SQL uma linguagem indispensável para todos os profissionais envolvidos no gerenciamento de bancos de dados do tipo relacional.

Essa linguagem é fundamental para a conectividade eficiente entre sistemas de software e bancos de dados relacionais. A partir das capacidades de consulta e manipulação de dados, o SQL viabiliza a integração harmoniosa entre aplicativos e o armazenamento desses dados.

Segundo Elmasri e Navathe (2018), a instrução SELECT, por exemplo, permite que o software recupere dados específicos de tabelas do banco de dados, possibilitando a exibição ou a utilização dessas informações em interfaces de usuário ou em processos de lógica de negócios.



Figura 2 - O trabalho de um administrador de banco de dados

Descrição da Imagem: são exibidos os conceitos com os quais um administrador de banco de dados lida durante o dia a dia de trabalho. À esquerda, há duas engrenagens que representam as ferramentas (softwares) utilizadas. Ao lado, encontra-se um componente que representa um servidor de banco de dados e, acima dele, uma representação de pasta com a escrita "SQL", que demonstra a conectividade das ferramentas com esse servidor. Ainda, são representadas, à direita, interfaces gráficas que determinam as telas utilizadas no computador. Por fim, é exibido um profissional que veste uma camisa vermelha, tem barba e cabelos escuros. Ele opera um laptop, a fim de representar o administrador do banco de dados. Fim da descrição.

A aplicação prática dessa linguagem na conectividade entre sistemas de software e bancos de dados relacionais é abrangente e essencial para o funcionamento eficaz de aplicativos modernos. Seja na recuperação, na manipulação, na estruturação ou na proteção das informações, o SQL desempenha um papel integral na construção de softwares conectados e adaptáveis.

O domínio do conceito de instruções SQL é crucial para um profissional

O domínio do conceito de instruções SQL é crucial para um profissional de desenvolvimento de software, visto que a integração eficaz de aplicações com bancos de dados é uma parte fundamental do desenvolvimento de sistemas robustos. A linguagem permite que os desenvolvedores se comuniquem de maneira eficiente com bancos de dados relacionais, realizando algumas operações, como consultas, inserções, atualizações e exclusões de dados. Isso é essencial para garantir que as aplicações possam acessar e manipular informações de forma rápida e precisa.

Além disso, o conhecimento profundo da linguagem SQL, de acordo com Alves (2021), permite aos desenvolvedores otimizar o desempenho das consultas, contribuindo para a eficiência global da aplicação. Ao aprenderem a escrever consultas eficientes e usar índices adequadamente, os desenvolvedores podem minimizar o tempo de resposta do sistema, melhorando a experiência do usuário e garantindo a escalabilidade da aplicação.

A segurança também é uma consideração crítica. Profissionais de desenvolvimento que dominam essa linguagem podem implementar práticas sólidas para se proteger contra injeções de SQL e outros ataques relacionados a bancos de dados, garantindo que a aplicação seja resistente a vulnerabilidades comuns.

O **armazenamento estruturado de dados** diz respeito à organização sistemática e lógica de informações dentro de um sistema utilizando uma estrutura predefinida para representar e armazenar dados. Nesse contexto, os dados são organizados em tabelas, registros e campos, seguindo um formato que permite a fácil consulta, a recuperação e a manipulação por meio de sistemas de gerenciamento de um banco de dados do tipo relacional. Esse método de armazenamento oferece eficiência na busca por informações específicas, contribuindo para a integridade e a consistência dos dados, aspectos cruciais em ambientes em que a precisão e a acessibilidade das informações são essenciais para operações eficazes e análises detalhadas.

A maestria nas instruções SQL é essencial para desenvolvedores que buscam criar aplicações eficientes, seguras e altamente funcionais, capazes de interagir de forma eficaz com bancos de dados do tipo relacional.

É de vital importância que um DBA domine os conceitos das instruções SQL e tenha consciência de que, uma vez realizada uma ação no banco de dados, os efeitos são irreversíveis, ou seja, não há volta.

Linguagem SQL: uma visão detalhada dos tipos

Existem vários tipos de linguagens SQL, cada uma servindo a propósitos específicos nos contextos da administração e do gerenciamento desse tipo de banco de dados. A primeira categoria a ser destacada é o DDL (Data Definition Language), destinada à definição e à modificação da estrutura do banco de dados. Instruções, tais como CREATE, ALTER e DROP, fazem parte dessa categoria, permitindo a criação de tabelas, a modificação das características delas e até mesmo a exclusão de elementos quando assim for necessário.

Outra categoria essencial é o DML (Data Manipulation Language), que se concentra na manipulação dos dados armazenados no banco. Instruções, como SELECT, INSERT, UPDATE e DELETE, proporcionam aos usuários a capacidade de recuperar informações, inserir novos dados, atualizar registros existentes e excluir informações, quando houver necessidade. Além disso, o DML é crucial para interações diárias com o banco de dados e é frequentemente utilizado por desenvolvedores de softwares, administradores, demais profissionais de TI e usuários finais.

Por fim, temos o DCL (Data Control Language), responsável pelo controle de acesso e permissões no banco de dados. Instruções, como GRANT e REVOKE, são fundamentais para garantir a segurança e a acessibilidade do sistema, permitindo, assim, que os administradores concedam ou retirem privilégios de usuários (*users*) e papéis (*roles*) específicos. Essa camada de controle de acesso é essencial para proteger informações sensíveis e manter a integridade do banco de dados em ambientes complexos.

As diferentes categorias de linguagens SQL desempenham papéis complementares, garantindo uma gestão completa e eficaz de bancos de dados do tipo relacional.

SQL DDL (Data Definition Language)

Em português, “Linguagem de Definição de Dados” pode ser classificada como instruções fundamentais para a criação e a modificação da estrutura do banco de dados, permitindo aos administradores criar, alterar ou excluir objetos, como tabelas (*tables*), índices (*index*) e visões (*views*).

Vamos explorar algumas dessas instruções e os impactos delas na arquitetura do banco de dados.

CREATE TABLE

Trata-se de uma instrução que é utilizada para criar uma nova tabela no banco de dados. Ao ser executada, definirá a estrutura dessa tabela, especificando o nome, as colunas (campos) e os tipos de dados que cada uma dessas colunas poderá armazenar.

Além disso, é possível incluir restrições, como chaves primárias, chaves estrangeiras, preenchimento obrigatório, autoincremento de valores numéricos e índices.

```
CREATE TABLE Funcionarios (  
  ID INT PRIMARY KEY,  
  Nome VARCHAR(50),  
  Cargo VARCHAR(30),  
  Salario DECIMAL(10, 2)  
);
```

Figura 3 - Exemplo da instrução CREATE TABLE / Fonte: os autores.

Descrição da Imagem: é demonstrada a instrução que criará uma tabela chamada "Funcionarios", com colunas para ID, Nome, Cargo e Salário. O ID é definido como chave primária. Fim da descrição.

Já a instrução CREATE DATABASE é utilizada para criar um novo banco de dados em um SGBD. Ao ser executada, essa instrução define um novo espaço de armazenamento para os dados, permitindo a criação e a organização de tabelas, índices, procedimentos armazenados, rotinas personalizadas, estruturação e todos os outros objetos relacionados a esse banco.

ALTER TABLE

Essa instrução é empregada para modificar a estrutura de uma tabela existente no banco de dados. Ao ser executada, permite adicionar, modificar ou excluir colunas, definir restrições, como chaves primárias ou estrangeiras, e realizar outras alterações que afetam a organização dos dados, desde que essas alterações não venham a ferir o princípio do ACID (Atomicidade, Consistência, Isolamento e Durabilidade).

```
ALTER TABLE Funcionarios
ADD COLUMN DataContratacao DATE;
```

Figura 4 - Exemplo da instrução ALTER TABLE / Fonte: os autores.

Descrição da Imagem: é exibida a instrução que agregará uma nova coluna chamada "DataContratacao" à tabela "Funcionarios". Fim da descrição.

DROP TABLE

Essa instrução é utilizada para remover permanentemente uma tabela existente em um banco de dados relacional. Ao ser executada, todos os dados, índices e demais objetos associados a essa tabela são devidamente eliminados, desde que essas exclusões não venham a ferir o princípio do ACID.

```
DROP TABLE Funcionarios;
```

Figura 5 - Exemplo da instrução DROP TABLE / Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução removerá completamente a tabela "Funcionarios" e todos os dados associados a ela. Fim da descrição.

Já a instrução DROP DATABASE é empregada para eliminar um banco de dados por completo, excluindo todas as tabelas, dados e objetos associados a ele. Ao ser executada, essa instrução remove permanentemente o espaço de armazenamento reservado para esse banco no sistema de gerenciamento, liberando recursos de máquina para outros fins.

Segundo Barboza e Freitas (2018), as instruções DDL são cruciais para a modelagem e a adaptação contínua da estrutura do banco de dados, proporcionando aos administradores as ferramentas necessárias para refletir as mudanças nos requisitos dos negócios e garantir uma organização eficiente dos dados.

SQL DML (Data Manipulation Language)

Em português, “linguagem de manipulação de dados” representa instruções essenciais para manipular os dados armazenados em tabelas dentro de um banco de dados.

Exploraremos algumas dessas instruções, destacando como elas afetam os registros e oferecem flexibilidade na interação com as informações contidas no banco.

SELECT

É uma instrução fundamental para recuperar dados de um banco de dados relacional. Ao ser utilizada, segundo Silberschatz (2020), permite a seleção de informações específicas de uma ou mais tabelas, atendendo a condições estabelecidas a partir de cláusulas, como WHERE.

```
SELECT Nome, Salario  
FROM Funcionarios  
WHERE Cargo = "Analista";
```

Figura 6 - Exemplo da instrução SELECT / Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução retornará os nomes e os salários de todos os funcionários com o cargo de “Analista” na tabela “Funcionarios”. Fim da descrição.

A instrução SQL é a mais utilizada em um banco de dados do tipo relacional. Devido à estrutura, o banco de dados é criado para fins de consulta, a fim de facilitar que o usuário encontre o mais rapidamente possível as informações que busca, sejam mais simples, sejam mais complexas. Por esse motivo, também é a instrução que oferece mais possibilidade de utilização ao administrador do banco.

INSERT

Essa instrução é utilizada para adicionar novos registros a uma tabela em um banco de dados relacional. Ao ser executada, essa instrução permite a inserção de dados em colunas específicas, associando valores aos campos correspondentes.

```
INSERT INTO Funcionarios(Nome, Cargo, Salario)
VALUES ('Joana Silva', 'Desenvolvedor', 6000);
```

Figura 7 - Exemplo da instrução INSERT / Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução inserirá um novo funcionário na tabela "Funcionarios", com nome, cargo e salário especificados. Fim da descrição.

UPDATE

Essa instrução é utilizada para modificar registros existentes em uma tabela de um banco de dados. Ao ser executada, permite a atualização de valores em campos (colunas) específicos com base em condições previamente estabelecidas.

```
UPDATE Funcionarios
SET Salario = Salario * 1.1
WHERE Cargo = 'Desenvolvedor';
```

Figura 8 - Exemplo da instrução UPDATE / Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução alterará o valor contido no campo "Salario", aumentando em 10% esse valor, que é referente ao salário de todos os registros existentes na tabela "Funcionarios". Fim da descrição.

DELETE

Essa instrução é utilizada para remover registros de uma tabela em um banco de dados. Ao ser executada, tem, por característica, excluir todos os registros que atendam às condições especificadas na cláusula WHERE.

```
DELETE FROM Funcionarios  
WHERE Nome = 'Joana Silva';
```

Figura 9 - Exemplo da instrução DELETE / Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução deletará o registro associado à funcionária Joana Silva na tabela "Funcionarios". Fim da descrição.

Ao utilizar a instrução DELETE, um DBA deve adotar precauções para evitar a exclusão acidental ou irreversível de dados críticos. É fundamental que o administrador execute a instrução com precisão, aplicando condições WHERE específicas para delimitar quais registros serão removidos, evitando a exclusão de conjuntos de dados inteiros. Além disso, é aconselhável realizar um backup prévio do banco de dados ou, então, da tabela-alvo, permitindo a recuperação em caso de erro. O uso de transações é recomendado para garantir a atomicidade da operação, assegurando que o DELETE seja executado integralmente ou revertido em caso de falha. Essas práticas são cruciais para manter a integridade e prevenir a perda irreparável de informações importantes durante a execução dessa instrução no ambiente do banco de dados.

Segundo Machado (2020), as instruções DML são cruciais para a manipulação eficaz dos dados em um banco de dados relacional, oferecendo aos usuários as capacidades de recuperar, inserir, modificar e excluir informações de maneira controlada e eficiente.

Variações da instrução SELECT

A instrução SQL DML SELECT é robusta e flexível, permitindo a recuperação de dados de tabelas em um banco de dados relacional e oferecendo uma variedade de cláusulas que possibilitam aos desenvolvedores e analistas de dados refinar as próprias consultas de maneira precisa, adaptando a recuperação de dados às necessidades específicas de cada situação. Essa flexibilidade torna a SQL uma ferramenta poderosa para a manipulação e a análise de informações em ambientes de banco de dados relacionais.

Vamos detalhar as principais cláusulas dela:

SELECT

indica as colunas que serão recuperadas da tabela.

Exemplo: `SELECT Nome, Salario.`

FROM

especifica a tabela da qual os dados serão recuperados.

Exemplo: `FROM Funcionarios.`

WHERE

filtra os resultados com base em uma condição.

Exemplo: `WHERE Cargo = 'Analista'.`

GROUP BY

agrupa os resultados com base em uma ou mais colunas.

Exemplo: `GROUP BY Cargo.`

HAVING

filtra os resultados após a aplicação da cláusula `GROUP BY`.

Exemplo: `HAVING COUNT(*) > 5.`

ORDER BY

ordena os resultados com base em uma ou mais colunas, podendo ser ascendente (ASC) ou descendente (DESC).

Exemplo: `ORDER BY Salario DESC.`

LIMIT/OFFSET

controla a quantidade de resultados retornados (LIMIT) e a partir de qual posição (OFFSET).

Exemplo: LIMIT 10 OFFSET 20.

```
SELECT Nome, Salario
FROM Funcionarios
WHERE Cargo = 'Analista';
GROUP BY Cargo
HAVING COUNT(*) > 5
ORDER BY Salario DESC
LIMIT 10 OFFSET 20;
```

Figura 10 - Exemplo da instrução SELECT utilizando as principais cláusulas / Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução recuperará os nomes e os salários dos analistas na tabela "Funcionarios", agrupando-os por cargo e filtrando-os para incluir apenas aqueles com mais de cinco ocorrências, ordenando pelo salário de forma descendente e limitando o resultado a 10 registros a partir do 20°. Essas cláusulas proporcionam uma poderosa capacidade de manipulação e recuperação de dados no SQL. Fim da descrição.

SQL DCL (Data Control Language)

Em bancos de dados relacionais, uma **ROLE** (ou papel) é um conceito fundamental que permite agrupar usuários com funções e permissões semelhantes, podendo, posteriormente, ser atribuída a um usuário ou a um grupo de usuários específicos. O termo **ROLE** pode ser definido como **um conjunto de permissões que pode ser atribuído a usuários específicos ou a outros papéis**. Essa abordagem proporciona uma maneira eficiente de gerenciar e conceder privilégios em larga escala, simplificando a administração do banco de dados.

Ao criar uma **ROLE**, segundo Pichetti, Vida e Cortes (2021), um administrador pode atribuir a ela certas permissões, como **SELECT**, **INSERT**, **UPDATE** ou

DELETE, em determinadas tabelas. Em seguida, essa ROLE pode ser associada a usuários individuais. Isso significa que, ao contrário de conceder permissões a cada usuário individualmente, o administrador atribui a ROLE ao usuário, que herda automaticamente as permissões associadas. Essa abordagem facilita a manutenção e o gerenciamento de privilégios, especialmente em ambientes com muitos usuários ou quando há uma lógica clara de grupos de acesso.

Por exemplo, uma ROLE chamada “analista” pode ter permissões de SELECT em várias tabelas relevantes para a análise de dados. Os usuários que desempenham o papel de analistas podem ser atribuídos à “analista”, garantindo que todos compartilhem as mesmas permissões. Isso não apenas simplifica a administração do banco de dados, mas também reforça a segurança, uma vez que é mais fácil controlar e auditar as permissões atribuídas aos grupos de usuários.

Nesse contexto, o uso de roles contribui para a eficiência, a otimização, a organização e a segurança na gestão de acessos e privilégios em bancos de dados do tipo relacional.

Suponha que estamos gerenciando um banco de dados para uma empresa e criaremos uma role chamada “AnalistaFinanceiro”, que terá permissões específicas para acessar e analisar dados financeiros. A seguir, é exibido um exemplo prático de utilização da instrução CREATE ROLE.

```
-- Criar a ROLE "AnalistaFinanceiro"
CREATE ROLE AnalistaFinanceiro

-- Atribuir permissões à ROLE
GRANT SELECT ON TabelaFinanceira TO AnalistaFinanceiro;
GRANT SELECT ON OutraTabelaFinanceira TO AnalistaFinanceiro;

-- Criar a ROLE "AnalistaFinanceiro"
GRANT AnalistaFinanceiro TO UsuarioJoao
```

Figura 11 - Exemplo da instrução CREATE ROLE Fonte: os autores.

Descrição da Imagem: é demonstrado que a instrução CREATE ROLE cria a ROLE “AnalistaFinanceiro”. Em seguida, as instruções GRANT SELECT são utilizadas para atribuir à ROLE permissões de leitura em duas tabelas relacionadas às finanças da empresa. Por fim, a instrução GRANT AnalistaFinanceiro TO UsuarioJoao associa a ROLE “AnalistaFinanceiro” ao usuário “UsuarioJoao”, permitindo que ele herde as permissões associadas à ROLE. Dessa forma, o usuário “UsuarioJoao” terá um acesso controlado às tabelas financeiras de acordo com as permissões definidas na ROLE “AnalistaFinanceiro”. Fim da descrição.

As instruções SQL do tipo DCL (Data Control Language), em português, “Linguagem de Controle de Dados”, são cruciais para o gerenciamento de acesso e controle de permissões em um banco de dados. As duas principais instruções são GRANT e REVOKE.

**ZOOM NO CONHECIMENTO****GRANT:**

Essa instrução é utilizada para conceder privilégios específicos. Ao ser executada, possibilita a atribuição de permissões para a realização de determinadas operações em objetos, como tabelas, visões ou procedimentos.

Exemplo:

GRANT SELECT, INSERT ON Funcionarios TO AnalistaRole;

Onde: concederá ao role “AnalistaRole” a permissão de realizar operações de SELECT e INSERT na tabela “Funcionarios”.

REVOKE:

Essa instrução retira privilégios concedidos. É utilizada para retirar privilégios previamente concedidos a usuários ou roles em um banco de dados relacional. Ao ser executada, a instrução REVOKE anula permissões específicas, revogando a capacidade de realizar operações em objetos, como tabelas, visões ou procedimentos.

Exemplo:

REVOKE DELETE ON Pedidos FROM Vendedor;

Onde: removerá a permissão de DELETE para o usuário ou role “Vendedor” na tabela “Pedidos”.

Essas instruções são essenciais para garantir a segurança e a integridade do banco de dados, controlando quem pode realizar as operações em objetos específicos dele. Ao conceder e revogar privilégios de forma cuidadosa, o DBA poderá garantir que apenas usuários autorizados tenham acesso a determinadas funcionalidades, protegendo, assim, informações sensíveis e mantendo a consistência do sistema.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Imbuídos do desejo de capacitar futuros profissionais de Tecnologia da Informação, é possível traçar conexões entre o estudo profundo da linguagem SQL e o ambiente profissional que te aguarda. Segundo Silberschatz (2020), aprofundar-se na complexidade e na versatilidade dessa linguagem é essencial para o trabalho bem-sucedido de um DBA, porque, no mundo corporativo, as habilidades de manipular e gerenciar bancos de dados constituem uma competência-chave para qualquer profissional da área.

Ao dominarem a linguagem SQL, os profissionais terão o poder de extrair informações valiosas a partir de dados armazenados em estruturas previamente montadas, proporcionando uma base sólida para a tomada de decisões estratégicas nas organizações. Projetar consultas sofisticadas, criar e otimizar estruturas de bancos de dados e solucionar problemas de desempenho são diferenciais significativos no mercado de trabalho. Esse conhecimento não apenas amplia as oportunidades de carreira, mas também possibilita contribuições eficazes para projetos de grande envergadura.

Além disso, a linguagem SQL, de acordo com Pichetti, Vida e Cortes (2021), transcende fronteiras organizacionais, sendo uma habilidade altamente requisitada em diversos setores. A aplicação prática dessas habilidades no ambiente profissional não apenas aprimora a eficiência operacional, mas também promove a confiabilidade e a segurança dos sistemas de informação. Portanto, ao embarcar nessa jornada de aprendizado, você está se equipando não apenas com conhecimento técnico, mas com as ferramentas necessárias para moldar positivamente o cenário tecnológico futuro.

A sincronia entre a aprendizagem da linguagem SQL e as exigências do ambiente corporativo é evidente na crescente demanda por profissionais com expertise nesse domínio. O futuro aguarda aqueles que se dedicam a essa jornada, oferecendo oportunidades de contribuir para inovações tecnológicas e desafios complexos no universo da Tecnologia da Informação.

AUTOATIVIDADE

1. A função de um Administrador de Banco de Dados (DBA) é crucial para garantir a eficiência, a segurança e a integridade dos dados em um sistema de gerenciamento de banco de dados. Esse profissional é o responsável por todo o gerenciamento da estrutura de armazenamento do bem mais importante e relevante de uma organização: os dados. Sendo altamente requisitado no mercado atual, o DBA é muito bem remunerado para exercer as próprias funções, afinal, ocupará um cargo extremamente relevante e estratégico na organização.

Considerando as principais responsabilidades de um Administrador de Banco de Dados (DBA), assinale a alternativa correta:

- a) Desenvolvimento de códigos de aplicação.
- b) Design de interfaces de usuário.
- c) Manutenção de hardware de servidores.
- d) Implementação de algoritmos de criptografia.
- e) Gerenciamento da segurança e integridade dos dados.

2. Consultas realizadas em um banco de dados relacional são processos fundamentais para recuperar, filtrar e analisar dados armazenados de maneira eficaz. A linguagem SQL é comumente empregada para formular consultas e a instrução SELECT é central nesse contexto. A eficiência das consultas é vital para obter *insights* rápidos e precisos. O projeto adequado de índices, a otimização das consultas e a compreensão das nuances do modelo relacional são aspectos cruciais para garantir um desempenho otimizado, ao realizar consultas em um banco de dados relacional, contribuindo significativamente para a eficácia e a agilidade na obtenção de informações necessárias.

Considerando a instrução SQL SELECT, analise as afirmativas a seguir:

- I - A cláusula ORDER BY é utilizada para filtrar os resultados de uma consulta com base em condições específicas.
- II - A cláusula WHERE é utilizada para agrupar os resultados da consulta com base em uma ou mais colunas.
- III - A cláusula GROUP BY é utilizada para ordenar os resultados de uma consulta de forma ascendente ou descendente.
- IV - A cláusula HAVING é utilizada para aplicar condições de filtro após a execução da cláusula GROUP BY.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

3. A estruturação de um banco de dados relacional é um processo essencial para organizar e armazenar dados de maneira eficiente, seguindo os princípios do modelo relacional. Esse tipo de BD é composto por tabelas: cada uma delas representa uma entidade específica e as relações delas são estabelecidas por meio de chaves estrangeiras. A definição adequada e a normalização são práticas fundamentais nesse contexto, visando eliminar redundâncias e garantir a integridade e a consistência dos dados. Além disso, a estruturação envolve a criação de índices para otimizar a velocidade das consultas, permitindo um acesso mais rápido e eficiente às informações. A escolha de tipos de dados apropriados para cada coluna, a definição de chaves primárias e estrangeiras, e a consideração de requisitos de desempenho são aspectos cruciais durante o processo de estruturação. A abordagem cuidadosa na criação e na manutenção da estrutura do banco é fundamental para garantir que ele atenda de maneira eficaz às necessidades do sistema, proporcionando uma base sólida para operações de armazenamento e recuperação de dados.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - A instrução SQL DDL permite a definição da estrutura do banco de dados.

PORQUE

II - A instrução SQL DDL inclui a criação, a modificação ou a remoção de tabelas.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

ALVES, W. P. **Banco de dados**: teoria e desenvolvimento. São Paulo: Érica, 2021.

BARBOZA, F. F. M.; FREITAS, P. H. C. **Modelagem e desenvolvimento de banco de dados**. Porto Alegre: SAGAH, 2018.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

MACHADO, F. N. R. **Banco de dados**: projeto e implementação. São Paulo: Érica, 2020.

PICHETTI, R. F.; VIDA, E. da S.; CORTES, V. S. M. P. **Banco de dados**. Porto Alegre: SAGAH, 2021.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

GABARITO

1. Opção E.

Dentre as responsabilidades fundamentais de um Administrador de Banco de Dados (DBA), o gerenciamento da segurança e integridade dos dados é uma tarefa crítica. O DBA é o responsável por implementar políticas de segurança, controlar o acesso aos dados, garantir backups regulares e monitorar a integridade do banco de dados para prevenir a perda ou a corrupção de informações. O desenvolvimento de códigos de aplicação e o design de interfaces de usuário são, geralmente, tarefas atribuídas a desenvolvedores de software. A manutenção de hardware de servidores é mais associada a profissionais de infraestrutura, enquanto a implementação de algoritmos de criptografia é uma atividade mais relacionada a especialistas em segurança da informação do que a um DBA.

2. Opção D.

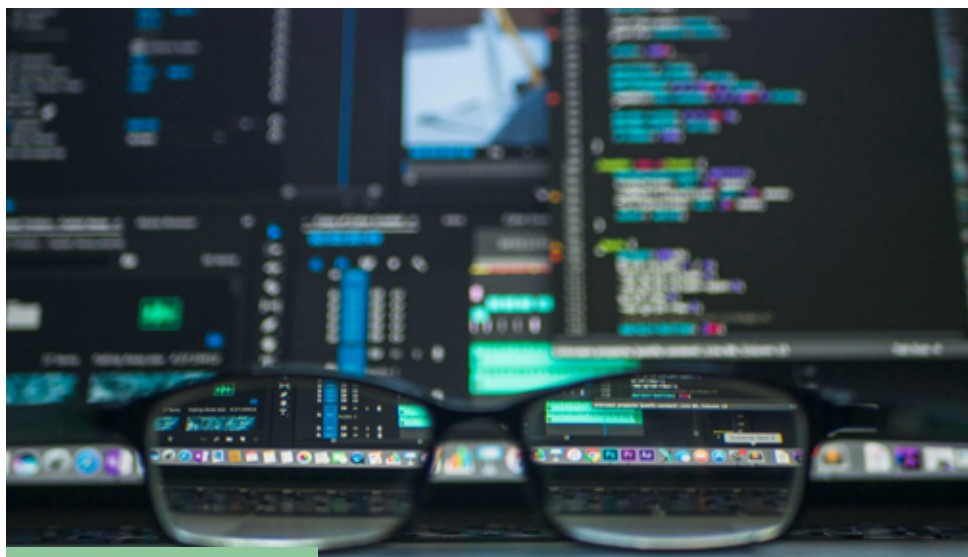
A afirmativa I é verdadeira, pois a cláusula ORDER BY é utilizada para ordenar os resultados de uma consulta com base em condições específicas. A afirmativa II é verdadeira, pois a cláusula WHERE é utilizada para filtrar os resultados da consulta com base em condições específicas. A afirmativa III é verdadeira, pois a cláusula GROUP BY é utilizada para agrupar os resultados da consulta com base em uma ou mais colunas. A afirmativa IV é falsa, pois a cláusula HAVING é utilizada para aplicar condições de filtro após a execução da cláusula GROUP BY, não antes.

3. Opção A.

A asserção I afirma corretamente que a instrução SQL DDL permite a definição da estrutura do banco de dados, enquanto a asserção II está correta e justifica a primeira, ao incluir a criação, a modificação ou a remoção de tabelas.



unidalle



TEMA DE APRENDIZAGEM 4

CRIANDO BANCOS DE DADOS SQL

MINHAS METAS

- Conhecer a criação de um banco de dados.
- Compreender os fundamentos do modelo relacional.
- Desenvolver habilidades práticas na linguagem SQL para a criação e a manutenção de bancos de dados.
- Entender a criação e a alteração de tabelas na prática.
- Verificar quais tipos de dados utilizar nos campos das tabelas.
- Explorar a chave primária na prática.
- Aplicar os conhecimentos adquiridos a partir da realização de projetos reais de criação de bancos de dados.

INICIE SUA JORNADA

Bem-vindo à empolgante jornada de criação de bancos de dados, uma habilidade essencial para qualquer estudante de Tecnologia da Informação (TI). Essa jornada será repleta de desafios e descobertas, proporcionando um profundo entendimento de como estruturar e gerenciar dados de forma eficaz. Para começar, é vital mergulhar nos fundamentos do modelo relacional, compreendendo conceitos-chave, como tabelas, chaves primárias, chaves estrangeiras e normalização. Essa base sólida será a chave para construir bancos de dados robustos e eficientes.

Ao avançar, o SQL se tornará o seu aliado mais próximo. Dominar alguns comandos, como CREATE TABLE, INSERT, UPDATE e DELETE, abrirá as portas para a criação e a manipulação eficazes. Aprender a aplicar essas instruções em contextos do mundo real, com atenção especial à integridade, será crucial para você se destacar como um(a) profissional de banco de dados. A prática consistente a partir de projetos te ajudará a solidificar essas habilidades.

Nesta caminhada, a experiência prática será o verdadeiro tesouro. Trabalhar em projetos reais permitirá a aplicação direta dos conhecimentos adquiridos, enfrentando desafios autênticos de design e implementação. Ao desenvolver bancos de dados para cenários específicos, você estará dando vida às suas habilidades, enfrentando decisões de design e compreendendo as nuances do desempenho do sistema. Esse é apenas o início: a jornada de criação de bancos de dados é dinâmica, prometendo um contínuo crescimento e aprimoramento das suas habilidades ao longo do caminho. Embarque com entusiasmo e prepare-se para uma trajetória repleta de aprendizado e conquistas.

```

84 DECLARE @invoiceDateTemp Datetime
85 SET @invoiceDateTemp = CONVERT(DATETIME, '2023-10-27 10:00:00')
86
87 INSERT #CTemp
88 (line, itemCode, itemBarCode, couponType, couponStatus,
89 couponSerialNoRef, dayShelfLife, expireDate, etc...)
90 SELECT DISTINCT
91 T.line, T.itemCode, T.itemBarCode, T.couponType, T.couponStatus,
92 DATEADD(day, I.dayShelfLife, @invoiceDateTemp) AS expireDate,
93 IIF(T.couponUpgrade = 'Y', T.nextOrderNo, NULL) AS nextOrderNo,
94 T.couponUpgrade
95 FROM
96 OrderDetail AS T INNER JOIN Item AS I ON T.itemCode = I.itemCode
97 WHERE
98 (T.companyCode = @companyCode) AND
  
```





PLAY NO CONHECIMENTO

Prepare-se para uma imersão completa no fascinante mundo da criação de bancos de dados, uma jornada vital para todo estudante de Tecnologia da Informação (TI). Agora, a oportunidade de aprofundar o seu conhecimento e expandir as suas habilidades se apresenta mediante um podcast exclusivo, denominado *Desbravando Dados*, que é dedicado a desvendar os segredos da construção de bancos de dados que sejam robustos e eficazes, com o intuito de atender às atuais demandas de mercado. Ao dar o *play* nessa incrível fonte de conhecimento, você terá acesso a *insights* valiosos sobre o modelo relacional, aprendendo a estruturar dados de maneira inteligente e eficiente.

No universo deste podcast, exploraremos as nuances da linguagem SQL, desmistificando comandos, como CREATE, INSERT e UPDATE. Você será guiado(a) por especialistas que compartilharão experiências práticas na criação e no gerenciamento de bases de dados, revelando dicas e truques que vão além dos livros didáticos. Cada episódio é uma oportunidade única de aprimorar as suas habilidades e aprender com profissionais que enfrentaram desafios do mundo real na criação e na manutenção de sistemas de software.

Ao ouvir o nosso podcast, você não se torna apenas um(a) ouvinte, mas também um(a) participante ativo(a) na construção do seu conhecimento. Engaje-se nas discussões, envie perguntas e esteja pronto(a) para absorver um conteúdo repleto de experiências práticas que impulsionarão a sua jornada na criação de bancos de dados. É hora de transformar o seu tempo de escuta em uma oportunidade de crescimento profissional.

```

16
17
18
19 SELECT
20   OrderH.invoiceNo, OrderH.invoiceDate, OrderH.customerCode, OrderH.orderDate
21   OrderD.itemCode, I.itemName, OrderD.qty, OrderD.unitPrice, OrderD.netPrice
22 FROM
23   OrderHeader AS OrderH
24   INNER JOIN Customer AS Cust ON OrderH.customerCode = Cust.customerCode
25   INNER JOIN OrderDetail AS OrderD ON OrderH.orderId = OrderD.orderId
26   INNER JOIN Item AS I ON OrderD.itemCode = I.itemCode
27 WHERE
28   OrderD.netPrice > 1000
29 ORDER BY
30   OrderH.customerCode, OrderD.netPrice
  
```

VAMOS RECORDAR?

A linguagem SQL (Structured Query Language) é uma ferramenta essencial dos bancos de dados relacionais. Ela foi desenvolvida para ser uma linguagem de consulta declarativa, permitindo a interação eficiente com os dados armazenados em tabelas relacionais. As instruções, como SELECT, INSERT, UPDATE e DELETE, capacitam os usuários a realizar operações variadas, desde consultas simples até manipulações complexas de dados.

A modelagem de dados é um processo crítico na criação do banco, utilizando o diagrama Entidade-Relacionamento (ER), em que as entidades representam os objetos do mundo real e são interligadas a partir de relacionamentos. Essa abordagem visual ajuda a definir a estrutura do banco de dados, estabelecendo como as informações serão organizadas e como as diferentes entidades se relacionam entre si.

A integração entre o SQL e a modelagem é um ponto crucial, e os benefícios dessa abordagem são significativos. A linguagem facilita a interação intuitiva e eficaz com os dados, enquanto a modelagem assegura uma estrutura bem definida, minimizando redundâncias e otimizando o desempenho. Dessa forma, a conjunção desses dois conceitos é essencial para o design, a implementação e a gestão bem-sucedidos de bancos de dados relacionais, atendendo às necessidades das organizações em diversos setores.

<https://www.youtube.com/watch?v=XfO3TRvESBo>

DESENVOLVA SEU POTENCIAL

CICLO DE DESENVOLVIMENTO DE UM BANCO DE DADOS RELACIONAL

Esse ciclo pode ser descrito como um processo estruturado que abrange desde a concepção da ideia inicial do projeto até o momento no qual a manutenção contínua do banco de dados é realizada. Inicia-se com a **análise de requisitos**, em que as necessidades do usuário e os objetivos do sistema são detalhadamente levantados e definidos. Nessa fase, a compreensão profunda dos dados que serão armazenados é fundamental, delineando os relacionamentos entre as entidades e identificando as principais operações a serem realizadas.

Na etapa de projeto, segundo Silberschatz (2020), a **modelagem de dados** ganha destaque. Utilizando ferramentas, como o diagrama Entidade-Relacionamento (ER), os DBAs delineiam a estrutura lógica do banco de dados, definindo entidades, atributos e relacionamentos. A normalização é aplicada para eliminar redundâncias e garantir a integridade. Ao final dessa fase, o esquema do banco de dados, em conjunto com a definição de chaves primárias e estrangeiras, é estabelecido.

Com o projeto em mãos, inicia-se a fase de **implementação**. Nesse momento, a linguagem SQL desempenha um papel central. Utilizando comandos, como CREATE TABLE, INSERT, UPDATE e DELETE, o esquema projetado é transformado em um banco de dados real. Esse processo inclui a criação de tabelas, a definição de índices, a inserção de dados iniciais e o estabelecimento de restrições de integridade. A precisão e a coerência da implementação, segundo Machado (2020), são cruciais para garantir a funcionalidade adequada desse banco.

Após a implementação, parte-se para a fase de **testes**, momento em que é realizada uma série de ações, com o objetivo de garantir que o sistema atenda aos requisitos especificados. Isso inclui verificar a precisão das consultas SQL, garantir a integridade referencial e validar o desempenho do banco de dados em condições diversas. Qualquer inconsistência ou anomalia identificada durante os testes é corrigida e refinada para garantir que o sistema esteja robusto e pronto para a produção.

A última etapa é a **manutenção contínua**. À medida que o sistema é utilizado, novos requisitos podem surgir ou, então, ajustes pontuais podem ser necessários para otimizar o desempenho.

A manutenção, segundo Elmasri e Navathe (2018), envolve a aplicação de alterações no esquema, a inclusão de novas funcionalidades e a correção de eventuais problemas. Essa fase é iterativa, assegurando que o sistema de banco de dados continue a atender de maneira eficaz às necessidades em constante evolução da organização.

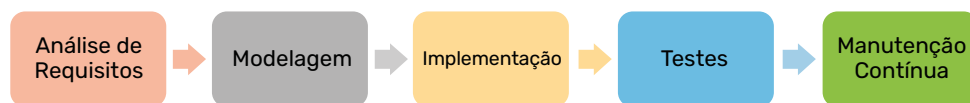


Figura 1 - Representação das etapas do ciclo de desenvolvimento de um banco de dados / Fonte: os autores.

Descrição da Imagem: a figura demonstra as etapas do ciclo. Ordenadas da esquerda para a direita, a primeira, na cor rosa, à esquerda, é chamada de "Análise de Requisitos". A segunda, logo à direita, na cor cinza, é a "Modelagem". A terceira, mais à direita, é a "Implementação" na cor amarela. Em seguida, na cor azul, encontra-se a etapa de "Testes". Por fim, a última etapa do ciclo, mais à direita, na cor verde, é chamada de "Manutenção Contínua". Fim da descrição.

O ciclo de desenvolvimento de um sistema de banco de dados relacional, portanto, é um processo contínuo e iterativo que abrange desde a concepção até a manutenção, garantindo a criação e a evolução eficaz de uma ferramenta vital para o gerenciamento eficiente dessa estrutura.

MYSQL: UM SISTEMA DE GERENCIAMENTO DE BANCO DE DADOS PODEROSO E VERSÁTIL

O MySQL é um SGBD de código aberto (*open source*) e multiplataforma que se destaca pela confiabilidade, desempenho e flexibilidade. Criado em 1995, na Suécia, por David Axmark, Allan Larsson e Michael Widenius, foi adquirido pela Oracle Corporation, tornando-se uma escolha muito popular entre os desenvolvedores de software atualmente. Ele é integrado à uma ampla variedade de aplicações, desde websites dinâmicos até sistemas empresariais complexos.

A arquitetura do MySQL segue o modelo cliente-servidor, em que um servidor gerencia e controla múltiplos clientes que podem acessar e manipular os dados armazenados. O banco de dados em si é composto por tabelas que organizam os dados em linhas e colunas, seguindo o modelo relacional. Suportando o padrão SQL, oferece uma ampla gama de comandos para realizar operações, como consultas, inserções, atualizações e exclusões.

O MySQL é reconhecido pelos recursos avançados, incluindo suporte para transações ACID (Atomicidade, Consistência, Isolamento e Durabilidade), replicação para garantir alta disponibilidade e escalabilidade e funções de segurança robustas, dando suporte à diversos tipos de índices para otimizar o desempenho das consultas. Não só, mas também oferece armazenamento de dados em diferentes formatos, incluindo MyISAM e InnoDB.

Uma das forças desse SGBD é a comunidade fortemente ativa de desenvolvedores e usuários. A vasta base de conhecimento disponível, os fóruns de discussão e a documentação extensiva tornam o aprendizado e a resolução de problemas acessíveis. Além disso, a Oracle oferece suporte comercial para organizações que buscam assistência especializada e garantia de desempenho.

O MySQL é altamente integrado a várias linguagens de programação, como PHP, Python,

**O MySQL é
altamente integrado
a várias linguagens
de programação**

Java, entre outras, facilitando a incorporação em diferentes aplicações. Também apresenta compatibilidade com diferentes sistemas operacionais, incluindo Windows, Linux e macOS, aumentando ainda mais a versatilidade e a facilidade de implementação em diversos ambientes.

Atualmente, está disponível em diferentes versões e edições para atender a uma variedade de necessidades e demandas do mercado atual. Dentre essas versões, podemos destacar as seguintes:

Community Edition: é gratuita e amplamente utilizada em projetos de código aberto de pequeno e médio porte.

Enterprise Edition: oferece recursos adicionais e suporte dedicado para projetos de grande porte voltados para ambientes corporativos mais complexos.

Enfim, o MySQL, segundo Pichetti, Vida e Cortes (2021), continua a ser uma escolha popular e confiável para o gerenciamento de um banco de dados devido à arquitetura sólida, recursos avançados, suporte ativo da comunidade e versatilidade. Seja para aplicações web, sistemas empresariais ou projetos de código aberto, esse SGBD continua a desempenhar um papel essencial no ecossistema que permite a utilização de bancos de dados relacionais.

MariaDB: Uma Alternativa Poderosa ao MySQL no Mundo dos SGBDs Relacionais

É um SGBD relacional criado como um fork do MySQL. Ele foi desenvolvido por Michael “Monty” Widenius, co-fundador do MySQL, após preocupações relacionadas à aquisição do MySQL pela Oracle Corporation. Com o objetivo de preservar a natureza de código aberto do MySQL e impulsionar a inovação, o MariaDB evoluiu para se tornar uma alternativa robusta e amplamente adotada no cenário de bancos de dados relacionais.

**ZOOM NO CONHECIMENTO**

Fork (bifurcação): em desenvolvimento de software, refere-se à criação de uma cópia independente do código-fonte de um projeto existente, podendo ocorrer por diversas razões, como divergências filosóficas, necessidades específicas ou descontentamento com o rumo do projeto original. Geralmente, o fork resulta em uma nova versão do software, gerida de forma autônoma, com desenvolvedores dedicados e uma possível evolução independente do projeto original. Esse conceito é comum em projetos de código aberto, permitindo a diversificação e o surgimento de alternativas que atendam a diferentes necessidades da comunidade de desenvolvedores.

O MariaDB mantém a maioria das características do MySQL, como a arquitetura cliente-servidor, a compatibilidade com SQL e uma estrutura de tabelas relacionais. No entanto, inclui melhorias e otimizações adicionais, como um notável otimizador de consultas, proporcionando um desempenho aprimorado em comparação com versões mais antigas do MySQL. Além disso, esse SGBD implementou recursos adicionais, como suporte para tipos de dados temporais aprimorados, extensões de índices e suporte para JSON.

Uma característica distintiva é o compromisso dele com o código aberto e a comunidade, sendo distribuído sob a Licença Pública Geral da GNU (GPL), mantendo o código acessível e livre. A comunidade ativa do MariaDB contribui para a evolução contínua do sistema, proporcionando correções de *bugs*, novos recursos e suporte abrangente por meio de fóruns de discussão e outras plataformas colaborativas. Empresas e organizações que buscam uma alternativa ao MySQL, muitas vezes, encontram nesse SGBD uma solução que oferece a estabilidade e a inovação necessárias para as aplicações.

Devido à compatibilidade com o MySQL, muitas aplicações e sistemas podem migrar facilmente de um para outro. Essa interoperabilidade simplifica a transição para usuários familiarizados com o MySQL. Empresas, como a Red Hat e a Google, demonstraram suporte ao MariaDB, ampliando a adoção dele em ambientes corporativos e de grande escala. Por isso, ele continua a ganhar popularidade como uma alternativa confiável no ecossistema de bancos de dados relacionais, oferecendo uma opção de código aberto flexível e de alto desempenho para uma variedade de casos de uso.

MYSQL WORKBENCH

É uma ferramenta de gerenciamento de banco de dados que oferece uma interface gráfica abrangente para facilitar o design, o desenvolvimento e a administração a partir do MySQL. A estrutura dele é dividida em diferentes componentes principais que se complementam para proporcionar uma experiência eficiente aos usuários.

A interface gráfica do Workbench é composta por áreas distintas, incluindo a seção de navegação, local em que os usuários podem acessar e explorar os próprios bancos de dados. O editor SQL permite a criação e a execução de consultas diretamente na interface, simplificando o desenvolvimento e a depuração de código escritos em linguagem SQL. Além disso, o modelo de dados permite a criação visual de esquemas de banco de dados, oferecendo uma visão intuitiva das tabelas, relacionamentos e índices.

Também inclui recursos de administração, como a gestão de usuários e privilégios, que facilitam a configuração e a manutenção de um ambiente de banco de dados seguro. Já o recurso de *backup* e *restore* permite a realização de cópias de segurança e a recuperação de dados de forma rápida e muito eficiente.

Outro recurso relevante é o de sincronização, que oferece a capacidade de comparar e sincronizar dados entre bancos de dados locais e remotos, facilitando a manutenção de ambientes distribuídos. Além disso, suporta a modelagem reversa, permitindo aos usuários gerar modelos visuais do MER baseando-se em bancos de dados existentes no servidor.

O MySQL Workbench proporciona uma solução abrangente para o gerenciamento de bancos de dados MySQL, unindo uma interface gráfica intuitiva com ferramentas poderosas de desenvolvimento, administração e manutenção de bancos de dados. Essa combinação faz da ferramenta uma escolha valiosa para profissionais de banco de dados que buscam eficiência e praticidade em suas tarefas diárias.

O termo “esquema” (*schema*) refere-se à estrutura que define a arquitetura lógica dos dados armazenados, descrevendo a maneira como as tabelas, os campos, os relacionamentos e as restrições são organizados no banco de dados. Pode ser definido como um *framework* que define a integridade e a consistência dos dados, estabelecendo as regras que determinam como as informações são armazenadas e acessadas.

Um esquema serve como um mapa abstrato que orienta a criação e a manipulação dos dados, proporcionando uma estrutura compreensível e organizada para facilitar a gestão eficiente das informações.

XAMPP

É um pacote de software que simplifica a configuração e o gerenciamento de um ambiente de desenvolvimento web. A estrutura dele abrange diversos componentes essenciais para criação e teste de aplicações web, incluindo o Apache, MySQL, PHP e Perl.

O Apache atua como um servidor web, permitindo a hospedagem local de sites e aplicativos. O MySQL é um sistema de gerenciamento de banco de dados que facilita o armazenamento e a recuperação de dados, enquanto o PHP e o Perl são linguagens de *script* amplamente utilizadas para o desenvolvimento dinâmico de conteúdo web.

O funcionamento prático do XAMPP é notável pela simplicidade e versatilidade. Após a instalação, o usuário pode iniciar ou parar os serviços do Apache e MySQL com apenas alguns cliques, tornando fáceis a criação e a interrupção de servidores locais, conforme necessário. O painel de controle fornece uma interface gráfica intuitiva para gerenciar os serviços e acessar configurações avançadas. Isso faz com que seja uma escolha popular entre os desenvolvedores que buscam uma solução rápida e eficiente para criar, testar e depurar aplicações web em um ambiente local antes de implantá-las em servidores remotos.

CRIANDO UM BANCO DE DADOS UTILIZANDO MYSQL WORKBENCH

A criação de um banco de dados relacional no MySQL, segundo Silberschatz (2020), envolve diversos passos essenciais para garantir uma estrutura organizada e eficiente para armazenar dados. O primeiro passo é planejar o esquema do banco de dados, definindo as tabelas necessárias, os campos e os relacionamentos entre elas. Isso geralmente é feito utilizando o modelo Entidade-Relacionamento (ER) para visualizar e projetar a estrutura de dados.

A partir de agora, veremos na prática o passo a passo para se criar um banco de dados. Nesse exemplo, o foco será na criação do banco de dados utilizando instruções SQL, e não na modelagem dele.

Partindo do princípio que o banco de dados já foi previamente modelado e que os softwares MySQL Workbench e XAMPP estão devidamente instalados, o primeiro passo é inicializar o serviço do MySQL. Isso é feito na interface chamada XAMPP Control.

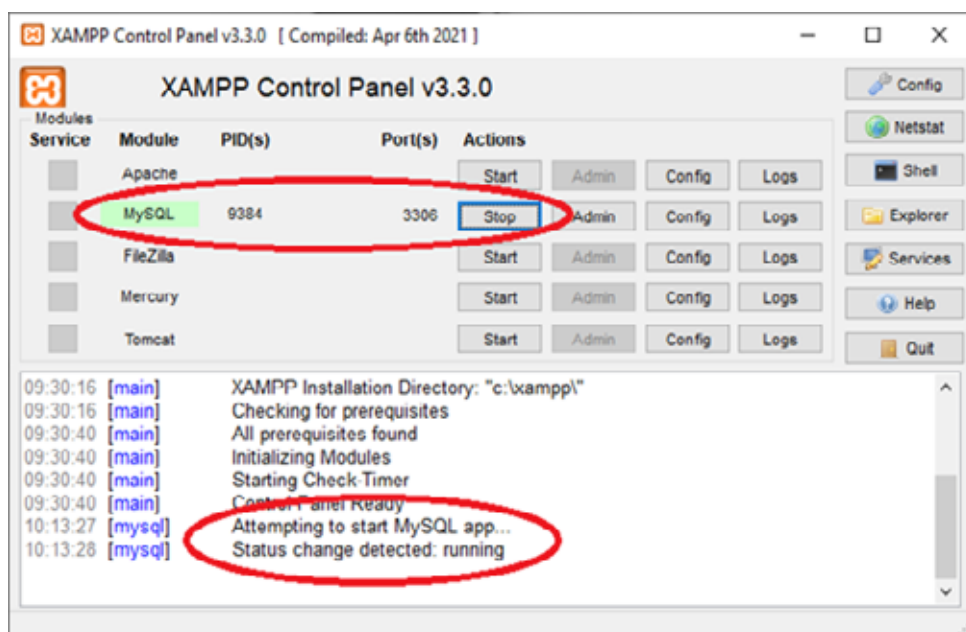


Figura 2 - Tela do XAMPP Control / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface do XAMPP Control. Nessa tela, será inicializado o serviço do MySQL. Há cinco botões START, um para cada serviço oferecido pela ferramenta (APACHE, MySQL, FILEZILLA, MERCURY e TOMCAT). O botão referente ao MYSQL será clicado e aparecerá a mensagem STOP nele, além dos números 9384 do PID(s), e 3306 da PORT(s). Há dois círculos vermelhos dando destaque ao serviço de inicialização do MySQL: o primeiro destaca as informações citadas, enquanto o segundo ressalta a mensagem de que houve mudança de status no serviço que se encontra em execução. Fim da descrição.

Após isso, é preciso abrir o MySQL Workbench e criar o servidor local do banco de dados. Para esse exemplo, criaremos um servidor chamado *Localhost*.

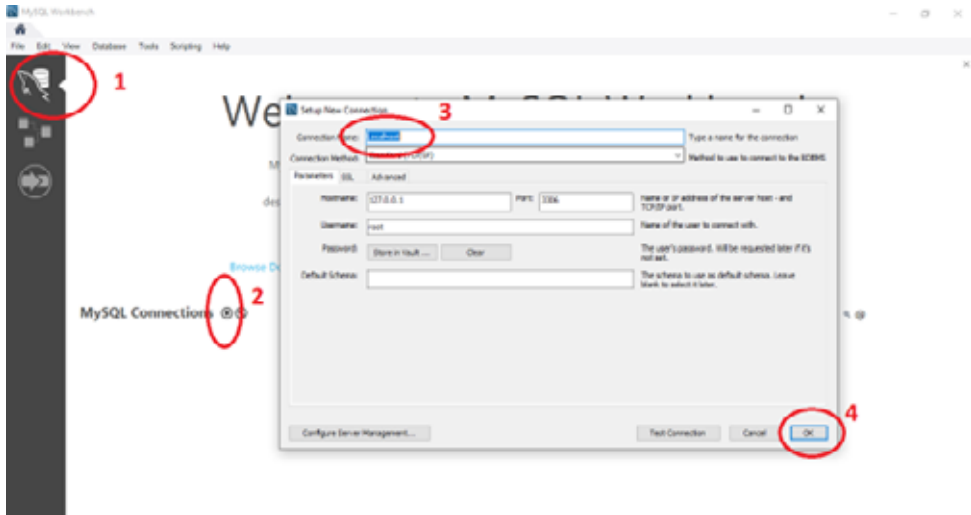


Figura 3 - Tela inicial do MySQL Workbench / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface do MySQL Workbench. Há uma tela de inicialização do software. Assim, é desejada boas vindas (WELCOME). À esquerda, encontra-se uma barra preta com uma barra de ferramenta (toolbar) desse software. Também há uma tela sobrepondo, que é a configuração do servidor do banco de dados, com dados referentes a ele, como "Nome", "Método", "Endereço IP (127.0.0.1)", "Porta de Conexão (3306)" e "Nome do Usuário de Conexão com o Servidor (ROOT)". Fim da descrição.

Nesse passo, é importante, inicialmente, verificar se a opção de manipulação de banco de dados está selecionada (1). Após isso, é preciso clicar no botão + (mais) na opção MySQL Connections (2). Uma janela será aberta e, nela, é necessário escrever o nome do servidor (3) e clicar no botão "OK" (4) para finalizar o processo. Também é possível fazer um teste de verificação da conexão do servidor a partir do botão "TEST CONNECTION", que dará um retorno de sucesso, caso os parâmetros estejam corretamente informados.

O próximo passo é carregar o servidor dando um clique duplo do mouse (*double click*) em Localhost para carregar a interface de manipulação. Após isso, iniciaremos o processo de criação do banco de dados propriamente dito.

Com o servidor inicializado, o próximo passo é utilizar a linguagem SQL para traduzir a modelagem em comandos que o MySQL compreenda. A instrução CREATE DATABASE é utilizada para criar o banco de dados em si. Já o CREATE TABLE é utilizado para criar as tabelas, especificando os nomes dos campos, tipos de dados e restrições, como chaves primárias e estrangeiras.

Na tela chamada Editor SQL, assim como o próprio nome remete, é o espaço no qual o DBA deverá inserir os códigos em linguagem SQL para manipulação e gerenciamento do banco de dados. No exemplo, criaremos duas tabelas: CLIENTES e VENDAS, ambas em um banco de dados chamado BD_LOJA.

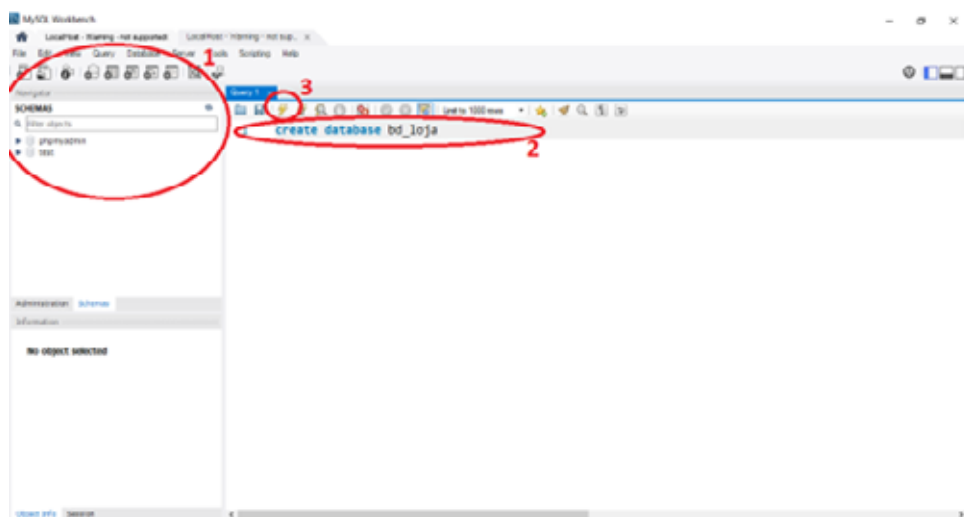


Figura 4 - Tela do editor de scripts SQL do MySQL Workbench / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Ela é utilizada para criar o banco de dados, que está na parte central da tela. À esquerda, é visualizada a seção chamada "SCHEMA", na qual ficam listados todos os bancos de dados instalados no servidor. Na parte superior da tela, são visualizados os menus de opções disponibilizados pela ferramenta. Fim da descrição.

Na tela do Editor SQL, escreva a instrução SQL CREATE DATABASE para criar o banco de dados BD_LOJA (2). Para executar, clique no ícone representado por um raio, logo acima do editor (3), ou, então, selecione, no MENU QUERY, a opção EXECUTE, que também pode ser realizada a partir da tecla de atalho CTRL+SHIFT+ENTER.

O próximo passo é ativar o banco de dados criado, tendo em vista que o servidor, normalmente, terá mais de um banco de dados hospedado. No Workbench, os bancos ficam listados na seção SCHEMAS, à esquerda da tela.

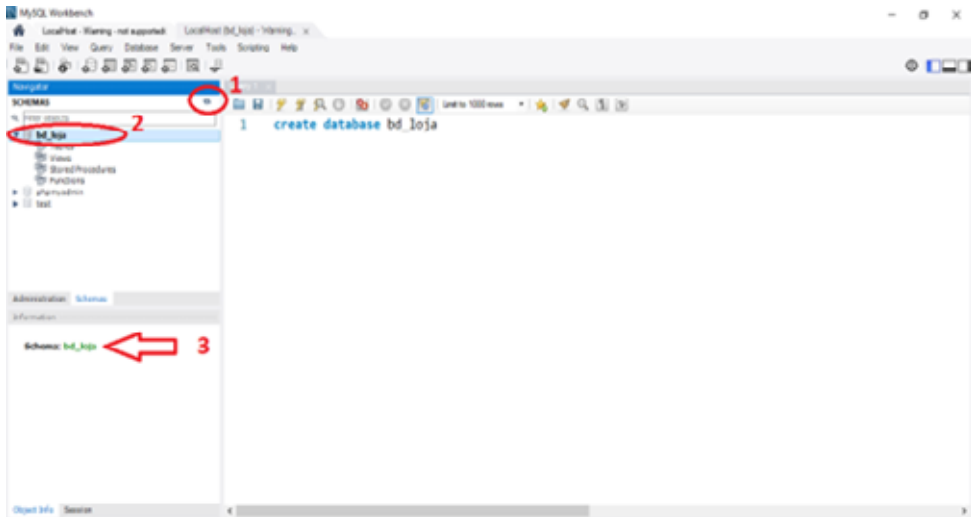


Figura 5 - Tela do editor de scripts SQL do MySQL Workbench / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Na parte central da tela, está escrito o comando CREATE DATABASE para criar o banco de dados BD_LOJA. À esquerda, é visualizada a seção chamada SCHEMA, na qual ficam listados todos os bancos de dados instalados no servidor, inclusive, com o banco de dados recém-criado, chamado BD_LOJA. Na parte superior da tela, são visualizados os menus de opções disponibilizados pela ferramenta. Fim da descrição.

Criado o banco de dados BD_LOJA, perceba que ele não aparece na seção SCHEMA. Para tanto, é necessário atualizar a lista. Para fazer isso, basta clicar em ATUALIZAR (1) e, assim que for feito, o novo banco aparecerá na lista (2).

É necessário ativar o BD_LOJA, para que ele possa ser devidamente manipulado no Workbench. Existem duas maneiras de se realizar essa ação. A primeira e mais simples é dar um duplo clique com o botão esquerdo do mouse no nome desse banco que consta na lista. A segunda é clicar com o botão direito do mouse nesse banco de dados e selecionar a opção SET AS DEFAULT SCHEMA. Em ambas situações, perceba que o nome do banco ficará em negrito, o que indica que ele está ativo no servidor e pronto para ser utilizado. No canto esquerdo da tela, é possível visualizar o nome do banco de dados que está ativo (3).

Ao criar as tabelas, segundo Barboza e Freitas (2018), é fundamental definir as chaves primárias para garantir a unicidade dos registros e estabelecer os devidos relacionamentos por meio de chaves estrangeiras. Isso contribui para a integridade referencial do banco de dados, assegurando a consistência dos dados.

No exemplo, agora, criaremos duas tabelas: CLIENTES e VENDAS. O relacionamento entre elas é do tipo 1 para N, em que um registro de CLIENTES estará relacionado a um ou a mais de VENDAS, que conterà a chave estrangeira (FK) desse relacionamento. Ambas as tabelas terão as respectivas chaves primárias (PK).

Primeiramente, criaremos a tabela CLIENTES.

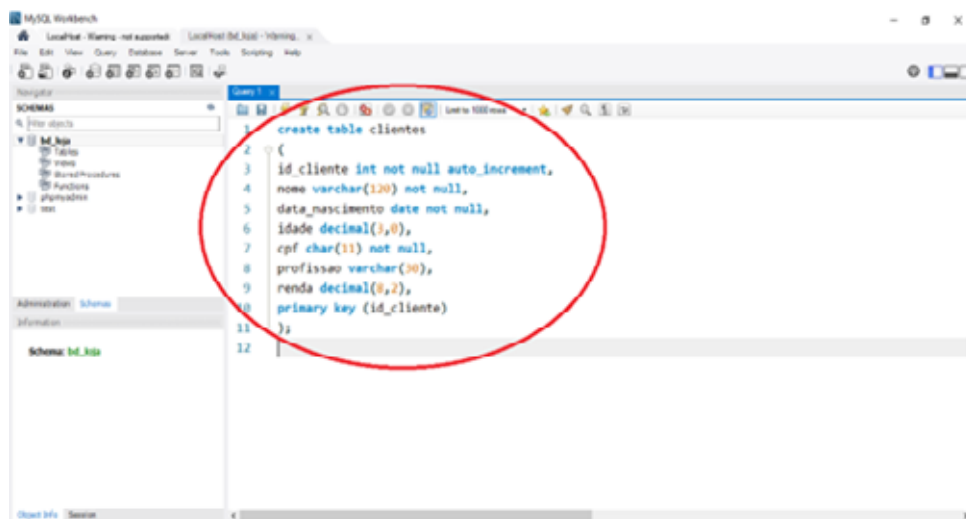


Figura 6 - Tela do editor de scripts SQL do MySQL Workbench utilizada para criar a tabela CLIENTES
Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Na parte central da tela, está escrito o comando CREATE TABLE, com todos os campos (atributos), os tipos de dados e as devidas restrições de integridade que compõem a tabela CLIENTES. À esquerda, é visualizada a lista de bancos de dados disponíveis no servidor Localhost. Há o BD_LOJA como o ativo, destacado com o nome em negrito nessa lista. Fim da descrição.

A tabela CLIENTES tem 7 (sete) campos distintos: ID_CLIENTE, NOME, DATA_NASCIMENTO, IDADE, CPF, PROFISSAO e RENDA. É muito importante salientar que, apesar de não ser uma regra específica do MySQL, recomenda-se fortemente a não utilização de caracteres especiais, símbolos ou acentuação nos nomes de campos.

Utilize somente letras e números. Também é muito interessante padronizar o tamanho das letras entre maiúsculas e minúsculas, assim como é possível visualizar no exemplo.

O campo ID_CLIENTE é definido como a chave primária da tabela CLIENTE, uma vez que ele será utilizado como referência dela nos devidos relacionamentos existentes nesse banco de dados.

A restrição NOT NULL indica que o campo é de preenchimento obrigatório, ou seja, quando um novo registro for inserido na tabela, esse atributo não poderá ficar vazio, sem nenhum valor atribuído a ele. Caso isso aconteça, ocorrerá um erro e a instrução de inserção não será realizada.

Já a restrição AUTO_INCREMENT define que o campo em questão terá a incrementação automática realizada diretamente pelo SGBD, garantindo, assim, que o valor não será repetido nos registros existentes na tabela.

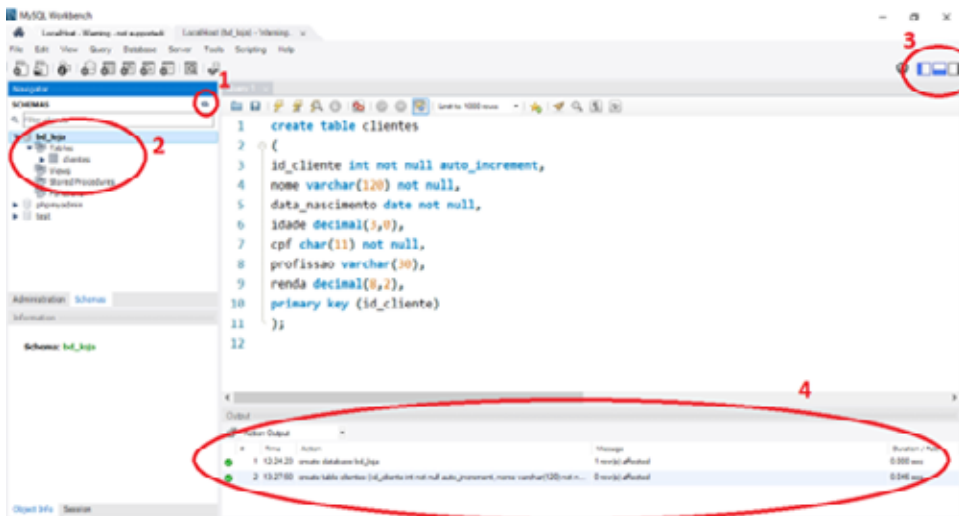


Figura 7 - Tela do editor de scripts SQL do MySQL Workbench utilizada para criar a tabela CLIENTES
Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Na parte central da tela, está escrito o comando CREATE TABLE, com todos os campos (atributos), os tipos de dados e as devidas restrições de integridade que compõem a tabela CLIENTES. À esquerda, é visualizada a lista de bancos de dados disponíveis no servidor Localhost. Há o BD_LOJA como o ativo, destacado com o nome em negrito nessa lista. Logo abaixo, em formato de lista de arquivos, é visualizada a tabela recém-criada chamada CLIENTES. Na parte inferior da tela, é visualizada uma seção chamada ACTION OUTPUT, que identifica o histórico das ações realizadas nesse banco de dados, como criar tabelas, banco de dados, dentre outras coisas. Fim da descrição.

O *script* de criação da tabela CLIENTES, após executado, fará com que ela esteja pronta para uso no banco de dados BD_LOJA. Para visualizá-la, clique no botão

de atualização (1) e essa tabela aparecerá na lista de *schemas* (2). No canto superior direito da tela (3), temos a opção de visualização de seções no Workbench. Uma opção interessante é a ACTION OUTPUT, que mostrará ao DBA todas as ações realizadas no banco de dados.

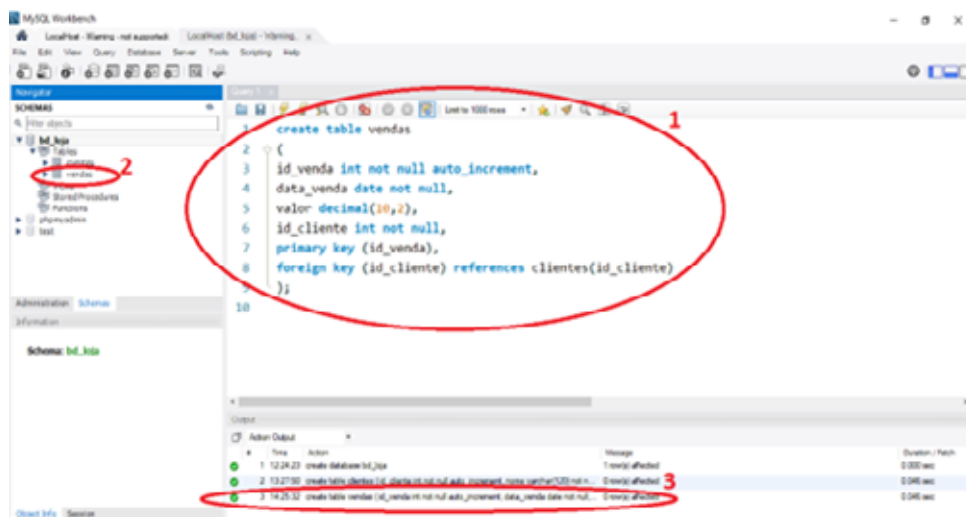


Figura 8 - Tela inicial do MySQL Workbench / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Na parte central da tela, está escrito o comando CREATE TABLE, com todos os campos (atributos), os tipos de dados e as devidas restrições de integridade que compõem a tabela VENDAS. À esquerda, é visualizada a lista de bancos de dados disponíveis no servidor Localhost. Há o BD_LOJA como o ativo, destacado com o nome em negrito nessa lista. Logo abaixo, em formato de lista de arquivos, é visualizada a tabela recém-criada chamada VENDAS. Na parte inferior da tela, é visualizada uma seção chamada ACTION OUTPUT, que identifica o histórico das ações realizadas nesse banco de dados, como criar tabelas, banco de dados, dentre outras coisas. Fim da descrição.

Seguindo o processo de criação, a instrução CREATE TABLE é utilizada para gerar a tabela VENDAS usando a mesma lógica da CLIENTES. A diferença é a inserção de um campo ID_CLIENTE, que será a referência entre as duas tabelas, ou seja, a chave estrangeira, permitindo, assim, o relacionamento entre elas.

O *script* (1) após a execução, fará com que a tabela VENDAS seja criada, permitindo a visualização na seção SCHEMAS (2), após a atualização dela. Na seção ACTION OUTPUT, é exibido o registro (*log*) da realização dessa ação (3) e é mostrado que ela foi executada com sucesso.

Após a criação do esquema, é possível inserir dados utilizando a instrução DML INSERT INTO e consultá-los a partir do SELECT. A manutenção do banco de dados envolve a atualização, a exclusão e a consulta de registros, sempre seguindo as boas práticas de modelagem relacional.

Nesse exemplo, agora, inseriremos dados nas duas tabelas, pois, quando criadas, elas se encontram completamente vazias, ou seja, sem nenhum registro inserido.

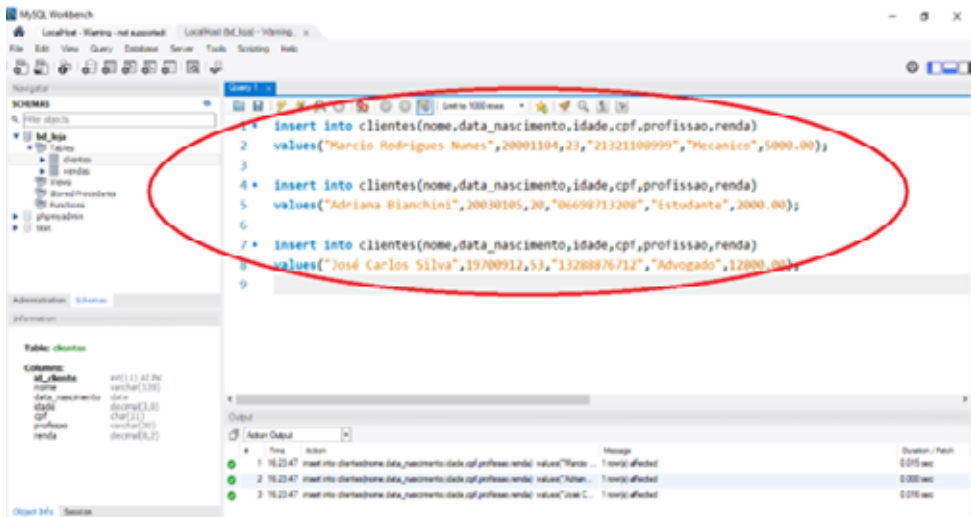


Figura 9 – Tela inicial do MySQL Workbench / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Na parte central da tela, está escrito o comando INSERT para realizar a inserção de dados na tabela CLIENTES. À esquerda, é visualizada a lista de bancos de dados disponíveis no servidor Localhost. Também há o BD_LOJA como o ativo, destacado com o nome em negrito nessa lista. Na parte inferior da tela, é visualizada uma seção chamada ACTION OUTPUT, que identifica o histórico das ações realizadas nesse banco de dados, como criar tabelas, banco de dados, dentre outras coisas. Fim da descrição.

O *script* de inserção de novos registros na tabela CLIENTES utiliza a instrução SQL DML INSERT. Nesse exemplo, é importante ressaltar que o campo ID_CLIENTE não aparece no código, devido ao fato de que ele foi definido durante a criação com a restrição AUTO_INCREMENT.

Já no campo DATA_NASCIMENTO, o valor inserido segue o padrão do tipo data existente no MySQL, que é AAAA-MM-DD, sem a necessidade de colocá-los separados, somente os números.

Nesse exemplo, foram inseridos 3 (três) registros diferentes na tabela CLIENTES. Para verificar os registros contidos nessa tabela, basta inserir a seguinte instrução SQL: **SELECT * FROM CLIENTES**

Assim, todos os registros existentes nessa tabela serão exibidos na tela.

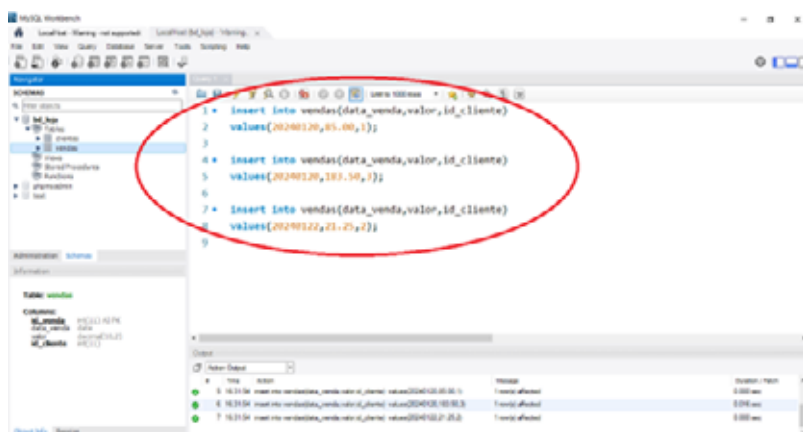


Figura 10 - Tela inicial do MySQL Workbench / Fonte: os autores.

Descrição da Imagem: é demonstrada a interface de escrita de scripts de código em linguagem SQL do MySQL Workbench. Na parte central da tela, está escrito o comando INSERT para realizar a inserção de dados na tabela VENDAS. À esquerda, é visualizada a lista de bancos de dados disponíveis no servidor Localhost. Há o BD_LOJA como o ativo, destacado com o nome em negrito nessa lista. Na parte inferior da tela, é visualizada uma seção chamada ACTION OUTPUT, que identifica o histórico das ações realizadas nesse banco de dados, como criar tabelas, banco de dados, dentre outras coisas. Fim da descrição.

Nesse exemplo, foram inseridos 3 (três) registros diferentes na tabela VENDAS. Para verificar os registros contidos nessa tabela, basta inserir a seguinte instrução SQL: **SELECT * FROM VENDAS**.

Assim, todos os registros existentes nessa tabela serão exibidos na tela.

É importante ressaltar que, devido ao fato de o campo ID_CLIENTE da tabela VENDAS ser uma chave estrangeira, o valor nele contido precisa ser um que esteja devidamente inserido na tabela CLIENTE.

A criação de um banco de dados relacional no MySQL requer um cuidadoso planejamento do esquema, a tradução desse esquema para instruções SQL, a criação de tabelas com chaves primárias e estrangeiras, e a inserção e a consulta conforme necessário. Essa abordagem estruturada proporciona um ambiente robusto e eficiente para armazenar e gerenciar dados de forma organizada.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Na trajetória acadêmica em Tecnologia da Informação (TI), os estudantes se deparam com uma diversidade de disciplinas que abrangem desde fundamentos de programação até conceitos avançados de redes e sistemas. Esse conhecimento multifacetado é como um quebra-cabeça, com cada peça representando uma habilidade ou conceito adquirido. Assim, é crucial que você compreenda como essas peças se encaixam, formando uma base sólida para enfrentar os desafios do futuro ambiente profissional de TI.

Ao longo do percurso acadêmico, você absorverá habilidades técnicas específicas, como linguagens de programação, design de banco de dados, administração de sistemas e segurança da informação. Essas habilidades se tornam ferramentas valiosas ao kit do profissional de TI, a fim de te capacitar para desenvolver soluções inovadoras e resolver problemas complexos no mundo real.

Além das habilidades técnicas, a capacidade de trabalhar em equipe, a comunicação eficaz e a compreensão dos aspectos éticos da tecnologia são aspectos fundamentais. A interconexão entre disciplinas, como a aplicação de princípios éticos na segurança da informação, destaca a importância de uma abordagem holística para a formação profissional em TI.

À medida que você se prepara para ingressar no ambiente profissional, é essencial antecipar as tendências emergentes na indústria de Tecnologia da Informação. Alguns conceitos, tais como computação em nuvem, Inteligência Artificial (IA), aprendizado de máquina e desenvolvimento sustentável, estão moldando o futuro do setor. Adaptar-se a essas mudanças e aprender continuamente são tão cruciais quanto o conhecimento adquirido durante a graduação.

Por fim, a jornada acadêmica em TI não é apenas uma acumulação de informações, mas uma construção de competências e perspectivas. Ao compreender como essas peças do quebra-cabeça se encaixam e ao olhar adiante para as demandas do ambiente profissional futuro, você está mais bem equipado(a) para enfrentar os desafios dinâmicos e contribuir significativamente para o avanço contínuo da área de TI.

AUTOATIVIDADE

1. As instruções SQL do tipo DDL (Data Definition Language) são as responsáveis por definir e gerenciar a estrutura do banco de dados, permitindo a criação, a alteração e a exclusão de objetos de banco de dados, tais como tabelas, índices, visões e procedimentos armazenados. Essas operações são essenciais para o design e a manutenção de bancos de dados, permitindo aos desenvolvedores controlar a organização e o formato dos dados armazenados.

Qual é o propósito da instrução SQL DDL CREATE TABLE?

- a) Alterar a estrutura de uma tabela existente.
 - b) Excluir todos os dados de uma tabela específica.
 - c) Criar uma nova tabela no banco de dados.
 - d) Selecionar registros de uma tabela.
 - e) Renomear uma tabela existente.
2. O desenvolvimento de um sistema de banco de dados é um processo multifásico que compreende diversas etapas cruciais. Inicia-se com a fase de projeto conceitual, na qual são identificados as entidades e os respectivos relacionamentos, resultando em um modelo de dados conceitual. Posteriormente, na fase de projeto lógico, o modelo conceitual é traduzido em um modelo lógico, especificando tabelas, chaves primárias e estrangeiras.

Considere as seguintes afirmativas sobre a criação de um banco de dados do tipo relacional:

- I - A instrução SQL DDL utilizada para criar uma tabela é CREATE TABLE.
- II - O relacionamento entre tabelas é estabelecido por meio da chave estrangeira.
- III - A instrução SQL DML para inserir dados em uma tabela é INSERT INTO.
- IV - A chave primária de uma tabela pode conter valores duplicados e redundantes.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

3. O processo de inserção de novos registros em um banco de dados relacional envolve a utilização da instrução SQL DML INSERT. Nessa operação, os dados são adicionados a uma tabela específica, sendo necessário fornecer os valores correspondentes para as colunas desejadas. A conformidade dos valores com os tipos de dados e as restrições definidas, como chaves primárias, é fundamental para a integridade dos dados. É possível inserir um único registro ou múltiplos registros em uma única operação, agilizando o processo de carga de dados. Ao realizar a inserção, é importante garantir a coerência dos dados, evitando duplicatas e mantendo a consistência do banco de dados relacional.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - O comando SQL DML INSERT permite apenas a inserção de um registro por vez em uma tabela.

PORQUE

II - A instrução SQL DML INSERT requer a especificação dos valores a serem inseridos para cada coluna da tabela.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

BARBOZA, F. F. M.; FREITAS, P. H. C. **Modelagem e desenvolvimento de banco de dados**. Porto Alegre: SAGAH, 2018.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

MACHADO, F. N. R.. **Banco de dados**: projeto e implementação. São Paulo: Érica, 2020.

PICHETTI, R. F.; VIDA, E. da S.; CORTES, V. S. M. P. **Banco de dados**. Porto Alegre: SAGAH, 2021.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

1. Opção C.

A instrução SQL DDL `CREATE TABLE` é utilizada para definir e criar uma nova tabela no banco de dados. Ela especifica os nomes das colunas, os tipos de dados e quaisquer restrições, como chaves primárias. Essa operação é fundamental para estruturar e organizar os dados de maneira eficiente.

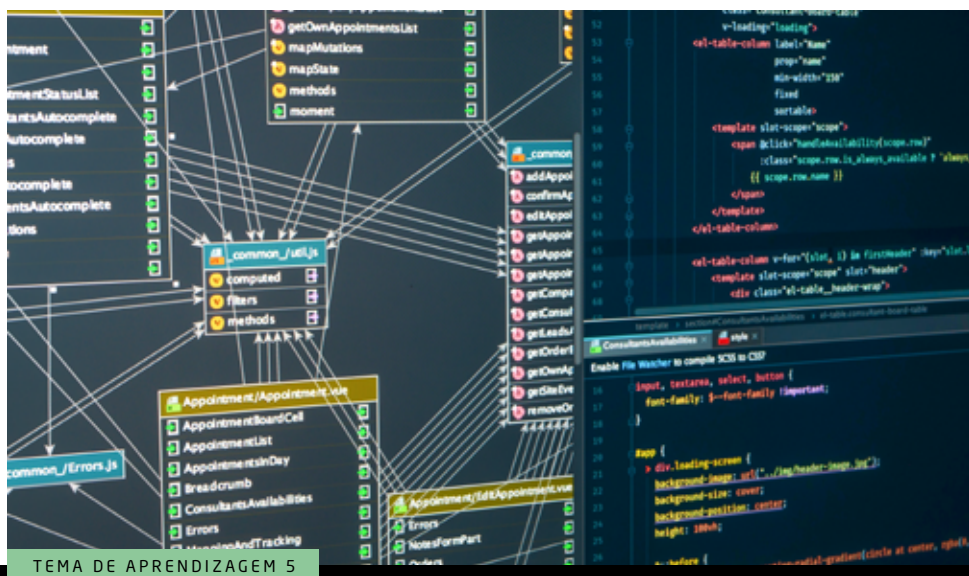
A instrução para alterar a estrutura de uma tabela é `ALTER TABLE`, e não `CREATE TABLE`. A exclusão de todos os dados de uma tabela é realizada com a instrução `DELETE`, e não `CREATE TABLE`. A seleção de registros de uma tabela é feita com a instrução `SELECT`, e não `CREATE TABLE`. Renomear uma tabela é feito com a instrução `ALTER TABLE RENAME`, e não `CREATE TABLE`.

2. Opção D.

O comando SQL DDL utilizado para criar uma tabela é `CREATE TABLE`. Ele é fundamental para a definição da estrutura da tabela, especificando os nomes das colunas, os tipos de dados e as restrições. O relacionamento entre tabelas é estabelecido por meio da chave estrangeira, que referencia a chave primária de outra tabela. Essa prática contribui para a integridade referencial do banco de dados. O comando SQL DML para inserir dados em uma tabela é `INSERT INTO`. Essa instrução é utilizada para adicionar novos registros à tabela. A chave primária de uma tabela não pode conter valores duplicados, sendo necessariamente única para cada registro.

3. Opção E.

A instrução SQL DML `INSERT` permite a inserção de um ou mais registros de uma vez, não estando limitada à inserção de apenas um registro por vez, o que torna a primeira asserção falsa. A segunda asserção também é falsa, pois é possível realizar a inserção de registros em uma tabela sem especificar valores para todas as colunas. Se os valores não forem fornecidos para todas as colunas, os valores padrões ou nulos (se permitidos) serão utilizados.



TEMA DE APRENDIZAGEM 5

CRIANDO UM BANCO DE DADOS E A LINGUAGEM SQL

MINHAS METAS

- Compreender a linguagem SQL.
- Aprender a projetar esquemas de banco de dados relacionais eficientes.
- Adquirir a habilidade de modelagem de dados utilizando ferramentas, como o Modelo Entidade-Relacionamento (ER).
- Aprimorar as habilidades na criação de consultas otimizadas utilizando índices, cláusulas WHERE e JOINS.
- Compreender os conceitos de transações e controle de concorrência.
- Aprender a automatizar e otimizar consultas.
- Expandir os conhecimentos para além da SQL, explorando tecnologias relacionadas.

INICIE SUA JORNADA

Inicie a sua jornada rumo à maestria em Tecnologia da Informação (TI) com a emocionante aplicação prática dos conceitos aprendidos na linguagem SQL para administração e gerenciamento de bancos de dados relacionais. Este é o momento crucial em que os conhecimentos teóricos se transformarão em experiência concreta e você terá a oportunidade de projetar e construir estruturas de dados sólidas e eficientes. Prepare-se para mergulhar nos bastidores do universo dos bancos de dados, dando vida aos modelos conceituais por meio da criação de tabelas, inserção de novos registros e execução de consultas, consolidando, assim, as suas habilidades em SQL.

Durante essa jornada prática, cada linha de código será um passo em direção ao domínio da linguagem SQL. Entender profundamente como os dados são manipulados e armazenados proporcionará uma base sólida para se tornar um(a) especialista na área. Este é um convite para explorar a arte da modelagem de dados, transformando-se em um(a) arquiteto(a) de conhecimento capaz de projetar estruturas que não apenas armazenam informações, mas também sustentam sistemas robustos e eficientes. Ao abraçar essa jornada, você estará moldando o futuro da TI com as suas habilidades práticas e conhecimentos valiosos.

À medida que avança, não apenas estará construindo bancos de dados, mas também a sua própria trajetória profissional. Desenvolva não apenas competências técnicas, mas também habilidades analíticas e uma mentalidade criativa que te diferenciarão no vasto campo da TI. Prepare-se para transformar a sua paixão por tecnologia em uma carreira significativa e impactante. A jornada está apenas começando: o futuro aguarda as suas contribuições. Avance e inicie essa caminhada que promete não apenas desafios, mas também conquistas gratificantes e um crescimento profissional extraordinário.

PLAY NO CONHECIMENTO

Dê um play no conhecimento com o podcast *Desbravando Dados*, a sua fonte de *insights* e descobertas fascinantes sobre a utilização da linguagem SQL na administração e no gerenciamento de bancos de dados relacionais. Em cada episódio, mergulharemos fundo nesse universo dinâmico, explorando desde os fundamentos até as estratégias avançadas para tirar o máximo proveito dessa ferramenta poderosa. Se você é um(a) entusiasta da Tecnologia da Informação (TI), um(a) estudante ávido(a) por aprendizado ou um(a) profissional que busca aprimorar as suas habilidades em SQL, este é o podcast feito para você.

Ao longo de nossa jornada auditiva, desmistificaremos conceitos, compartilharemos dicas práticas e discutiremos sobre os profissionais da área, que desbravam diariamente os desafios e as inovações no campo dos bancos de dados relacionais. Prepare-se para uma imersão de conhecimento que vai além do tradicional, conectando teoria e prática de maneira envolvente. Seja você um(a) iniciante curioso(a) ou um(a) experiente programador(a) SQL, *Desbravando Dados* é o seu guia para o caminho da maestria. Suba a bordo, aperte o *play* e junte-se a nós nesta jornada apaixonante pelo fascinante mundo dos dados.

VAMOS RECORDAR?

O armazenamento de dados é um processo de coleta, organização e retenção de informações de forma sistemática para posterior uso, abrangendo diferentes métodos, desde arquivamento físico até sistemas digitais modernos. Um banco de dados relacional é uma estrutura organizada que grava dados em tabelas ligadas entre si, conectadas por chaves, utilizando um modelo para representar e manipular esses dados, oferecendo flexibilidade e eficiência na gestão de informações. Permite, desse modo, a realização de consultas complexas.

O modelo relacional é utilizado em aplicações comerciais, acadêmicas e governamentais, dado que apresenta a capacidade de lidar com grandes volumes de informações de forma eficiente e segura, sendo fundamental para armazenar dados de forma organizada, garantindo consistência e integridade.

Além do mais, permite a realização eficaz de consultas complexas e a manipulação, contribuindo para a tomada de decisões e o desenvolvimento de aplicações robustas. O armazenamento de dados e o modelo relacional desempenham um papel crucial na gestão e na utilização eficiente de informações em diversas áreas, proporcionando uma base sólida para o gerenciamento, permitindo fácil recuperação, atualização e análise, o que é essencial para o funcionamento eficaz de sistemas em um mundo cada vez mais digitalizado.

https://www.youtube.com/watch?v=E-2Ci1c_BXs

DESENVOLVA SEU POTENCIAL

OTIMIZAÇÃO DE UM BANCO DE DADOS RELACIONAL

Criar um banco de dados do tipo relacional utilizando linguagem SQL envolve uma série de etapas essenciais. Inicialmente, é crucial definir o modelo que apresentará as entidades e os respectivos relacionamentos.

Após a criação das tabelas, é necessário estabelecer os relacionamentos entre elas. Para isso, são usadas as chaves estrangeiras (FK) para vincular os registros de uma tabela com os registros de outras, sendo eles devidamente identificados a partir da chave primária (PK). Isso permite representar conexões significativas entre os dados, garantindo, assim, a consistência e a integridade referenciais.

No contexto dos bancos de dados relacionais, segundo Machado (2020), a utilização de JOINS é fundamental, porque permite combinar dados armazenados em diferentes tabelas em uma única consulta.

Existem tipos específicos, como INNER JOIN, LEFT

JOIN e RIGHT JOIN, cada um adequado para diferentes necessidades, seguindo princípios matemáticos. Essas operações são cruciais para extrair informações completas de conjuntos de dados distribuídos em várias tabelas.

Outro conceito avançado é o uso de VIEWS, que podem ser descritas como uma consulta armazenada que pode ser tratada como uma **tabela virtual**, permitindo simplificar consultas complexas, ao criar uma “visão” específica dos dados, facilitando a análise e a recuperação de informações específicas sem a necessidade de escrever consultas longas e repetitivas.

Finalmente, os TRIGGERS são blocos de código SQL que são automaticamente executados em resposta a eventos específicos no banco de dados, como criar uma estrutura para atualizar automaticamente a quantidade de estoque de um livro sempre que uma venda for registrada. Isso ajuda a manter a consistência e a integridade dos dados de maneira automatizada.

JOINS

São fundamentais para unificar informações provenientes de duas ou mais tabelas com base em colunas relacionadas entre si. Essa capacidade é essencial em um sistema de banco de dados relacional, pois permite que os dados sejam armazenados de forma normalizada, reduzindo redundâncias e melhorando a integridade.

Ao mesmo tempo, os JOINS, segundo Silberschatz (2020), possibilitam a reconstrução dessas informações separadas em consultas complexas, fornecendo uma visão completa e detalhada dos dados necessários para análises ou relatórios específicos.

Existem vários tipos de JOINS, cada um servindo a um propósito distinto dependendo da natureza da informação que se deseja extrair. O mais comum é o INNER JOIN, que retorna apenas os registros que têm correspondência em ambas as tabelas. Por outro lado, os OUTER JOINS, que podem ser classificados como LEFT, RIGHT ou FULL, são utilizados para incluir registros que não têm correspondência em uma das tabelas, permitindo uma análise mais abrangente dos dados. Essa flexibilidade faz dos JOINS

**Facilitando a análise
e a recuperação
de informações
específicas**

uma ferramenta poderosa para resolver uma ampla gama de necessidades de consulta.

Para executar um JOIN, é necessário que exista uma chave primária em uma tabela e a respectiva chave estrangeira em outra, estabelecendo, assim, um relacionamento entre elas. Por exemplo, em uma base de dados de uma loja, uma tabela de “Clientes” pode ser relacionada a uma outra de “Pedidos” mediante o atributo ID do cliente, sendo ele presente em ambas as tabelas. Ao realizar um INNER JOIN, segundo Alves (2021), é possível facilmente obter uma lista de todos os pedidos feitos por cada cliente, combinando os registros com base no ID do cliente.

A sintaxe para a execução em SQL é intuitiva, começando com a seleção das tabelas a serem unidas, especificando o tipo de JOIN a ser realizado e a condição que relaciona essas tabelas.

No entanto, segundo Elmasri e Navathe (2018), é importante salientar que o uso extensivo de JOINS em consultas complexas ou em bancos de dados com volumes massivos de dados pode impactar a performance do sistema. Planejar adequadamente a estrutura do banco, otimizar as consultas e considerar a utilização de índices podem ajudar significativamente a mitigar potenciais problemas de desempenho.

Apesar desses desafios, a capacidade de unir registros de tabelas relacionadas é indispensável para extrair *insights* significativos e apoiar a tomada de decisões baseada em dados complexos e relacionados.

Consideremos um exemplo prático para ilustrar o conceito de JOINS: utilizemos o cenário de uma livraria que mantém registros de livros e vendas em duas tabelas separadas.

- **LIVROS:**
- ID_Livro [PK]
- Título
- Autor
- **VENDAS:**
- ID_Venda [PK]
- ID_Livro [FK]
- Data
- Quantidade

Suponha que é necessário gerar um relatório que liste todos os livros que foram vendidos, incluindo os seguintes atributos: título de cada livro, a data da venda e a quantidade vendida. Para isso, é preciso realizar um INNER JOIN entre as tabelas utilizando o ID_Livro que é comum entre ambas. Seguindo essa lógica, a consulta SQL para gerar esse relatório seria:

```
SELECT Livros.Titulo, Vendas.Data, Vendas.Quantidade  
FROM Livros  
INNER JOIN Vendas ON Livros.ID_Livro = Vendas.ID_Livro
```

Figura 1 - Instrução SQL para exemplificar o uso da cláusula INNER JOIN / Fonte: os autores.

Descrição da Imagem: é exibida uma tela preta representando a tela de prompt do computador. Ela ilustra a instrução SQL SELECT com três linhas de código, sendo a última a cláusula INNER JOIN, que buscará dados em duas tabelas diferentes: VENDAS e LIVROS. Fim da descrição.

Essa consulta SQL efetivamente combina os registros das tabelas LIVROS e VENDAS em que os IDs dos livros correspondem, resultando em uma lista de vendas que inclui os atributos solicitados. Graças ao JOIN, é possível facilmente cruzar as informações de diferentes tabelas para produzir relatórios complexos e detalhados que seriam difíceis de se obter trabalhando com cada uma dessas tabelas sendo consultadas de forma isolada.

Por fim, esse exemplo prático demonstra o poder dos JOINS para conectar dados devidamente relacionados, permitindo análises e relatórios que exigem informações de múltiplas tabelas.

Segundo Silberschatz (2020), é uma técnica indispensável para profissionais que trabalham com bancos de dados, oferecendo uma maneira flexível e eficiente de acessar e combinar dados conforme necessário.

VIEWS

São estruturas que representam uma abstração poderosa que permite aos usuários visualizar e interagir com os dados de maneira simplificada, sem alterar as tabelas subjacentes.

Uma VIEW (visão) é essencialmente uma consulta SQL armazenada que é executada para produzir um conjunto de resultados que pode ser tratado como uma **tabela virtual**. Esse conceito facilita o acesso a dados complexos, permitindo definir perspectivas específicas sobre os registros que são mais relevantes para determinadas operações ou análises, sem a necessidade de duplicar ou alterar os valores originais.

Criar uma VIEW, segundo Pichetti, Vida e Cortes (2021), envolve a execução de uma instrução CREATE VIEW seguida por uma consulta SQL que define a lógica de seleção de dados. Essa consulta pode combinar várias tabelas, utilizar funções, filtros e realizar JOINS, assim como qualquer outra consulta.

No entanto, ao contrário de executar a consulta cada vez que os registros são necessários, a visão armazena essa definição, permitindo aos usuários acessá-la com uma simplicidade comparável à de consultar uma única tabela. Isso não apenas economiza tempo e esforço, especialmente em operações repetitivas, mas também ajuda a manter a consistência nas análises de dados realizadas por diferentes usuários ou aplicações.

Além de simplificar a acessibilidade, a VIEW também desempenha um papel crucial na questão relativa à segurança dos dados. Ao ser criada, é possível restringir o acesso a certas colunas ou linhas nas tabelas subjacentes, expondo apenas o que for necessário para o usuário ou aplicação específica. Isso é particularmente útil em ambientes em que múltiplos usuários acessam o mesmo banco de dados, mas nem todos devem ter visibilidade completa ou permissões para modificar todos os registros. Uma visão pode atuar como uma camada de abstração que protege dados sensíveis ou restritos.

Um aspecto importante em uma VIEW é o fato de ela ser dinâmica. Segundo Barboza e Freitas (2018), quando os dados nas tabelas originais são atualizados, as alterações são refletidas automaticamente na VIEW que acessa esses registros, garantindo, assim, o acesso à versão mais atual dos dados, proporcionando uma base consistente para relatórios e análises sem a necessidade de uma manutenção adicional em si. Em outras palavras, trata-se de uma opção extremamente útil para a criação de *dashboards* dinâmicos e relatórios em tempo real.

No entanto, é essencial entender que, apesar das vantagens, utilizar VIEWS não proporciona apenas benefícios. O desempenho é uma consideração crucial em um sistema de software e pode ser totalmente comprometido quando houver a necessidade de executar buscas complexas em grandes volumes de dados. A otimização de consultas subjacentes às VIEWS e a estruturação adequada dos índices nas tabelas originais podem mitigar problemas de performance. Por isso, é fundamental que o DBA construa bem o banco de dados.

Para exemplificar esse conceito, consideremos um banco de dados de uma escola que armazena informações sobre ALUNOS, PROFESSORES e TURMAS, conforme a definição exibida a seguir:

- **ALUNOS:**

ID

Nome

Ano

- **PROFESSORES:**

ID

Nome

Disciplina

- **TURMAS:**

ID

ID_Professor

ID_Aluno

Disciplina

Suponha que é necessário gerar frequentemente relatórios sobre quais alunos estão em quais turmas e quem são os respectivos professores. Todavia, o intuito é simplificar o acesso a essas informações mais complexas, sem expor diretamente as tabelas subjacentes.

Nesse contexto, é possível criar uma VIEW que será denominada *VisaoTurmasAlunos*: ela unirá essas informações de maneira lógica e acessível. A instrução SQL para se realizar essa ação seria:

```
CREATE VIEW VisaoTurmasAlunos AS
SELECT Alunos.Nome AS NomeAluno, Alunos.Ano, Professores.Nome AS NomeProfessor,
Turmas.Disciplina
FROM Turmas
JOIN Alunos ON Turmas.ID_Aluno = Alunos.ID
JOIN Professores ON Turmas.ID_Professor = Professores.ID;
```

Figura 2 - Instrução SQL para exemplificar o uso da cláusula JOIN / Fonte: os autores.

Descrição da Imagem: é exibida uma tela preta representando a tela de prompt do computador. Ela ilustra a instrução SQL SELECT com cinco linhas de código. As duas últimas exibem a cláusula JOIN, que buscará dados em duas tabelas diferentes: ALUNOS e PROFESSORES. Fim da descrição.

Assim, *VisaoTurmasAlunos* permitirá que qualquer usuário autorizado possa ver, de forma simplificada, quais alunos estão em cada turma, quem são os professores e a disciplina correspondente, sem a necessidade de se executar múltiplos JOINS sempre que essa informação for necessária. Isso não apenas facilita a geração de relatórios e consultas para fins administrativos ou acadêmicos, mas também protege a integridade das tabelas originais, evitando alterações acidentais ou acessos não autorizados.

Além disso, se houver atualizações em qualquer uma das tabelas, essas mudanças serão automaticamente refletidas em *VisaoTurmasAlunos*, sem a necessidade de qualquer intervenção adicional. Isso assegura que os usuários sempre tenham acesso às informações mais recentes, mantendo a VIEW mais relevante e atualizada.

A utilização de VIEWS, assim como no exemplo exposto, demonstra a eficácia dela em simplificar o acesso a dados complexos, reforçar a segurança ao limitar a visibilidade direta das tabelas e garantir que as informações apresentadas estejam sempre atualizadas. Isso ilustra como as VIEWS podem servir como uma ferramenta valiosa para as organizações que necessitam de um meio eficiente e seguro para acessar o banco de dados.

STORED PROCEDURES

São um conjunto poderoso de instruções SQL que são salvas e armazenadas no próprio servidor do banco de dados para que possam ser executadas conforme necessário, funcionando como as funções em linguagens de programação,

permitindo o encapsulamento de operações complexas em um único bloco de código que pode ser reutilizado diversas vezes. Isso não apenas economiza tempo, evitando a reescrita de consultas complexas, mas também ajuda a manter a consistência do código e da lógica de negócios em toda a aplicação.

Uma das principais vantagens é a capacidade de executar operações que envolvem várias etapas a partir de uma única chamada ao banco de dados. Por exemplo, uma Stored Procedure pode ser escrita para inserir dados em várias tabelas, atualizar registros com base em certas condições e, ao mesmo tempo, realizar verificações de integridade ou cálculos complexos. Isso é particularmente útil para manter a atomicidade das transações, garantindo que uma série de operações seja concluída com sucesso como uma unidade única de trabalho ou que nenhuma dessas seja realizada em caso de erro.

Outro benefício significativo, segundo Silberschatz (2020), é em relação à segurança. Ao encapsular a lógica de negócios dentro do banco de dados, o acesso direto às tabelas e dados é limitado, reduzindo o risco de injeção de SQL e outros vetores de ataque. Além do mais, é possível definir permissões específicas para a execução da Stored Procedure, controlando quem pode executar operações sensíveis. Isso é muito importante em ambientes com múltiplos usuários e aplicações acessando simultaneamente o mesmo banco de dados.

Do ponto de vista do desempenho também existem vantagens, uma vez que o código, ao ser compilado e armazenado no banco de dados, tende a ser executado mais rapidamente do que as consultas SQL enviadas diretamente pelo aplicativo. Isso ocorre, porque o plano de execução da consulta é gerado e otimizado no momento da criação da Stored Procedure. Além disso, a necessidade de enviar menos dados pela rede (apenas os parâmetros necessários) pode reduzir significativamente a latência de rede, melhorando a experiência do usuário em aplicações dependentes de acesso frequente ao banco de dados.

Por fim, uma Stored Procedure facilita a manutenção do sistema, pois, ao centralizar a lógica de acesso e manipulação do banco de dados, qualquer mu-

dança necessária, seja por ajustes na lógica de negócios, seja por otimização de desempenho, pode ser feita em um único local. Isso elimina a necessidade de atualizar e redistribuir o aplicativo cada vez que uma mudança é feita, simplificando o processo de manutenção e garantindo que todos os usuários se beneficiem imediatamente das melhorias implementadas.

Essa abordagem, segundo Barboza e Freitas (2018), centralizada e modular para o processamento de dados representa uma prática robusta de design de sistemas alinhada com os princípios de desenvolvimento de software eficiente e seguro.

Vamos considerar um exemplo prático de uma Stored Procedure em um banco de dados de uma empresa que gerencia informações de empregados e departamentos. Essa empresa deseja inserir um novo empregado no sistema e, ao mesmo tempo, atualizar o número total de empregados no departamento correspondente. Esse cenário envolve operações em duas tabelas diferentes: Empregados e Departamentos.

Primeiro, observe a estrutura simplificada das tabelas:

- **Empregados:**

ID_Empregado

Nome

ID_Departamento

- **Departamentos:**

ID_Departamento

Nome_Departamento

Total_Empregados

A Stored Procedure será criada para automatizar o processo de adicionar um novo empregado e atualizar o total de empregados no departamento correspondente. A seguir, é exibido um exemplo de como essa ação poderá ser realizada via código SQL.

```

CREATE PROCEDURE AdicionarEmpregado
@Nome NVARCHAR(100),
@ID_Departamento INT

AS
BEGIN
    — Inserir novo registro na tabela EMPREGADOS
    INSERT INTO Empregados (Nome, ID_Departamento)
    VALUES (@Nome, @ID_Departamento);

    — Atualizar o total de empregados no departamento correspondente
    UPDATE Departamentos
    SET Total_Empregados = Total_Empregados + 1
    WHERE ID_Departamento = @ID_Departamento;
END;

```

Figura 3 - Instrução SQL para exemplificar a criação de uma STORED PROCEDURE / Fonte: os autores.

Descrição da Imagem: é exibida uma tela preta representando a tela de prompt do computador. Ela ilustra a instrução SQL CREATE PROCEDURE, criando a Stored Procedure chamada ADICIONAREMPREGADO, com duas variáveis locais: NOME e DEPARTAMENTO. São duas instruções SQL realizadas: INSERT, na tabela EMPREGADOS, e UPDATE, na tabela DEPARTAMENTOS. Fim da descrição.

Para executar essa Stored Procedure e adicionar um novo empregado ao departamento de TI, o comando seria:

```
EXEC AdicionarEmpregado @Nome = 'João Silva', @ID_Departamento = 3;
```

Figura 4 - Instrução SQL para exemplificar a execução da Stored Procedure AdicionarEmpregado / Fonte: os autores.

Descrição da Imagem: é exibida uma tela preta representando a tela de prompt do computador. Ela ilustra a instrução SQL EXEC com uma linha de código para executar, no servidor do banco de dados, a STORED PROCEDURE AdicionarEmpregado, com os valores referentes a NOME e ID_DEPARTAMENTO. Fim da descrição.

Nesse exemplo, *AdicionarEmpregado* realiza duas operações fundamentais em uma única chamada: insere um registro na tabela EMPREGADOS e atualiza o contador de empregados em DEPARTAMENTOS. Isso não apenas simplifica o

processo de inserção de novos registros no sistema, eliminando a necessidade de executar manualmente múltiplas consultas SQL, mas também garante a consistência dos dados, ao manter atualizado o total de empregados de cada departamento.

Além disso, encapsular essas operações em uma Stored Procedure aumenta a segurança, reduzindo a exposição direta às tabelas e permitindo um controle mais refinado sobre quem pode executar essas operações no banco de dados.

TRIGGERS

São blocos de código SQL que são automaticamente executados em resposta a eventos específicos no banco de dados, funcionando como mecanismos de **gatilho**, respondendo a alterações nos dados ou no esquema da tabela. Esses eventos podem incluir a inserção, a atualização ou a exclusão de registros em uma tabela, proporcionando uma maneira eficiente de automatizar ações e manter a integridade e consistência dos dados.

A utilidade do TRIGGER abrange diversas áreas, desde a validação de dados até a automação de tarefas complexas, podendo ser classificados em BEFORE e AFTER, indicando se a ação deve ocorrer antes ou depois do evento que a desencadeou (gatilho). No caso de um TRIGGER BEFORE INSERT, por exemplo, valida-se se há estoque suficiente de um produto antes de permitir a inserção de um novo registro de venda dele. Essa abordagem permite a prevenção de ações indesejadas antes de afetarem o banco de dados.

A flexibilidade de um TRIGGER, segundo Silberschatz (2020), também se estende à possibilidade de ele ser aplicado em nível de linha ou em nível de instrução. No primeiro caso, ele é acionado para cada linha afetada pela ação que o desencadeia. Já no segundo caso, é acionado uma vez, independentemente do número de linhas afetadas. Esse conceito oferece controle granular sobre como as ações são realizadas, adaptando-se às necessidades específicas de cada contexto.

**ZOOM NO CONHECIMENTO**

Qual é a diferença entre STORED PROCEDURE e TRIGGER? A principal diferença entre esses conceitos é que o STORED PROCEDURE precisa ser chamado para ser executado. Já o TRIGGER, uma vez criado, sempre será executado quando o gatilho for disparado.

Entretanto, é importante usar o TRIGGER com bastante cautela, porque, se mal implementado, pode impactar negativamente o desempenho do banco de dados. A execução de tarefas complexas ou demoradas dentro desse TRIGGER, segundo Machado (2020), pode resultar em lentidão nas operações do sistema. Portanto, é crucial equilibrar a automação proporcionada com a consideração cuidadosa do impacto na performance.

É possível afirmar que o TRIGGER é uma ferramenta valiosa que permite a automação de ações em resposta a eventos específicos, seja garantindo a integridade, seja realizando validações, seja simplificando tarefas rotineiras. A implementação cuidadosa dele contribui para a eficiência e a consistência operacional do sistema de banco de dados.

Para ilustrar esse conceito, vamos considerar um cenário prático em uma loja virtual. Nesse exemplo, busca-se garantir que sempre que um produto seja vendido, a quantidade em estoque desse produto seja automaticamente atualizada, diminuindo a quantidade vendida do estoque disponível.

Estrutura das Tabelas:

Produtos

ID_Produto (Chave Primária)

Nome

Preco

Estoque

ID_Produto (Chave Estrangeira referenciando Produtos.ID_Produto)

QuantidadeDisponivel

Vendas:

ID_Venda (Chave Primária)

ID_Produto (Chave Estrangeira referenciando Produtos.ID_Produto)

QuantidadeVendida

DataVenda

É válido criar um TRIGGER que, após a inserção de um novo registro na tabela VENDAS, automaticamente atualize a *QuantidadeDisponivel* em ESTOQUE para o *ID_Produto* correspondente.

```
DELIMITER //
```

```
CREATE TRIGGER AtualizaEstoqueAposVenda
AFTER INSERT ON Vendas
FOR EACH ROW
BEGIN
    UPDATE Estoque
    SET QuantidadeDisponivel = QuantidadeDisponivel - NEW.QuantidadeVendida
    WHERE ID_Produto = NEW.ID_Produto;
END;
```

```
DELIMITER ;
```

Figura 5 - Representação das etapas do ciclo de desenvolvimento de um banco de dados / Fonte: os autores.

Descrição da Imagem: são demonstradas as etapas do ciclo. Ordenadas da esquerda para a direita: a primeira, na cor rosa, à esquerda, é chamada de "Análise de Requisitos". A segunda, logo à direita, na cor cinza é a "Modelagem". A terceira, mais à direita, é a "Implementação", na cor amarela. Em seguida, na cor azul, encontra-se a etapa "Testes". Por fim, a última etapa do ciclo, mais à direita, na cor verde, é a "Manutenção Contínua". Fim da descrição.

Vamos destrinchar essa instrução SQL:

- **DELIMITER //** e **DELIMITER ;** são utilizados para mudar de forma temporária o delimitador padrão do SQL (*que é ;*), permitindo, assim, que o corpo do TRIGGER contenha ; sem encerrar prematuramente a sua definição.

- **CREATE TRIGGER AtualizaEstoqueAposVenda:** inicia a criação de um TRIGGER chamado *AtualizaEstoqueAposVenda*.
- **AFTER INSERT ON Vendas:** define que o TRIGGER será acionado após a inserção de registros na tabela *Vendas*.
- **FOR EACH ROW:** especifica que o TRIGGER será executado para cada linha inserida na tabela *Vendas*.
- **BEGIN ... END;** delimita o bloco de instruções SQL a serem executadas pelo TRIGGER.

Dentro do bloco *BEGIN ... END;*, a instrução *UPDATE* ajusta a *QuantidadeDisponivel* em *ESTOQUE*, subtraindo a *QuantidadeVendida* da venda recém inserida (*NEW.QuantidadeVendida*). *NEW* é uma palavra-chave especial que se refere aos valores da nova linha inserida na tabela que disparou o TRIGGER.

Esse exemplo prático demonstra como esse conceito pode ser utilizado para manter a integridade e a consistência dos dados em um banco de dados, automatizando tarefas que, de outra forma, exigiriam intervenção manual ou lógica adicional no lado da aplicação.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

A jornada de aprendizado em Tecnologia da Informação (TI), especialmente na esfera dos bancos de dados relacionais e linguagem SQL, prepara os estudantes para enfrentar e resolver problemas complexos no ambiente profissional moderno. O conhecimento adquirido sobre criação, manuseio e otimização de BD não é apenas uma habilidade técnica: é uma porta de entrada para entender como as organizações gerenciam e interpretam grandes volumes de dados para tomar decisões estratégicas.

Ao dominar a arte de estruturar um banco de dados, você se torna apto(a) a implementar sistemas robustos que servem como a espinha dorsal de aplicações críticas

nos negócios. A habilidade de utilizar SQL para criar, consultar e manipular dados permite que você transforme requisitos complexos de negócios em soluções eficientes. Mais ainda: ao aplicar conceitos avançados, como JOINS, VIEWS e TRIGGERS, você aprende a otimizar consultas para melhor desempenho e a automatizar tarefas essenciais, elevando a eficiência e a precisão dos sistemas de informação.

No ambiente profissional, a capacidade de integrar dados de múltiplas fontes mediante JOINS e de criar VIEWS para simplificar e otimizar o acesso a informações específicas são habilidades inestimáveis, permitindo que você desenvolva relatórios detalhados, *dashboards* interativos e funcionalidades analíticas avançadas que suportam a tomada de decisão estratégica. Por outro lado, a implementação de TRIGGERS possibilita a automação de processos, como atualizações de estoque, registros de *log* e alertas em tempo real, o que é fundamental para a manutenção da integridade e da segurança dos dados.

Além disso, o conhecimento profundo em bancos de dados e SQL abre caminhos para áreas de especialização, como *Data Science*, *Business Intelligence* e Desenvolvimento de Aplicações, em que a manipulação e a análise de dados são essenciais. A capacidade de extrair *insights* valiosos, combinada com uma compreensão sólida de como estruturar e otimizar o armazenamento de dados, coloca o(a) profissional de TI em uma posição estratégica dentro das organizações.

Portanto, ao conectar os conceitos de criação e otimização de bancos de dados relacionais às demandas do ambiente profissional, você se equipa não apenas com habilidades técnicas, mas com uma visão estratégica. Essa combinação é fundamental para enfrentar os desafios da era digital, permitindo que você contribua significativamente para o crescimento e a inovação nas organizações em que atua.

AUTOATIVIDADE

1. O Administrador de Banco de Dados (DBA) desempenha um papel crucial na estruturação de um sistema de banco de dados do tipo relacional. A responsabilidade dele abrange desde o design inicial do esquema, garantindo uma modelagem eficiente e normalizada, até a implementação de políticas de segurança e otimização de desempenho. Esse profissional é encarregado de criar e manter as tabelas, índices e relacionamentos, visando à integridade e à eficiência dos dados. Além disso, realiza tarefas essenciais, como o ajuste de consultas, o gerenciamento de backups e a implementação de estratégias de recuperação em caso de falhas. Logo, o DBA desempenha um papel central na garantia da disponibilidade, confiabilidade e desempenho do sistema de banco de dados, assegurando que ele atenda de forma eficaz e segura às necessidades da organização.

Qual é a principal razão para se estruturar corretamente um banco de dados do tipo relacional?

- a) Facilitar a leitura de consultas SQL.
 - b) Minimizar a redundância de dados.
 - c) Aumentar a complexidade das consultas.
 - d) Agilizar a instalação do sistema.
 - e) Reduzir a necessidade de backups.
2. A otimização de um banco de dados relacional diz respeito ao conjunto de práticas e estratégias aplicadas para melhorar o desempenho, a eficiência e o tempo de resposta nas operações. Isso envolve a análise e o ajuste de vários elementos, como índices, estrutura das consultas, configurações do sistema e distribuição de dados. O objetivo principal é maximizar a eficácia das consultas, minimizar o tempo de execução e reduzir a carga sobre o sistema, proporcionando uma experiência mais ágil e eficiente para os usuários. A otimização contínua é essencial para lidar com o crescimento do banco de dados e garantir que ele atenda de maneira eficaz às demandas operacionais, mantendo-se alinhado com os requisitos de desempenho da aplicação.

Assinale a alternativa que melhor descreve o conceito de Stored Procedures em um banco de dados do tipo relacional:

- a) São consultas SQL utilizadas para recuperar dados de múltiplas tabelas.
- b) Representa scripts executáveis ao lado do cliente para otimizar a interface gráfica do usuário.
- c) Consiste em blocos de código SQL armazenados no banco de dados para execução posterior.
- d) É utilizado exclusivamente para a criação de índices em tabelas.
- e) Refere-se às operações de criação e modificação de tabelas no banco de dados.

AUTOATIVIDADE

3. A maestria no conceito de VIEWS é crucial para um Administrador de Banco de Dados (DBA) em ambientes relacionais, pois elas desempenham um papel estratégico na facilitação da manipulação e da recuperação eficiente de dados. Ao criar VIEWS, o DBA pode simplificar consultas complexas, encapsulando a lógica de acesso aos dados e oferecendo uma camada abstrata que oculta a complexidade subjacente das tabelas. Essa capacidade não apenas aprimora a eficiência nas operações de consulta, mas também permite a implementação de políticas de segurança, restringindo o acesso a determinadas colunas ou linhas. Além disso, as VIEWS proporcionam uma visão personalizada dos dados, contribuindo para uma gestão mais flexível e adaptada às necessidades específicas dos usuários e da aplicação.

Considere as seguintes afirmativas sobre o conceito de VIEWS em um Banco de Dados Relacional:

- I - VIEWS são tabelas físicas que armazenam dados temporários, facilitando a manipulação de registros.
- II - VIEWS podem ser utilizadas para simplificar consultas complexas, encapsulando a lógica de acesso aos dados.
- III - Modificar dados diretamente em uma VIEW afeta automaticamente os dados na tabela subjacente.
- IV - VIEWS são exclusivamente utilizadas para criar relatórios estáticos e não podem conter condições de filtro.

É correto o que se afirma em:

- a) I e IV, apenas.
- b) II e III, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) II, III e IV, apenas.

REFERÊNCIAS

ALVES, W. P. **Banco de dados**: teoria e desenvolvimento. São Paulo: Érica, 2021.

BARBOZA, F. F. M.; FREITAS, P. H. C. **Modelagem e desenvolvimento de banco de dados**. Porto Alegre: SAGAH, 2018.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

MACHADO, F. N. R. Banco de dados: projeto e implementação. São Paulo: Érica, 2020.

PICHETTI, R. F.; VIDA, E. da S.; CORTES, V. S. M. P. **Banco de dados**. Porto Alegre: SAGAH, 2021.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

GABARITO

1. Opção B.

A estruturação correta de um banco de dados relacional busca minimizar a redundância de dados, o que significa evitar a repetição desnecessária de informações. Isso promove a eficiência na gestão de dados, economizando espaço de armazenamento e mantendo a consistência das informações. A normalização, um princípio-chave na estruturação do banco, é projetada para reduzir a redundância e melhorar a integridade dos dados. Embora uma estruturação adequada possa facilitar a leitura de consultas SQL, ela não é o principal objetivo. A estruturação correta busca simplificar, e não aumentar, a complexidade das consultas. A estruturação do banco de dados não está diretamente relacionada à agilidade na instalação do sistema. A estruturação do banco de dados não afeta diretamente a necessidade de *backups*, pois os *backups* são essenciais, independentemente da estruturação.

2. Opção C.

Stored Procedures são, de fato, blocos de código SQL armazenados no banco de dados. Esses códigos podem incluir uma sequência de instruções SQL que realizam operações específicas, como inserção, atualização ou recuperação de dados, e podem ser executadas posteriormente por meio de uma chamada explícita. As Stored Procedures não são apenas consultas SQL: elas podem incluir blocos de código com várias instruções e lógica de controle. Stored Procedures são executadas no servidor, e não ao lado do cliente, e não são destinadas para otimizar a interface gráfica do usuário. As Stored Procedures não são exclusivamente usadas para a criação de índices: elas têm uma variedade de finalidades. Embora as Stored Procedures possam realizar operações de criação e modificação de tabelas, esse não é o único propósito: elas podem abranger uma variedade de tarefas.

3. Opção B.

VIEWS são, de fato, utilizadas para simplificar consultas complexas, encapsulando a lógica de acesso aos dados. Modificar dados diretamente em uma VIEW afeta automaticamente os dados na tabela subjacente, pois uma VIEW é uma representação virtual dos dados da tabela. VIEWS não são tabelas físicas, mas consultas armazenadas que não armazenam dados temporários. VIEWS podem conter condições de filtro, sendo uma forma eficaz de restringir dados em consultas.



TEMA DE APRENDIZAGEM 6

INTRODUÇÃO AOS BANCOS DE DADOS NOSQL

MINHAS METAS

- Compreender os conceitos fundamentais de bancos de dados NoSQL.
- Explorar os diversos modelos de dados NoSQL.
- Aplicar conhecimentos práticos por meio de exercícios práticos e projetos.
- Analisar as diferenças e as semelhanças entre bancos de dados NoSQL e relacionais.
- Aprender a otimizar o desempenho em ambientes NoSQL.
- Entender os aspectos de segurança em bancos de dados NoSQL.
- Explorar a integração de bancos de dados NoSQL às tecnologias modernas.

INICIE SUA JORNADA

Bem-vindo(a) à fascinante jornada pelo universo dos bancos de dados NoSQL! Você está prestes a mergulhar em um campo dinâmico e inovador que desafia as convenções dos bancos de dados relacionais tradicionais. NoSQL (ou “Not Only SQL”) representa uma abordagem flexível e escalável para o gerenciamento. Ela é adequada aos desafios impostos pelos ambientes modernos de desenvolvimento e às necessidades de grandes conjuntos de dados.

Durante a sua jornada, você terá a oportunidade de explorar a diversidade de modelos de dados NoSQL. Desde os modelos orientados a documentos, como o MongoDB, até os baseados em chave-valor, como o Redis, cada um traz consigo características únicas e vantagens específicas. Compreender esses tipos será fundamental para tomar decisões informadas sobre qual abordagem adotar em diferentes contextos de aplicação.

À medida que avançamos, o aspecto prático ganhará destaque. Você não apenas aprenderá os conceitos teóricos, mas também poderá colocar a mão na massa, implementando soluções práticas com os principais bancos de dados NoSQL. Mediante exercícios, você ganhará confiança na aplicação dos conhecimentos adquiridos, consolidando a sua compreensão e habilidades.

Além disso, exploraremos as implicações estratégicas da adoção do NoSQL. Entenderemos quando e por que escolher essa abordagem sobre os bancos de dados relacionais, e como a flexibilidade desses sistemas pode se alinhar às necessidades de projetos modernos. Prepare-se para desvendar os mistérios e os desafios que esse emocionante domínio apresenta, construindo um alicerce sólido para a sua jornada no vasto campo dos bancos de dados NoSQL.

**PLAY NO CONHECIMENTO**

Bem-vindo(a) ao Desbravando Dados, o podcast que te levará a uma emocionante viagem pelos conceitos iniciais dos bancos de dados não relacionais, também conhecidos como NoSQL ou No Sequel. Neste episódio, mergulharemos fundo no fascinante mundo desse tipo de armazenamento e gerenciamento de dados, desvendando as razões por trás da ascensão e destacando como eles redefinem a gestão em ambientes modernos de TI.

Ao longo dessa jornada auditiva, exploraremos a diversidade de modelos NoSQL, desde os versáteis, orientados a documentos, até os eficientes sistemas baseados em chave-valor e grafos. Desvendaremos, também, os benefícios práticos dessas abordagens, analisando como elas proporcionam escalabilidade, flexibilidade e desempenho otimizado para os mais diversos casos de uso.

Prepare-se para uma imersão informativa, repleta de insights e dicas práticas. Se você está iniciando a sua jornada pelo mundo dos bancos de dados NoSQL ou é um(a) entusiasta experiente em busca de novos horizontes, Desbravando Dados é o seu guia confiável nessa exploração fascinante. Aperte o play e embarque conosco nesta jornada de descobertas!

VAMOS RECORDAR?

O armazenamento de dados é um pilar fundamental da Tecnologia da Informação (TI), representando os processos de retenção e preservação de informações digitalizadas para uso futuro. Esse conceito abrange diversas formas e tecnologias, desde sistemas físicos, tais como discos rígidos e servidores, até soluções mais modernas, baseadas em nuvem. A eficácia desse armazenamento é crucial para garantir a acessibilidade, a integridade e a segurança das informações ao longo do tempo.

No contexto da TI, ele é essencial para suportar uma variedade de operações, desde a simples preservação de arquivos até o suporte a complexas infraestruturas de aplicativos e bancos de dados. A evolução tecnológica trouxe opções diversificadas, como cloud computing, que permite o acesso remoto e flexível aos dados, facilitando a colaboração e a escalabilidade. Além disso, as tecnologias de armazenamento estão cada vez mais focadas em otimizar o desempenho, a eficiência energética e a redundância para garantir a confiabilidade e a disponibilidade dos dados.

Assim, o armazenamento de dados na Tecnologia da Informação (TI) é um elemento essencial para gerenciar a explosão de informações digitais, garantindo que as organizações e os indivíduos possam guardar, acessar e proteger dados de maneira eficiente, cumprindo os requisitos operacionais e estratégicos da era digital.

<https://www.youtube.com/watch?v=lkBg4RVR3Zo>

DESENVOLVA SEU POTENCIAL

ARMAZENAMENTO DE GRANDES VOLUMES DE DADOS

O armazenamento de grandes volumes de dados, conhecido como Big Data, tornou-se uma faceta crucial da era digital, em que a quantidade e a complexidade das informações crescem exponencialmente.

Esse conceito, segundo Silberschatz (2020), vai além de simplesmente guardar dados: trata-se de abraçar a expansão e a variedade de dados gerados em diferentes formatos e velocidades. Para gerir esses volumes massivos, é necessário compreender a tríade fundamental desse conceito: volume, velocidade e variedade.

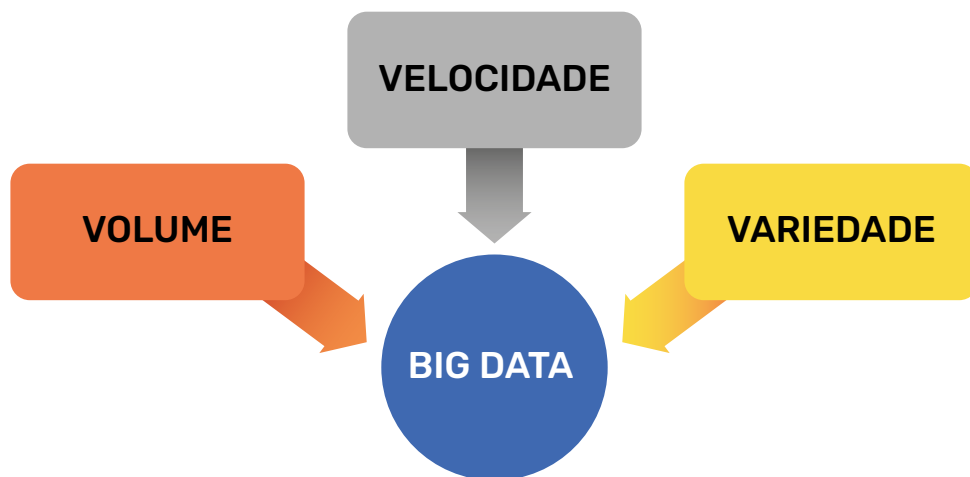


Figura 1 – 3Vs - Estrutura base do Big Data / Fonte: o autor.

Descrição da Imagem: é exibido um círculo na cor azul com a palavra "BIG DATA" ao centro. À esquerda, há um retângulo de cor laranja com a palavra "VOLUME". Ao centro, mais acima, há outro retângulo na cor cinza com a palavra "VELOCIDADE" e, por fim, à direita, há um retângulo na cor amarela com a palavra "VARIEDADE". Fim da descrição.

Vamos identificar cada um dos componentes do Big Data:

- **Volume:** refere-se à escala maciça dos dados gerados todos os dias em sistemas computacionais. Atualmente, as organizações lidam com escalas de *petabytes* e *exabytes* de informações provenientes de fontes, como transações on-line, redes sociais, sensores e registros históricos. Esse aumento exponencial implica em estratégias robustas de armazenamento, desde tecnologias tradicionais até plataformas em nuvem, que permitem escalabilidade praticamente ilimitada.
- **Velocidade:** representa a taxa na qual os dados são gerados, processados e analisados em tempo real. Com a ascensão de dispositivos conectados e sistemas automatizados, a capacidade de lidar com informações em alta velocidade torna-se crucial para tomar decisões rápidas e eficazes. Soluções, como bancos de dados *in-memory* e arquiteturas de processamento em tempo real, desempenham um papel vital nesse contexto.

- **Variedade:** destaca a diversidade de formatos e tipos de dados. O Big Data gerirá dados, em sua grande maioria, não estruturados, como texto, imagens, vídeos e áudio. Ferramentas, como Hadoop e Spark, são projetadas para processar essa variedade, permitindo uma compreensão mais holística do panorama informacional.

Além da tríade VVV, o Big Data também incorpora os conceitos de **veracidade**, referindo-se à confiabilidade, e o **valor**, focando na extração de *insights* e valor significativo desses conjuntos de dados massivos. Esses princípios fundamentais orientam as estratégias de armazenamento, processamento e análise necessárias para explorar o potencial transformador em setores que vão desde a ciência e pesquisa até negócios e governança.

Por fim, é correto afirmar, segundo Basso (2020), que o armazenamento de grandes volumes de dados transcende a mera capacidade de guardar informações: trata-se de uma disciplina multifacetada que impulsiona a inovação e a tomada de decisões em um mundo inundado por dados.

Ferramentas de Gestão de Grandes Massas de Dados

Essas ferramentas se tornaram pilares fundamentais na era do Big Data, oferecendo soluções robustas para lidar com o volume exponencial e a complexidade dos dados gerados diariamente. Um dos líderes nesse cenário é o Apache Hadoop, uma estrutura *open source* que permite o processamento distribuído em grande escala e utiliza o modelo MapReduce para dividir tarefas complexas em operações menores, permitindo a execução em paralelo em *clusters* de computadores, garantindo escalabilidade e desempenho.

MapReduce é um modelo de programação e processamento projetado para facilitar a manipulação de grandes conjuntos de dados em ambientes distribuídos. Desenvolvido pela Google, esse paradigma divide as operações em duas etapas principais: o "Map", que aplica uma função a cada elemento do conjunto de dados original, gerando pares chave-valor intermediários, e o "Reduce", que combina os resultados intermediários com base em suas chaves. Essa abordagem simplifica o processamento paralelo em *clusters* de servidores, permitindo a análise eficiente de grandes volumes de dados.

O MapReduce é frequentemente utilizado para realizar tarefas de processamento distribuído em escala, sendo fundamental em diversas aplicações, desde análise de dados até indexação de motores de busca.

Outra ferramenta proeminente desse conceito é o Apache Spark, que amplia as capacidades do Hadoop, ao proporcionar processamento em memória, acelerando significativamente a análise de dados. Ele oferece uma abordagem mais rápida e flexível, suportando não apenas tarefas *batch*, mas também *streaming* em tempo real, possibilitando análises em tempo real de grandes volumes de dados. Essa flexibilidade, segundo Silva *et al.* (2021), faz do Spark uma escolha versátil para uma variedade de casos de uso.

No âmbito dos bancos de dados, o MongoDB se destaca como uma ferramenta eficaz para gerenciar grandes volumes de dados não estruturados. Ao adotar um modelo orientado a documentos, essa ferramenta permite a escalabilidade horizontal, facilitando o armazenamento e a recuperação eficientes em ambientes dinâmicos. A capacidade dele de lidar com dados variados, combinada com consultas eficientes, constitui uma opção valiosa para aplicações modernas.

Além disso, as plataformas de armazenamento em nuvem, como Amazon S3 e Microsoft Azure Blob Storage, oferecem soluções escaláveis para o armazenamento de dados. Essas plataformas permitem a alocação flexível de recursos de armazenamento, proporcionando às organizações a capacidade de expandir ou reduzir a própria infraestrutura de acordo com as necessidades, eliminando a preocupação com limitações físicas.

Por fim, as ferramentas de visualização de informações, como Tableau e Power BI, desempenham um papel crucial na extração de *insights* significativos a partir de grandes volumes de dados. Essas ferramentas simplificam a análise e a apresentação de valores complexos de maneira visualmente compreensível, permitindo que as organizações tomem decisões informadas com base nos padrões e nas tendências identificados.

As ferramentas de gestão de grandes massas de dados constituem um ecossistema diversificado e interconectado, abrangendo desde estruturas distribuídas para processamento até plataformas de armazenamento escaláveis e ferramentas de visualização avançada. A escolha e a integração eficaz dessas ferramentas, segundo Elmasri e Navathe (2018), tornam-se essenciais para as organizações que buscam explorar o potencial transformador dos grandes volumes em um mundo cada vez mais orientado pela informação.

INTRODUÇÃO A BANCO DE DADOS NOSQL

Os bancos de dados NoSQL, ou “Not Only SQL”, representam uma revolução na forma como são armazenados e gerenciados os dados em um mundo digital em constante expansão. Ao contrário dos relacionais, que adotam uma estrutura tabular, esse tipo de banco abraça a diversidade de modelos e proporciona uma flexibilidade que se torna crucial diante da crescente complexidade dos dados modernos.

Em primeiro lugar, o modelo orientado a documentos é um dos mais proeminentes no universo NoSQL. O MongoDB permite armazenar dados em documentos semelhantes a JSON, proporcionando uma flexibilidade significativa. Cada documento pode ter a própria estrutura, permitindo que organizações gerenciem eficientemente com dados variados, sem a necessidade de esquemas rigidamente definidos.

Outro modelo prevalente é o baseado em chave-valor, exemplificado por sistemas, como o Redis. Essa abordagem simplifica a estrutura de dados para pares chave-valor, permitindo um acesso rápido e eficiente às informações. É especialmente útil para casos de uso que demandam alta velocidade de leitura e gravação, como caches em memória.

Além disso, os bancos de dados NoSQL incluem modelos de grafos, como o Neo4j, que são ideais para representar e explorar as relações complexas entre entidades. Esse modelo é fundamental para áreas, como redes sociais, análise de fraudes e sistemas que dependem fortemente de conexões entre dados.

A escalabilidade é, segundo Silva *et al.* (2021), uma das vantagens mais destacadas do NoSQL, já que ele é devidamente projetado para administrar com grandes volumes de dados e pode ser facilmente escalado horizontalmente, adicionando mais servidores conforme a demanda cresce. Essa capacidade é particularmente valiosa em um ambiente onde a expansão rápida é a norma.

Por fim, o NoSQL representa uma mudança de paradigma na gestão de dados, oferecendo flexibilidade, escalabilidade e eficiência em cenários que desafiam as capacidades dos relacionais tradicionais. À medida que as organizações enfrentam a complexidade crescente dos dados, a adoção de bancos de dados NoSQL emerge como uma estratégia essencial para atender às demandas de uma era digital cada vez mais dinâmica.

Ferramentas de Gestão de Grandes Massas de Dados

O NoSQL apresenta uma diversidade de modelos que se adaptam a diferentes tipos de dados e requisitos de aplicativos.

Cada tipo, segundo Sadalage e Fowler (2013), oferece abordagens únicas para o armazenamento e a recuperação de informações, proporcionando flexibilidade em um cenário dinâmico e complexo.

ORIENTADO A DOCUMENTOS

neste modelo, os dados são armazenados em documentos semelhantes a JSON, contendo pares chave-valor. Cada documento pode ter uma estrutura distinta, permitindo flexibilidade no esquema. É eficaz para representar dados complexos e variados, sendo uma escolha comum para aplicações que demandam agilidade e escalabilidade. Exemplo: MongoDB.

BASEADO EM CHAVE-VALOR

simplifica a estrutura de dados para pares chave-valor, em que cada valor é associado a uma chave única. É extremamente eficiente para operações de leitura e gravação rápidas, tornando-se ideal para sistemas de cache em memória e casos de uso que exigem alta velocidade. Exemplo: Redis.

ORIENTADO A COLUNAS

armazena os dados em colunas, ao contrário de linhas. Essa estrutura é otimizada para consultas analíticas e agregações, sendo particularmente útil em cenários de Big Data. É uma escolha frequente para aplicações que demandam escalabilidade horizontal. Exemplo: Apache

BASEADO EM GRAFOS

representa os dados como nós e arestas, sendo ideal para representar e explorar relações complexas entre entidades. Esse tipo é amplamente utilizado em cenários que envolvem redes sociais, análise de fraudes e sistemas que dependem fortemente de conexões entre dados. Exemplo: Neo4j.

ORIENTADO A OBJETOS

permite armazenar e recuperar objetos diretamente, sem a necessidade de mapeamento para estruturas tabulares. Isso facilita a integração com linguagens de programação orientadas a objetos, sendo uma escolha vantajosa para aplicações que trabalham com dados complexos e relacionamentos ricos. Exemplo: db4o.

MULTIMODELO

adota uma abordagem multimodelo, combinando características de diferentes tipos. Isso oferece versatilidade, permitindo aos desenvolvedores escolher o modelo mais adequado para cada parte da aplicação, proporcionando uma abordagem híbrida conforme necessário. Exemplo: ArangoDB.

Nesse contexto, a diversidade do NoSQL reflete a necessidade de abordagens flexíveis diante da complexidade e da diversidade dos dados modernos. A escolha entre esses tipos, segundo Sadalage e Fowler (2013), depende das características específicas da aplicação e dos requisitos de dados, proporcionando às organizações a liberdade de selecionar a ferramenta mais adequada para suas demandas.

Banco de Dados Orientado a Documentos

Refere-se a uma categoria significativa dentro do ecossistema NoSQL projetada para lidar com a diversidade e as demandas de aplicações modernas. Nesse modelo, os dados são armazenados em documentos, que podem ser comparados a objetos JSON ou BSON (Binary JSON). Cada documento contém pares chave-valor e pode ter uma estrutura distinta, oferecendo flexibilidade no esquema e adaptabilidade a dados variados.

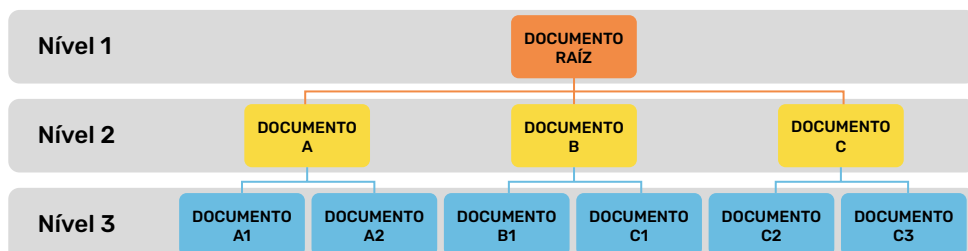


Figura 2 - Estrutura de armazenamento orientada a documentos / Fonte: o autor.

Descrição da Imagem: são ilustrados três níveis de abstração. No nível 1, há um retângulo na cor laranja com a expressão "Documento Raíz". No nível 2, são demonstrados três retângulos na cor amarela com as palavras: Documento A, Documento B e Documento C, respectivamente. Por fim, há o nível três, com seis retângulos de cor azul com as palavras: Documento A1, Documento A2, Documento B1, Documento C1, Documento C2 e Documento C3, respectivamente. Fim da descrição.

Uma característica marcante é a capacidade de armazenar informações complexas e hierárquicas de maneira eficiente. Isso é particularmente útil em cenários onde os dados não têm uma estrutura uniforme, como em aplicações web que lidam com informações de usuários com diferentes atributos.

Além disso, segundo Sadalage e Fowler (2013), a flexibilidade do esquema permite que os desenvolvedores adicionem campos ou modifiquem a estrutura dos documentos conforme necessário, sem a necessidade de alterar um esquema predefinido.

Outro aspecto crucial é, de acordo com Sadalage e Fowler (2013), a escalabilidade horizontal. Um exemplo desse tipo é o MongoDB, que é projetado para lidar com

**Capacidade
de armazenar
informações
complexas e
hierárquicas**

grandes volumes de dados distribuídos em *clusters* de servidores. Isso permite que as organizações dimensionem as próprias operações conforme a demanda cresce, mantendo um desempenho consistente. A capacidade de escalar horizontalmente é particularmente relevante em um cenário em que a expansão rápida dos dados é a norma.

A eficiência na leitura e na escrita também é um ponto forte nesse modelo. A estrutura de documentos facilita a recuperação eficiente de dados e índices podem ser aplicados para otimizar consultas específicas. Essa eficiência é essencial para aplicações que exigem respostas rápidas e baixa latência, como os sistemas de gerenciamento de conteúdo, *e-commerce* e análise em tempo real.

Esse tipo de banco de dados representa uma abordagem flexível e dinâmica para gerenciar dados em ambientes modernos. A capacidade dele de lidar com a diversidade de dados, a escalabilidade horizontal e a eficiência operacional os torna uma escolha valiosa para organizações que buscam soluções adaptáveis em um cenário digital em constante evolução.

O MongoDB se destaca pela abordagem orientada a documentos, proporcionando flexibilidade e eficiência na gestão. Baseado em documentos BSON, permite que os desenvolvedores armazenem dados em formato semelhante ao JSON, usando pares chave-valor. Essa flexibilidade de esquema torna o MongoDB ideal para ambientes dinâmicos, em que a estrutura pode evoluir com o tempo, e diferentes documentos podem ter estruturas distintas.

Um dos diferenciais do MongoDB é a capacidade dele de escalabilidade horizontal, que permite distribuir a carga de trabalho em vários servidores, proporcionando um desempenho consistente mesmo diante de volumes massivos de dados. Com a escalabilidade horizontal, o banco de dados é capaz de lidar eficientemente com aplicações que demandam expansão rápida e simultânea.

Além disso, o MongoDB oferece recursos avançados, tais como índices para otimizar consultas, suporte a transações para garantir a consistência dos dados e *sharding* para distribuição eficiente em ambientes distribuídos.

Essas funcionalidades, segundo Silva *et al.* (2021), fazem desse banco uma escolha popular para uma variedade de casos de uso, desde aplicações web até sistemas de análise de dados em grande escala.

Assim, é possível afirmar que o MongoDB redefiniu a gestão de dados NoSQL, proporcionando flexibilidade, escalabilidade e recursos robustos para atender às demandas complexas da era digital.

Bancos de Dados Orientado a Colunas

São projetados para otimizar a recuperação e a análise de grandes volumes de dados, que são organizados e armazenados em colunas, ao contrário do tipo relacional, que utiliza linhas. Essa abordagem traz benefícios significativos para cenários analíticos e de Big Data, em que a necessidade de consultas eficientes e escalabilidade é premente.

A estrutura de colunas permite que os dados sejam armazenados de forma altamente eficiente, especialmente para operações que envolvem a recuperação de um subconjunto específico, ao contrário de linhas inteiras. Isso é particularmente valioso em cenários de análise, em que a capacidade de acessar rapidamente colunas específicas para cálculos e agregações é essencial. Esse tipo de banco de dados, como o Apache Cassandra, segundo Silva *et al.* (2021), adota essa abordagem para atender às demandas analíticas em tempo real.

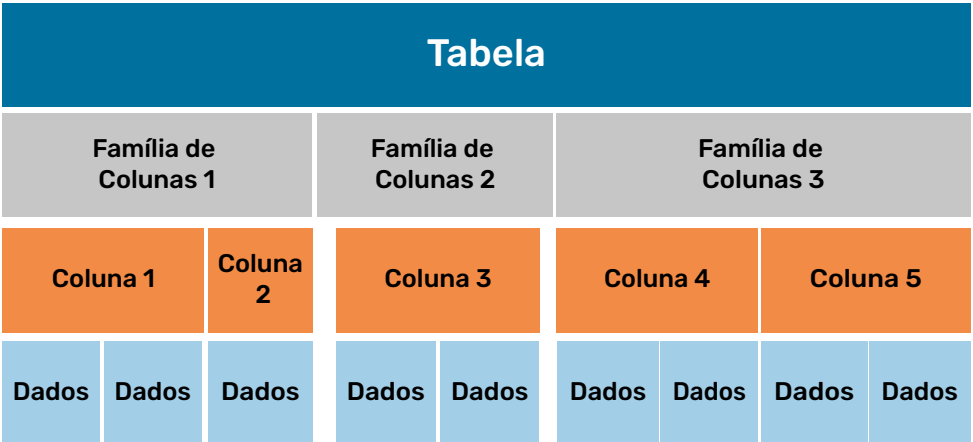


Figura 3 - Estrutura de armazenamento orientada a colunas / Fonte: o autor.

Descrição da Imagem: é exibida uma representação multinível. No nível 1, há um retângulo de cor azul com a palavra "Tabela". No segundo nível, há três retângulos na cor cinza com as palavras: Família de Coluna 1, Família de Coluna 2 e Família de Coluna 3. No terceiro nível, encontram-se cinco retângulos de cor laranja com as palavras: Coluna 1, Coluna 2, Coluna 3, Coluna 4 e Coluna 5. No último nível, há nove retângulos de cor azul-claro, todos com a palavra "Dados". Fim da descrição.

A capacidade de escalabilidade horizontal é outra característica distintiva desse modelo. Ao distribuir dados entre diversos nós, os bancos de dados orientados a colunas podem lidar eficientemente com grandes volumes e garantir respostas rápidas mesmo em ambientes distribuídos. Isso é particularmente relevante em cenários em que a expansão rápida é a norma, assim como em sistemas que gerenciam registros de *logs* ou informações provenientes de sensores em tempo real.

Além disso, a flexibilidade do esquema é preservada, uma vez que cada linha pode ter diferentes conjuntos de colunas. Isso facilita a adaptação a mudanças nos requisitos de dados sem a necessidade de alterações extensivas no esquema, o que permite uma evolução mais ágil do sistema em resposta às demandas em constante mudança.

Assim, um banco de dados orientado a colunas representa uma abordagem eficaz para lidar com a gestão de grandes volumes de dados em ambientes analíticos e de Big Data. A capacidade dele de otimizar consultas, a escalabilidade horizontal e a flexibilidade no esquema fazem dele uma escolha valiosa para organizações que buscam soluções eficientes e ágeis diante da complexidade dos dados modernos.

O Apache Cassandra é projetado para oferecer escalabilidade e alta disponibilidade em ambientes distribuídos. Adotando uma arquitetura descentralizada, permite a expansão horizontal, distribuindo dados em vários nós sem ponto único de falha. Essa característica torna esse banco de dados ideal para cenários em que a demanda por escalabilidade é alta, como em grandes plataformas on-line e sistemas de Big Data.

Dois princípios fundamentais dele são a tolerância a falhas e a resiliência. O modelo descentralizado e distribuído garante que, mesmo em situações de falhas de hardware ou conexões, o sistema continue operando sem perda significativa de desempenho. Isso é essencial para garantir a disponibilidade contínua de serviços críticos e a integridade dos dados.

O Cassandra também se destaca pela capacidade dele de lidar com grandes volumes de dados e consultas de baixa latência. Ao adotar um modelo de armazenamento baseado em colunas, otimiza a recuperação de dados para consultas analíticas e agregações. Isso faz com que ele seja uma escolha popular em cenários que demandam análises em tempo real, como em sistemas de monitoramento e análise de *logs*.

É totalmente possível afirmar que esse banco de dados se destaca pela arquitetura distribuída, tolerância a falhas e desempenho otimizado, tornando-se uma escolha robusta para ambientes que exigem escalabilidade e disponibilidade em larga escala.

Banco de Dados Orientado a Grafos

Refere-se a uma categoria especializada que se destaca pelas habilidades de modelar e explorar relações complexas entre entidades. Ele é diferente de outros modelos NoSQL, que se concentram em estruturas mais simplórias, sendo projetados para representar grafos, onde nós representam entidades, e arestas representam as relações entre elas. Essa abordagem é particularmente eficaz em cenários que envolvem redes sociais, sistemas de recomendação, análise de fraudes e qualquer aplicação que venha a depender fortemente das conexões entre dados.

Grafo: é uma estrutura de dados composta por nós (ou vértices) e arestas que conectam esses nós. Essa representação é utilizada para visualizar as relações e as conexões entre entidades, em que cada nó representa uma entidade e cada aresta denota uma relação entre essas entidades. Os grafos são amplamente utilizados para modelar redes sociais, sistemas de recomendação, mapas de estradas, entre outros, proporcionando uma abordagem intuitiva para expressar e analisar interconexões complexas. A flexibilidade e a eficiência dos grafos na representação de relações fazem deles uma ferramenta valiosa em diversas áreas da computação, desde algoritmos de busca até bancos de dados orientados a grafos, em que a análise e a navegação eficientes de conexões são essenciais.

A representação visual de dados em forma de grafo facilita a compreensão das relações complexas, permitindo que desenvolvedores e analistas identifiquem padrões e tomem decisões informadas. A eficiência na representação e na navegação de relações é uma das principais características desse tipo de armazenamento, tornando-o ideal para situações em que a análise de conexões é central.

A escalabilidade é uma consideração fundamental nesse modelo. Um exemplo é o Neo4j, que é projetado para lidar com grandes volumes de dados e complexidade de relacionamentos. Isso é alcançado mediante estratégias, como indexação eficiente e particionamento inteligente, permitindo que a busca e a análise de relações permaneçam rápidas mesmo em ambientes distribuídos.

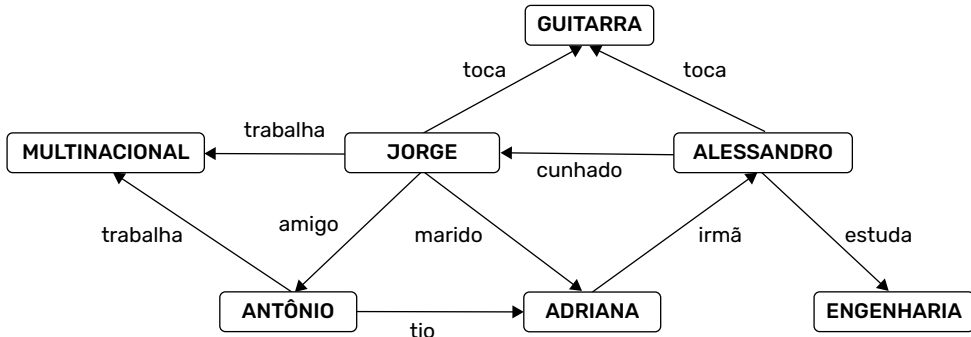


Figura 4 - Estrutura de armazenamento orientada a grafos / Fonte: o autor.

Descrição da Imagem: são exibidos sete retângulos com as palavras GUITARRA, MULTINACIONAL, JORGE, ANTÔNIO, ADRIANA, ENGENHARIA e ALESSANDRO. As setas interligam esses retângulos entre si, e cada uma dessas setas apresenta uma palavra que indica essa interligação. Fim da descrição.

A linguagem de consulta específica, como o Cypher, no caso do Neo4j, facilita a expressão de padrões e a navegação eficiente pelo grafo. Essa linguagem é otimizada para realizar consultas que exploram as relações, permitindo que os usuários recuperem informações específicas de maneira rápida e intuitiva.

Esse tipo de banco de dados fornece uma abordagem especializada para lidar com dados interconectados. A capacidade dele de representar os grafos complexos de forma eficiente, a escalabilidade e a facilidade de exercer consultas direcionadas a relações fazem dele uma escolha poderosa para aplicações que dependem fortemente de análises de rede e conexões entre entidades.

O Neo4j é um banco de dados NoSQL projetado para representar e analisar dados mediante estruturas de grafos. Diferentemente de modelos tradicionais, destaca-se pela capacidade de modelar relacionamentos complexos de maneira intuitiva. Nele, os dados são organizados em nós, que representam entidades, e arestas, que denotam as conexões entre essas entidades. Essa abordagem é particularmente poderosa para cenários em que a compreensão de relações é essencial, como em redes sociais, sistemas de recomendação e análise de redes.

A flexibilidade e a escalabilidade do Neo4j são fundamentais para lidar com a natureza dinâmica das estruturas relacionais. Assim, os grafos permitem uma representação clara de padrões complexos de interconexão entre esses dados, facilitando consultas que seriam desafiadoras em modelos convencionais. A capacidade de escalabilidade horizontal também contribui para o processamento eficiente de grandes conjuntos de dados interconectados.

Além disso, esse banco de dados oferece uma linguagem de consulta chamada Cypher, especialmente projetada para explorar e manipular os grafos de maneira intuitiva. Essa linguagem simplifica a expressão de consultas complexas, tornando a navegação pelos dados uma tarefa mais acessível. Em resumo, o Neo4j se destaca como uma ferramenta valiosa para organizações que buscam extrair significado de relações complexas em seus conjuntos de dados, proporcionando uma abordagem inovadora e eficaz para o gerenciamento no NoSQL.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Na era digital em constante evolução, compreender os conceitos básicos dos bancos de dados NoSQL (Not Only SQL) se tornou um imperativo para os estudantes de Tecnologia da Informação (TI) que buscam não apenas entender as bases tecnológicas atuais, mas também antecipar as demandas do futuro ambiente profissional. Esse tipo de banco de dados representa uma abordagem inovadora em relação aos tradicionais relacionais, proporcionando a flexibilidade e a escalabilidade necessárias para lidar com volumes massivos de dados em tempo real.

Ao explorar esses bancos de dados, você mergulha em uma variedade de modelos, como orientado a documentos, baseado em chave-valor e de grafos. Essa diversidade permite a adaptação a diferentes tipos de dados e requisitos de aplicativos, promovendo uma compreensão profunda da flexibilidade estrutural proporcionada pelo NoSQL. Essa flexibilidade é crucial para o desenvolvedor que busca criar aplicações ágeis e adaptáveis.

Os profissionais de TI que dominam o NoSQL estão preparados para enfrentar os desafios da eficiência operacional e do desempenho otimizado. Com a capacidade de distribuir dados em larga escala e adaptar-se a modelos de dados dinâmicos, esses bancos de dados permitem que as organizações alcancem níveis de escalabilidade que seriam difíceis de alcançar com abordagens mais tradicionais. Isso se traduz em aplicações mais rápidas, respostas instantâneas e melhor experiência do usuário.

O conhecimento sólido em NoSQL se torna uma vantagem estratégica à medida que você adentra o ambiente profissional. Com a crescente adoção desses sistemas em empresas inovadoras e projetos de tecnologia de ponta, os profissionais de TI que compreendem os conceitos básicos dos bancos de dados NoSQL estão aptos a projetar, implementar e otimizar soluções eficazes, contribuindo para o sucesso das organizações em um cenário digital em constante transformação.

Em última análise, o NoSQL não é apenas uma ferramenta, mas representa uma mentalidade de adaptabilidade e aprendizado contínuo. Os profissionais de TI que se aprofundam nesses conceitos estão preparados para enfrentar não apenas os desafios atuais, mas também para abraçar inovações futuras. À medida que novas tecnologias emergem, a base sólida nos bancos de dados NoSQL proporciona uma vantagem competitiva, permitindo aos profissionais de TI liderar a carga na criação de soluções robustas e eficientes.

AUTOATIVIDADE

1. O armazenamento de dados representa os processos de retenção e organização de informações digitais de maneira estruturada para facilitar o acesso, a recuperação e a manipulação posterior. Essa prática essencial na computação envolve o alojamento seguro e duradouro, garantindo a integridade ao longo do tempo. Ele pode ocorrer de diferentes formas, desde bancos de dados tradicionais até sistemas mais modernos, como armazenamento em nuvem. A eficácia do armazenamento de dados é crucial para suportar diversas aplicações, desde sistemas corporativos até dispositivos pessoais, permitindo a preservação e a utilização eficientes da vasta quantidade de informações geradas na era digital.

Qual é uma característica essencial do armazenamento eficiente de grandes massas de dados? Assinale a alternativa correta:

- a) Estrutura de Banco de Dados Relacional.
 - b) Modelo de Grafos.
 - c) Armazenamento em Nuvem.
 - d) Bancos de Dados NoSQL.
 - e) Processamento MapReduce.
-
2. Big Data é um conceito que se refere às capacidades de coletar, processar e analisar imensos volumes de dados, muitas vezes, em tempo real. Caracterizado pelos 3Vs principais (volume, velocidade e variedade), enfrenta o desafio de lidar com enormes quantidades de informações estruturadas e não estruturadas. Essa abordagem não apenas reconhece a importância da quantidade massiva de dados gerados, mas também destaca a necessidade de processamento rápido e eficiente, e a diversidade de formatos e tipos de dados. Utilizando tecnologias avançadas e ferramentas analíticas, o Big Data busca revelar padrões, tendências e *insights* valiosos, fornecendo uma base sólida para a tomada de decisões estratégicas em diversos setores, desde negócios até pesquisa científica.

AUTOATIVIDADE

Considerando os 3Vs do Big Data (volume, velocidade e variedade), analise as afirmativas a seguir:

- I - Volume, velocidade e variedade são conceitos fundamentais do Big Data.
- II - Volume se refere à quantidade massiva de dados gerados e armazenados.
- III - A velocidade destaca a diversidade de formatos e tipos de dados.
- IV - A variedade enfatiza a rapidez com que os dados são processados e analisados.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

AUTOATIVIDADE

3. O Apache Hadoop é um *framework* de software de código aberto que viabiliza o processamento distribuído de grandes volumes de dados em *clusters* de computadores. Baseado nos princípios do MapReduce, permite a escalabilidade horizontal, possibilitando que organizações processem e analisem quantidades massivas de dados de maneira eficiente. Além do modelo MapReduce, o Hadoop incorpora o Hadoop Distributed File System (HDFS), um sistema de arquivos distribuído que oferece armazenamento confiável e de alto desempenho para os dados processados. Essa combinação torna o Apache Hadoop uma ferramenta valiosa para lidar com os desafios do Big Data, proporcionando uma infraestrutura robusta para o armazenamento e a análise distribuída em escala.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

- I - O MapReduce é um modelo de programação e processamento de dados que simplifica a manipulação de grandes conjuntos de dados em ambientes distribuídos.

PORQUE

- II - No modelo MapReduce, a fase "Map" aplica uma função a cada elemento do conjunto de dados original, gerando pares chave-valor intermediários.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

BASSO, D. E. **Big Data**. Curitiba: Contentus, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

SADALAGE, P. J.; FOWLER, M. **Nosql Essencial**. São Paulo: Novatec, 2013.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

SILVA, L. F. C. *et al.* **Banco de dados não relacional**. Porto Alegre: SAGAH, 2021.

GABARITO

1. Opção C.

Em cenários de grandes volumes de dados, a infraestrutura em nuvem oferece escalabilidade, flexibilidade e recursos sob demanda, permitindo a gestão eficiente de grandes massas de dados. As opções A, B, D e E, embora possam ter aplicabilidade em determinados contextos, não são características específicas voltadas para o armazenamento eficiente em grande escala.

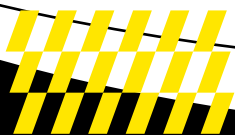
2. Opção A.

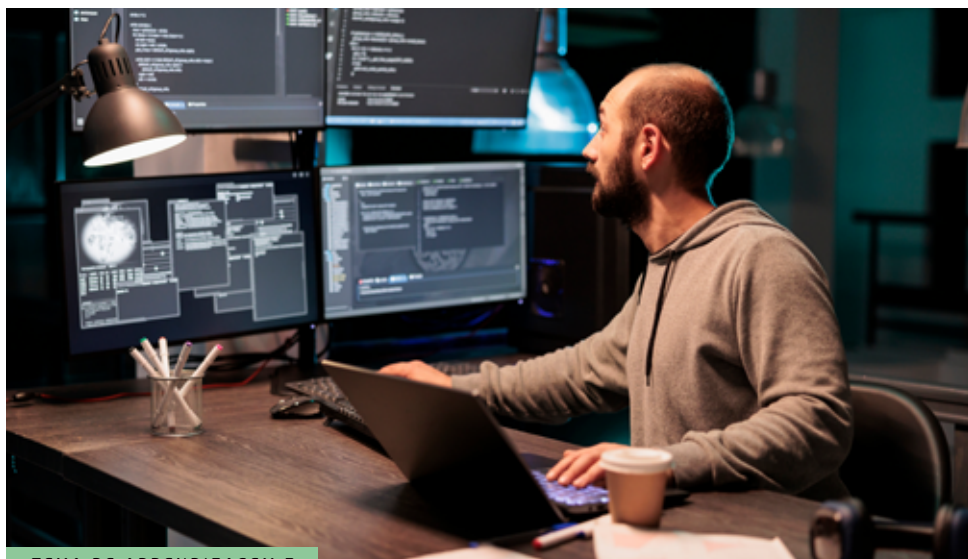
Nos 3Vs do Big Data, o volume se refere à quantidade massiva de dados gerados e armazenados. A velocidade destaca a rapidez com que os dados são processados e analisados, enquanto a variedade envolve a diversidade de formatos e tipos de dados.

3. Opção A.

O MapReduce é, de fato, um modelo de programação e processamento de dados que simplifica a manipulação de grandes conjuntos de dados em ambientes distribuídos. Na fase "Map", uma função é aplicada a cada elemento do conjunto de dados original, gerando pares chave-valor intermediários, justificando, assim, a eficácia do MapReduce em lidar com processamento distribuído em larga escala.

unidade





TEMA DE APRENDIZAGEM 7

BANCOS DE DADOS PARA SÉRIES TEMPORAIS ORIENTADOS A CHAVE-VALOR

MINHAS METAS

- Entender os princípios fundamentais dos bancos de dados NoSQL orientados a chave-valor.
- Explorar as diferentes tecnologias disponíveis no mercado que implementam o modelo NoSQL orientado a chave-valor.
- Aprender a criar e a manipular modelos de dados flexíveis.
- Entender a importância da escalabilidade horizontal nesse contexto.
- Desenvolver habilidades práticas na implementação de soluções utilizando bancos de dados NoSQL orientados a chave-valor.
- Aprofundar-se nas técnicas de otimização de desempenho.
- Conhecer as integrações possíveis entre os bancos de dados NoSQL orientados a chave-valor e outras tecnologias relevantes no ecossistema de Tecnologia da Informação (TI).

INICIE SUA JORNADA

Ao dar os primeiros passos pela vasta arena da Tecnologia da Informação (TI), você se deparará com uma promissora jornada, que é a exploração de bancos de dados do tipo NoSQL orientados a chave-valor. Nesse momento inicial, é crucial compreender que essa escolha não é apenas uma alternativa, mas uma abordagem inovadora para lidar com a crescente complexidade e volume de dados na atual era digital. Esse modelo, essencialmente simples e flexível, promete revolucionar a forma como as informações são gerenciadas e acessadas diariamente pelos milhões de sistemas computacionais espalhados pelo mundo, proporcionando uma perspectiva única e eficiente sobre o mundo dos bancos de dados.

Ao imergir nos estudos, você encontrará a base conceitual do NoSQL, em que os registros são armazenados mediante pares únicos de chave e valor. Essa simplicidade aparente é a essência de uma poderosa ferramenta que permite uma modelagem mais dinâmica e adaptável. Essa abordagem ressoa de forma impactante, especialmente ao se considerar cenários em que a escalabilidade e a flexibilidade são cruciais, como em sistemas que administram grandes volumes de dados distribuídos.

A segunda etapa dessa jornada te leva à descoberta de tecnologias específicas que incorporam o paradigma chave-valor. Explorar plataformas, como Redis, DynamoDB e Riak, torna-se essencial para compreender as nuances e as peculiaridades dessas soluções no mundo real. Cada uma dessas tecnologias traz consigo casos de uso específicos, vantagens e desafios, oferecendo a você um espectro rico de possibilidades para moldar as suas habilidades.

Ao longo dessa jornada inaugural, você perceberá que os bancos de dados NoSQL orientados a chave-valor não são apenas ferramentas técnicas, mas catalisadores de uma nova mentalidade na gestão de dados. O entendimento desse paradigma não apenas enriquecerá o seu conjunto de habilidades técnicas, mas também abrirá portas para abordagens inovadoras na solução de problemas complexos. Nesse início promissor, a jornada está apenas começando e as possibilidades se revelam tão vastas quanto os próprios dados que esse novo mundo promete gerenciar.



PLAY NO CONHECIMENTO

Bem-vindo(a) ao podcast *Desbravando Dados*, o seu guia definitivo pelo vasto território da Tecnologia da Informação (TI). Neste episódio, embarcaremos em uma jornada fascinante pelo mundo dos bancos de dados NoSQL orientados a chave-valor. Prepare-se para desvendar os mistérios desse modelo inovador, em que cada registro é uma peça única, identificado pela chave exclusiva. Junte-se a nós enquanto exploramos as tecnologias líderes nesse espaço, como Redis e DynamoDB, desvendando as aplicações práticas e os desafios.

Nossos convidados especialistas compartilharão *insights* valiosos sobre como esse paradigma transformador está moldando o cenário da gestão de dados. Mergulharemos nas nuances da modelagem flexível, da escalabilidade horizontal e das estratégias de otimização, que fazem dos bancos de dados do tipo NoSQL que são orientados a chave-valor uma escolha cada vez mais essencial no universo da TI. Se você está começando a sua jornada ou é um(a) veterano(a) em busca de atualizações, este episódio oferecerá uma visão profunda e esclarecedora sobre as possibilidades que esse fascinante campo proporciona.

Não perca a chance de se tornar um(a) verdadeiro(a) desbravador(a) de dados. Subscriba-se agora e embarque conosco nessa exploração fascinante do banco de dados NoSQL orientado a chave-valor. Este é *Desbravando Dados*, em que cada episódio é uma oportunidade de expandir seus horizontes e aprofundar o seu entendimento sobre as revolucionárias tecnologias que moldam o futuro da informação.

VAMOS RECORDAR?

Os bancos de dados não relacionais representam uma categoria diversificada de sistemas de gerenciamento que fogem do modelo relacional tradicional. Ao contrário dos relacionais, que seguem uma estrutura tabular rígida, os NoSQL abraçam uma abordagem mais flexível, permitindo o armazenamento de dados em formatos variados. Esse paradigma surge como resposta às crescentes demandas de escalabilidade, agilidade e variedade frequentemente encontradas em ambientes modernos de TI.

O NoSQL engloba várias categorias, cada uma adaptada a diferentes necessidades. Dentre elas, destacam-se os modelos orientados a documentos, chave-valor, colunas e grafos. Cada modelo aborda características específicas, oferecendo soluções otimizadas e direcionadas para casos de uso particulares. Essa diversidade permite que as organizações escolham o tipo de NoSQL mais adequado para as próprias aplicações, proporcionando eficiência na gestão de dados em cenários dinâmicos e em constante evolução.

A essência desse tipo de banco de dados reside na flexibilidade, na escalabilidade horizontal e na capacidade de lidar com grandes volumes de dados sem sacrificar o desempenho. Essa abordagem não apenas desafia as limitações dos modelos relacionais tradicionais, mas também oferece soluções inovadoras para os desafios contemporâneos de armazenamento e processamento de dados. Assim, o NoSQL emerge como uma peça fundamental no quebra-cabeça da gestão moderna de dados, proporcionando a flexibilidade necessária para enfrentar os desafios da era da informação. <https://www.youtube.com/watch?v=1B64oqE8PLs>

DESENVOLVA SEU POTENCIAL

ENTENDENDO O BANCO DE DADOS NOSQL

Esse tipo de banco de dados surge como uma resposta inovadora às limitações dos modelos relacionais. Esse paradigma, uma abreviação para “Not Only SQL” ou “Não Somente SQL” ou “No Sequel”, representa uma categoria diversificada de sistemas de gerenciamento de dados projetado para lidar com os desafios contemporâneos da era digital.

Ao contrário dos modelos relacionais, segundo Sadalage e Fowler (2013), o NoSQL não impõe uma estrutura tabular rígida, permitindo uma flexibilidade considerável na forma como os dados são armazenados e manipulados.

A principal característica dos bancos de dados NoSQL reside na abordagem não estruturada e não relacional. Isso significa que não há uma única linguagem de consulta, como o SQL, e os dados não precisam ser organizados em tabelas predefinidas.

Essa flexibilidade, segundo Silva *et al.* (2021), é particularmente valiosa em cenários em que a estrutura é dinâmica e não pode ser facilmente representada por tabelas relacionais. No contexto do NoSQL, os dados podem ser armazenados em diferentes formatos, como documentos, pares chave-valor, grafos ou famílias de colunas, dependendo do modelo adotado.

A escalabilidade horizontal é uma característica distintiva do NoSQL.

Dentro do vasto ecossistema NoSQL, existem várias categorias de modelos, cada uma projetada para atender a necessidades específicas. Os modelos orientados a documentos, como MongoDB, armazenam dados em arquivos JSON ou BSON, promovendo uma estrutura hierárquica. Já os bancos de dados orientados a chave-valor, como Redis e DynamoDB, armazenam informações em pares distintos de chave e valor, oferecendo alta performance e escalabilidade. Outros modelos, como orientados a colunas e grafos, adaptam-se a diferentes contextos e casos de uso, oferecendo eficiência em situações específicas.

A escalabilidade horizontal é uma característica distintiva do NoSQL. À medida que as demandas de dados aumentam, é possível adicionar novos servidores e distribuir a carga entre eles, garantindo um desempenho consistente. Essa capacidade de expansão facilita a gestão de grandes volumes de dados e torna os bancos de dados NoSQL uma escolha preferida para ambientes de Big Data.

Apesar da versatilidade e eficiência em determinados cenários, é importante destacar que os bancos de dados NoSQL não são uma panaceia. A escolha entre esse modelo ou o relacional depende das características específicas do projeto e

dos requisitos da aplicação. Em muitos casos, ambientes híbridos, que combinam o melhor de ambos os mundos, proporcionam soluções mais abrangentes.

O NoSQL representa uma revolução na gestão de dados, oferecendo flexibilidade, escalabilidade e eficiência em uma variedade de contextos. À medida que as organizações enfrentam o desafio de gerenciar dados cada vez mais complexos e dinâmicos, a adoção de modelos NoSQL se torna uma estratégia essencial para garantir a agilidade e a eficácia na gestão de informações.

Como Modelar um Banco de Dados NoSQL

A modelagem de sistemas de banco de dados é uma prática essencial no desenvolvimento de aplicações que requerem armazenamento estruturado e eficiente. Esse processo, segundo Alves (2021), envolve a criação de representações abstratas que descrevem como os dados serão organizados, relacionados e acessados dentro do banco de dados. Os modelos resultantes podem assumir diferentes formas, sendo os modelos relacional, hierárquico, de rede e orientado a objetos alguns dos paradigmas comuns. A escolha do modelo adequado depende das necessidades específicas da aplicação e dos requisitos de consulta.

A fase inicial da modelagem geralmente envolve a identificação de entidades e relacionamentos relevantes para o domínio da aplicação. Essas entidades são, então, representadas por tabelas ou coleções, enquanto os relacionamentos são expressos por meio de chaves estrangeiras ou referências. Esse processo não apenas organiza a estrutura de dados, mas também considera a integridade, a normalização e a eficiência nas operações de consulta e atualização. Essa abordagem proativa na definição dessa estrutura, segundo Silberschatz (2020), contribui significativamente para o desenvolvimento de sistemas robustos, flexíveis e de alto desempenho.

A modelagem no NoSQL se difere substancialmente do processo tradicional em bancos de dados relacionais. A primeira etapa crucial é entender o modelo de dados escolhido, seja ele orientado a documentos, chave-valor, colunas ou grafos. Por exemplo, em um banco de dados orientado a documentos, como o MongoDB, é fundamental conceber a estrutura, considerando quais campos são essenciais e como as relações entre eles serão representadas.

Esse modelo proporciona flexibilidade, permitindo a inclusão de novos campos conforme necessário, o que pode ser uma vantagem em ambientes dinâmicos.

Em seguida, é vital considerar a distribuição de dados, uma vez que a escalabilidade horizontal é uma característica fundamental no NoSQL. Ao projetar a estrutura do banco de dados, deve-se avaliar como esses dados serão distribuídos entre os nós do sistema. Isso impacta diretamente o desempenho e a capacidade de expansão do banco. Alguns sistemas NoSQL, como os orientados a chave-valor, priorizam a distribuição simples, enquanto outros, como os orientados a documentos, podem oferecer estratégias mais complexas, incluindo a distribuição por *sharding*. Por fim, é essencial compreender as consultas que serão realizadas no banco de dados para garantir uma modelagem eficiente.

Segundo Elmasri e Navathe (2018), em alguns casos, é necessário realizar normalizações ou criar índices específicos para otimizar consultas frequentes. A experiência prática e a compreensão profunda das características específicas do banco escolhido são cruciais para criar uma modelagem que atenda aos requisitos específicos do projeto. Essa abordagem pragmática na modelagem NoSQL permite aos desenvolvedores explorar todo o potencial dessa tecnologia, garantindo uma gestão de dados de maneira eficaz e ágil.

Consideremos um exemplo prático de modelagem de um banco de dados NoSQL, especificamente usando o modelo orientado a documentos, para criar um sistema de gerenciamento de biblioteca utilizando o software MongoDB, que é orientado a documentos.

- **Coleção de livros:** em um banco de dados NoSQL, os dados são armazenados em documentos flexíveis, geralmente, no formato JSON ou BSON. Cada livro pode ser representado como um documento na coleção “LIVROS”. Um exemplo é visível a seguir.

```
{
  "_id": 1,
  "titulo": "Dom Casmurro",
  "autor": "Machado de Assis",
  "ano_publicacao": 1899,
  "genero": "Romance",
  "quantidade_exemplares": 30,
  "emprestados": 5,
  "editora": {
    "nome": "Editora A",
    "localizacao": "Rio de Janeiro"
  }
}
```

Figura 1 – Script de um arquivo JSON referente aos dados do livro a ser inserido no banco / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo script JSON com 10 linhas de código referentes aos dados de livros a serem inseridos no banco de dados. Os dados são referentes a: ID, TITULO, AUTOR, ANO_PUBLICACAO, GENERO, QUANTIDADE_EXEMPLARES, EDITORA (NOME e LOCALIZACAO). Fim da descrição.

- **Coleção de usuários:** já os dados referentes aos usuários do sistema podem ser armazenados em outra coleção. Assim, a representação de um usuário com as informações básicas ficaria da seguinte maneira:

```
{
  "_id": 101,
  "nome": "Maria Silva",
  "idade": 28,
  "endereco": {
    "cidade": "São Paulo",
    "bairro": "Centro"
  }
  "livros_emprestados": [1, 3, 5]
}
```

Figura 2 – Script de um arquivo JSON referente aos dados do usuário a ser inserido no banco / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo script JSON com sete linhas de código referentes aos dados de usuários a serem inseridos no banco de dados. Os dados são referentes a: ID, NOME, IDADE, ENDEREÇO (CIDADE e BAIRRO). Uma última linha, denominada LIVROS EMPRESTADOS, indica os números 1, 3 e 5. Fim da descrição.

- **Relacionamentos:** os documentos podem conter referências uns aos outros. No exemplo anterior, a lista “LIVROS_EMPRESTADOS” contém os IDs dos livros emprestados pelo usuário.

Essa abordagem flexível e escalável é uma característica marcante dos bancos de dados NoSQL. Em um sistema mais complexo, é possível ter coleções adicionais para gêneros, autores, transações de empréstimos, entre outras coisas. A estrutura do banco de dados pode ser adaptada conforme as necessidades do aplicativo, sem a rigidez imposta pelos modelos relacionais tradicionais.

BANCO DE DADOS NOSQL DO TIPO CHAVE-VALOR

Trata-se de uma categoria única no universo dos sistemas de gerenciamento de dados, sendo caracterizados pela simplicidade e desempenho eficiente. Nesse modelo, as informações são armazenadas como pares de chave-valor, em que uma chave única está associada a um determinado valor.

Segundo Basso (2020), esse formato minimalista permite uma escalabilidade horizontal significativa, tornando-o ideal para ambientes que demandam alta performance e agilidade.

Ao contrário dos modelos relacionais, o banco de dados NoSQL chave-valor não impõe uma estrutura fixa aos dados. Cada valor pode ser qualquer tipo de informação, desde uma *string* simples até estruturas mais complexas, como JSON ou até mesmo objetos binários. Essa flexibilidade confere uma adaptabilidade excepcional aos requisitos específicos de cada aplicação.

A recuperação de dados nesse tipo de banco é extraordinariamente rápida, pois a busca por um valor é direta e instantânea, bastando a referência à chave associada. É importante perceber que a consulta por valores específicos é eficiente, mas a realização de consultas complexas pode ser mais desafiadora, se comparada a modelos mais elaborados.

Outra característica marcante é a capacidade de escalabilidade horizontal fluida. À medida que a quantidade de dados aumenta, é possível distribuir esses pares chave-valor em diversos servidores, mantendo a performance mesmo diante de volumes massivos de informações. Essa escalabilidade facilita a gestão de grandes volumes de dados em ambientes dinâmicos e crescentes.

Um exemplo prático desse modelo é a implementação em sistemas de armazenamento em cache, em que a chave representa a identificação, enquanto o valor demarca o próprio dado em si. Essa abordagem se mostra eficaz em cenários que demandam recuperação rápida e direta de informações, sem a complexidade associada a modelos mais robustos.

Os bancos de dados NoSQL chave-valor oferecem simplicidade, escalabilidade e desempenho notáveis, sendo uma escolha estratégica para aplicações que demandam agilidade e eficiência em ambientes de dados dinâmicos. Seja na gestão de cache, seja em catálogos de produtos on-line, seja em sistemas distribuídos, o modelo chave-valor se destaca como uma alternativa poderosa e versátil no cenário do NoSQL.

Software InfluxDB

Trata-se de uma solução robusta de banco de dados NoSQL projetada especificamente para armazenar e recuperar dados de séries temporais com eficiência e desempenho excepcionais.

Desenvolvido pela InfluxData, esse software se destaca em cenários em que a coleta, o armazenamento e a análise de dados temporais são cruciais, como em ambientes de monitoramento, Internet das Coisas (IoT) e análise de métricas. Conheça as principais características:

MODELO DE DADOS ESPECÍFICO PARA SÉRIES TEMPORAIS

adota um modelo de dados nativo que se alinha perfeitamente a séries temporais. Ele organiza os dados em campos, tags e timestamps, otimizando as consultas e as agregações temporais.

EFICIÊNCIA NA GRAVAÇÃO E RECUPERAÇÃO DE DADOS

é construído com foco na eficiência, permitindo a gravação rápida e a recuperação eficiente de grandes volumes de dados temporais. Essa característica é essencial em ambientes em que a captura de dados em alta frequência é necessária.

SQL-LIKE QUERY LANGUAGE (INFLUXQL)

possui uma sintaxe similar ao SQL, facilitando a transição para usuários familiarizados com bancos de dados relacionais. Isso simplifica a interação com o banco de dados, mesmo em ambientes complexos de séries temporais.

RETENÇÃO DE DADOS AUTOMÁTICA

oferece a funcionalidade de retenção automática de dados, permitindo a configuração de políticas de expiração com base no tempo. Essa característica é crucial para otimizar o armazenamento e garantir que apenas os dados relevantes sejam mantidos.

CAPACIDADE DE ESCALABILIDADE HORIZONTAL

suporta a escalabilidade horizontal, permitindo a distribuição em vários nós. Isso garante que a solução possa crescer de acordo com as demandas, mantendo o desempenho.

ECOSSISTEMA DE FERRAMENTAS

o InfluxDB é parte integrante do ecossistema TICK (Telegraf, InfluxDB, Chronograf, KAPACITOR). Ambos oferecem uma variedade de ferramentas para coleta, armazenamento, visualização e processamento de dados temporais.

Cenários de Aplicação do InfluxDB

Esse SBGD é especializado em séries temporais. Ele encontra ampla aplicação em cenários que demandam a coleta e a análise eficientes ao longo do tempo. Em ambientes de monitoramento, seja para infraestrutura de TI, redes ou dispositivos IoT, o InfluxDB se destaca pela capacidade de lidar com grandes volumes de dados temporais, oferecendo uma solução escalável e de alto desempenho.

Por exemplo, em sistemas de monitoramento de servidores, o InfluxDB é frequentemente utilizado para armazenar métricas, como CPU, uso de memória e latência de rede. Esses dados são coletados em intervalos regulares e armazenados no InfluxDB, possibilitando a geração de gráficos, alertas em tempo real e análises retrospectivas.

Com a capacidade de fazer consultas rápidas e eficientes em séries temporais, o InfluxDB contribui para uma gestão proativa, identificação de tendências e tomada de decisões informadas em diversos setores.

- **Monitoramento de infraestrutura e serviços:** é amplamente utilizado para monitorar o desempenho de servidores, aplicativos e serviços, fornecendo *insights* valiosos sobre o comportamento ao longo do tempo.
- **IoT e sensores:** devido à eficiência na manipulação de dados temporais, o InfluxDB é a escolha ideal para armazenar informações provenientes de dispositivos IoT e sensores.
- **Análise de métricas em tempo real:** a capacidade de processar dados em tempo real faz do InfluxDB uma ferramenta valiosa para a análise de métricas instantâneas.

O InfluxDB se destaca como uma opção notável para sistemas de séries temporais, oferecendo desempenho, eficiência e flexibilidade para lidar com os desafios únicos apresentados por dados temporais em diversos setores.



APROFUNDANDO

Dados temporais, segundo Basso (2021), referem-se ao cenário em que as informações são coletadas, armazenadas e analisadas considerando a dimensão de tempo. Nesse contexto, o elemento temporal é crucial, porque permite entender como os dados evoluem ao longo do tempo, revelando padrões, tendências e variações significativas. Essa abordagem é fundamental em cenários, como monitoramento, IoT e análise de séries, em que a temporalidade dos dados desempenha um papel crucial na compreensão das mudanças e na tomada de decisões informadas. Essa contextualização enriquece a interpretação dos dados, possibilitando uma visão mais completa e dinâmica do comportamento dos fenômenos representados pelas informações armazenadas.

Software Redis

Trata-se de um exemplo proeminente do software de gerenciamento de bancos de dados NoSQL do tipo chave-valor, projetado para proporcionar desempenho excepcional e escalabilidade horizontal. Esse sistema robusto se destaca pela simplicidade e eficiência, sendo amplamente utilizado em cenários que demandam respostas rápidas e baixa latência.

- **Estrutura Chave-Valor:** no Redis, os dados são armazenados como pares chave-valor, em que cada chave única está associada a um valor correspondente. Essa abordagem permite a flexibilidade na representação de dados, desde *strings* simples até estruturas de dados mais complexas, incluindo listas, conjuntos e *hashes*. Por exemplo, podemos armazenar dados de um usuário da seguinte forma:

```
SET usuario:1 '{"nome": "João", "idade": 30, "cidade": "São Paulo"}'
```

Figura 3 - Linha de código para armazenar os dados referentes ao usuário / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta com uma linha de código demonstrando o comando SET para inserir dados na tabela USUARIO, com valores referentes à NOME, IDADE e CIDADE. Fim da descrição.

- **Alta Performance:** a principal vantagem do Redis está na rapidez com que pode recuperar dados. Devido à arquitetura em memória e à natureza direta da busca por chaves, as consultas são extremamente eficientes, tornando-o ideal para casos de uso, como armazenamento em cache, filas de mensagens e contadores em tempo real.
- **Estruturas de Dados Avançadas:** além do simples armazenamento chave-valor, o Redis oferece estruturas de dados avançadas que permitem manipulações complexas diretamente no servidor. Isso inclui operações, como incremento atômico, manipulação de listas, conjuntos e ordenação de conjuntos, proporcionando versatilidade em diversas aplicações.

- **Persistência Opcional:** apesar de ser conhecido pela natureza em memória, o Redis oferece opções de persistência de dados para casos em que a recuperação de dados após uma reinicialização é crucial. Essa característica permite que o Redis seja utilizado em uma variedade de cenários, desde o simples armazenamento em cache até aplicações mais robustas.
- **Escalabilidade Horizontal:** o Redis suporta escalabilidade horizontal, permitindo distribuir os dados em vários nós para atender a demandas crescentes. Essa característica é vital em ambientes que necessitam de expansão contínua para lidar com volumes massivos de dados.

Suponha que estamos desenvolvendo um aplicativo *web* e precisamos gerenciar sessões de usuário de forma eficiente. O Redis é uma escolha ideal para esse cenário, oferecendo alta velocidade e flexibilidade. A seguir, é exposto um exemplo prático.

- **Configuração Inicial:** após instalar e configurar o Redis, conectamos o nosso aplicativo *web* ao servidor Redis.
- **Início de Sessão de Usuário:** quando um usuário realiza *login*, é gerada uma chave única para a sessão no Redis. Essa chave pode ser composta pelo ID desse usuário e um identificador único de sessão. Exemplo: *sessao:usuario123:abcd*.

```
// Exemplo em Node.JS utilizando biblioteca Redis

const redis = require('redis');
const client = redis.createClient();

const userID = 123;
const sessionID = 'abcd';
const chaveSessao = 'sessao:usuario${userID}:${sessionID}';

// Armazenar dados da sessão do Redis
client.set(chaveSessao, JSON.stringify({ userID, username: 'joao', tempoExpiracao: 3600 }));
```

Figura 4 - Scripts NodeJS para criar uma sessão de conexão no banco de dados Redis / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta com um script para efetuar uma sessão para conexão de uma aplicação com um banco de dados Redis. São seis linhas de código nas quais são fornecidas cinco constantes (CONST). São elas: REDIS, CLIENT, USERID, SESSIONID e CHAVESESSAO. Ainda, há uma última linha, na qual a sessão será executada. Fim da descrição.

- **Recuperação de Dados da Sessão:** ao precisar dos dados da *session* de conexão de um usuário, as informações são recuperadas utilizando a chave associada à essa sessão.

```
// Exemplo de recuperação de dados da sessão em Node.js

client.get(chaveSessao, (err, dadosSessao) => {

  if (err) throw err;

  const informacoesSessao = JSON.parse(dadosSessao);
  console.log('Informações da Sessão: ', informacoesSessao);

});
```

Figura 5 - Script de recuperação de dados da *session* escrito em NodeJS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta com um script escrito em NodeJS para recuperação de dados oriundos de uma *session* com quatro linhas de código, incluindo uma verificação de erro em caso de divergências de parâmetros de conexão (tratamento de erro). Fim da descrição.

- **Atualização e Expiração de Sessão:** é possível atualizar informações na sessão, como o tempo de expiração, para garantir que os dados estejam sempre atualizados. Além disso, será estipulado um tempo de expiração para as chaves no Redis, garantindo a limpeza automática de sessões inativas e a alta performance do sistema.

```
// Exemplo de atualização de dados da sessão e configuração de tempo de expiração

const novoTempoExpiracao = 7200; // 2 horas

client.setex(chaveSessao, novoTempoExpiracao, dadosSessao);
```

Figura 6 - Script de atualização de dados da *session* escrito em NodeJS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta com um script em NodeJS para atualização de dados a partir de uma *session* com duas linhas de código, incluindo uma definição de constante para definir o tempo de expiração dessa *session*, além da função propriamente dita de atualização. Fim da descrição.

O Redis exemplifica a eficiência e a simplicidade que um banco de dados NoSQL chave-valor pode oferecer. Seja em ambientes que exigem alta performance, como sistemas de jogos digitais on-line, seja em plataformas de redes sociais, o Redis se destaca como uma escolha notável para o gerenciamento de dados em tempo real e aplicações sensíveis à latência.

O termo **latência** se refere ao tempo que um sistema leva para processar e responder a uma requisição. É a medida de demora entre o envio de uma ação e a obtenção da resposta desejada. Geralmente expressa em milissegundos, pode ser influenciada por diversos fatores, incluindo a velocidade de transmissão de dados, a eficiência do hardware, a capacidade de processamento e a distância física entre os componentes do sistema.



ZOOM NO CONHECIMENTO

Baixa latência é desejada em ambientes em que a resposta rápida é crítica, como em sistemas financeiros, jogos on-line e redes de comunicação, enquanto a **alta latência** pode impactar negativamente o desempenho e a experiência do usuário.

Um banco de dados NoSQL, segundo Sadalage e Fowler (2013), do tipo chave-valor é caracterizado pela simplicidade e eficiência na gestão de dados. Nesse modelo, o alcance desse tipo do banco abrange aplicações que demandam alta performance e flexibilidade, especialmente em casos em que a estrutura dos dados pode variar dinamicamente. A implementação dele é ideal para sistemas que lidam com grandes volumes de dados simples, como caches, sistemas de sessões de usuário e armazenamento de configurações, destacando-se pela velocidade nas operações de leitura e escrita.

A simplicidade desse modelo oferece vantagens notáveis em termos de escalabilidade horizontal, facilitando a distribuição e o gerenciamento eficaz de dados em ambientes que exigem flexibilidade e resposta rápida. Esses bancos de dados NoSQL são amplamente utilizados em cenários em que a agilidade na recuperação de informações é crucial, destacando-se como uma solução eficaz para aplicações que buscam otimizar o desempenho e a gestão eficiente de dados com estrutura simples e direta.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Ao adentrar o universo do banco de dados NoSQL orientado a chave-valor, você inicia uma jornada de descobertas que transcende os limites dos modelos tradicionais. Nesse cenário, o entendimento profundo de como as chaves únicas identificam registros e como os valores associados a essas chaves podem variar amplia o espectro de possibilidades na gestão de dados. Essa base sólida te proporciona uma compreensão crítica da flexibilidade inerente a esse modelo, uma habilidade valiosa no contexto profissional em constante transformação.

No ambiente profissional futuro, o conhecimento detalhado de tecnologias específicas que incorporam o paradigma chave-valor, como Redis e DynamoDB, torna-se uma vantagem estratégica. Essas ferramentas são cada vez mais adotadas em diversos setores, impulsionando aplicações inovadoras que demandam escalabilidade e desempenho eficiente. Você, ao internalizar os princípios dessas tecnologias, estará apto(a) a contribuir para projetos que demandam gerenciamento ágil de grandes volumes de dados, um diferencial competitivo no mercado de TI.

A compreensão prática de como essas soluções se integram ao ecossistema tecnológico mais amplo é essencial. Ao aliar conhecimentos de chave-valor com outras tecnologias, como sistemas de mensageria, *frameworks* de desenvolvimento e ferramentas analíticas, você estará preparado(a) para enfrentar desafios do mundo real. Essa abordagem integrada não apenas fortalece as habilidades técnicas, mas também fomenta a capacidade do profissional de TI de se adaptar a ambientes dinâmicos, um requisito cada vez mais crucial em uma indústria em constante evolução.

Construir conexões entre o conhecimento construído sobre o banco de dados NoSQL orientado a chave-valor e o ambiente profissional futuro é uma tarefa estratégica para você. Essa integração não apenas solidifica a base técnica, mas também proporciona uma visão abrangente das aplicações práticas e das oportunidades que esse modelo traz consigo. Ao se tornar um(a) arquiteto(a) habilitado(a) nesse campo, você não apenas se prepara para enfrentar os desafios da TI moderna, mas também se destaca como um(a) profissional capacitado(a) a liderar e inovar em um cenário cada vez mais dinâmico.

AUTOATIVIDADE

1. O termo NoSQL diz respeito a uma abordagem de gerenciamento de dados que se distancia dos relacionais tradicionais. Esse tipo de banco de dados busca proporcionar flexibilidade e escalabilidade em ambientes em que a estrutura dos dados pode variar dinamicamente. Ao contrário dos modelos relacionais que adotam esquemas fixos, o NoSQL aceita diferentes formas de armazenamento, como documentos, pares chave-valor, grafos e famílias de colunas. Essa diversidade de modelos permite a adaptação a diferentes necessidades de aplicação, tornando-se uma escolha valiosa para ambientes modernos e dinâmicos, em que a agilidade na manipulação de dados é crucial.

Qual característica define a estrutura de um banco de dados NoSQL do tipo chave-valor? Assinale a alternativa correta:

- a) Esquema fixo e rígido.
- b) Armazenamento hierárquico de dados.
- c) Modelagem baseada em documentos.
- d) Relacionamentos complexos entre tabelas.
- e) Ênfase na linguagem SQL para consultas.

2. Ao contrário dos bancos de dados SQL, o NoSQL segue uma estrutura tabular e esquematizada, adotando abordagens mais variadas, como chave-valor, documentos, colunas ou grafos. Essa diversidade permite uma adaptabilidade notável a diferentes tipos de dados, cenários e necessidades, promovendo escalabilidade horizontal e eficiência em ambientes com grandes volumes de informações não estruturadas. Assim, o NoSQL desempenha um papel fundamental na era da computação distribuída e do Big Data, oferecendo soluções especializadas para lidar com a crescente complexidade e variedade de dados na era digital.

Considere as seguintes afirmativas sobre os tipos de bancos de dados NoSQL:

- I - Bancos de dados do tipo chave-valor armazenam dados como pares de chave-valor, sendo eficientes para operações de busca direta.
- II - Bancos de dados do tipo documento permitem o armazenamento de dados em documentos flexíveis, muitas vezes, usando formatos, como JSON ou BSON.
- III - Bancos de dados do tipo grafo são adequados para representar relações complexas entre entidades, utilizando nós e arestas.
- IV - Bancos de dados do tipo colunar organizam os dados em colunas, proporcionando uma eficiente busca e agregação de dados.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

3. O modelo chave-valor é uma abordagem simples e eficiente para armazenamento em que cada informação é identificada por uma chave única associada a um valor correspondente. Essa estrutura proporciona agilidade na recuperação e na manipulação de dados, permitindo acesso direto a informações específicas por meio de chaves. Os bancos de dados que adotam esse modelo oferecem flexibilidade na definição de valores, tornando-se uma escolha versátil para cenários que demandam escalabilidade e eficiência na gestão de dados, sem a rigidez estrutural de modelos tradicionais.

A respeito dos bancos de dados NoSQL do tipo chave-valor, analise as afirmativas a seguir:

- I - Cada dado é identificado por uma chave única, e o valor associado a essa chave pode conter apenas dados numéricos.
- II - Esses bancos oferecem uma abordagem estruturada e rígida para o armazenamento de dados, sem permitir flexibilidade na definição de valores associados às chaves.
- III - A principal característica é a descentralização da gestão de dados, possibilitando a escalabilidade horizontal de maneira eficiente.
- IV - Bancos de dados NoSQL chave-valor também podem ser aproveitados em ambientes que demandam estruturas de dados altamente normalizadas.

É correto o que se afirma em:

- a) I e IV, apenas.
- b) II e III, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) II, III e IV, apenas.

REFERÊNCIAS

ALVES, W. P. **Banco de dados: teoria e desenvolvimento**. São Paulo: Érica, 2021.

BASSO, D. E. **Big Data**. Curitiba: Contentus, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

SADALAGE, P. J.; FOWLER, M. **Nosql Essencial**. São Paulo: Novatec, 2013.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

SILVA, L. F. C. et al. **Banco de dados não relacional**. Porto Alegre: SAGAH, 2021.

GABARITO

1. Opção C.

Os bancos de dados NoSQL do tipo chave-valor se caracterizam pela flexibilidade na representação dos dados utilizando estruturas de documentos para o armazenamento. Diferentemente dos sistemas relacionais, não têm um esquema fixo e rígido, permitindo a inclusão de campos adicionais nos documentos conforme necessário.

2. Opção D.

Os bancos de dados NoSQL do tipo chave-valor utilizam pares de chave-valor para armazenar dados. Os do tipo documento permitem estruturas flexíveis, como JSON ou BSON, e os do tipo grafo são ideais para representar relações complexas entre entidades. A afirmativa IV está incorreta, pois os bancos de dados colunares organizam os dados em colunas, mas não necessariamente proporcionam uma eficiente busca e agregação.

3. Opção C.

A afirmativa I está incorreta, pois os valores associados às chaves em bancos de dados NoSQL do tipo chave-valor podem ser diversos tipos de dados, não se limitando apenas a numéricos. A afirmativa II está incorreta, uma vez que a característica marcante desses bancos é a flexibilidade na definição de valores, não impondo uma estrutura rígida. A afirmativa III está correta, pois a descentralização é uma das principais vantagens, possibilitando escalabilidade horizontal. A afirmativa IV está correta, pois esses bancos são adequados tanto para ambientes com estruturas normalizadas quanto para ambientes mais flexíveis e dinâmicos.



CONEXÕES E RELACIONAMENTOS EM BANCOS DE DADOS ORIENTADOS A GRAFOS

MINHAS METAS

- Entender o modelo de grafos.
- Dominar as linguagens de consulta específicas para bancos de dados orientados a grafos.
- Desenvolver habilidades para projetar eficientemente estruturas de grafos que atendam aos requisitos específicos de um sistema.
- Aplicar algoritmos essenciais para grafos, explorando diferentes técnicas.
- Aprender a integrar bancos de dados orientados a grafos com diferentes aplicações.
- Adquirir habilidades para otimizar o desempenho de consultas em grandes conjuntos de dados.
- Aplicar os conhecimentos adquiridos em projetos práticos.

INICIE SUA JORNADA

Iniciar a jornada pelos estudos dos bancos de dados NoSQL orientados a grafos é embarcar em uma experiência fascinante de descobertas. Ao contrário dos modelos relacionais, essa abordagem oferece uma visão inovadora para representar e explorar as relações entre os dados, focando em valores que não apresentam nenhum tipo de estrutura prévia, isto é, que são armazenados sem tratamento anterior, no formato em que foram coletados. Esse é o ponto de partida para uma jornada que abrirá as portas para entender a complexidade das interconexões em sistemas de informação modernos.

Ao mergulhar nos estudos, você explorará a modelagem de grafos, a qual permite representar entidades e as respectivas interações de maneira visual e eficiente. Essa abordagem é particularmente poderosa para os domínios em que as relações entre os dados são fundamentais, como redes sociais, sistemas de recomendação e logística. Compreender como os grafos capturam nuances e conexões intrínsecas proporcionará uma nova perspectiva sobre a gestão de informações.

Essa jornada também incluirá a compreensão efetiva dos desafios enfrentados na implementação de bancos de dados orientados a grafos. Algumas questões, tais como escalabilidade e otimização de consultas, serão exploradas, proporcionando uma visão abrangente sobre as considerações práticas na adoção dessa tecnologia. Aprender a contornar esses desafios é uma parte integrante do processo de aprimoramento das habilidades necessárias para projetar e gerenciar eficientemente esses sistemas.

Ao concluir a introdução desse tipo de banco de dados, você estará preparado(a) para enfrentar os desafios de um ambiente profissional em constante evolução. Essa jornada de aprendizado não apenas proporcionará *insights* valiosos sobre tecnologias emergentes, mas também abrirá portas para oportunidades profissionais empolgantes. Esse é apenas o começo de uma jornada que promete expandir os seus horizontes no vasto campo da tecnologia da informação e se destacar no mercado de trabalho.

Aperte os cintos e boa viagem!

**PLAY NO CONHECIMENTO**

Bem-vindo(a) ao *Desbravando Dados*, o podcast que mergulha nas profundezas do vasto campo da gestão de dados. Neste episódio, exploraremos a fascinante dimensão dos bancos de dados NoSQL orientados a grafos. Se você está curioso(a) para entender como essa abordagem inovadora pode transformar a maneira como podemos administrar informações inter-relacionadas entre si em bases de dados de grande porte, você está no lugar certo.

Prepare-se para uma jornada em que desvendaremos o poder por trás da modelagem de grafos, representando entidades e as respectivas conexões de maneira intuitiva. Ao longo do episódio, apresentaremos casos reais de aplicação em que essa tecnologia brilha, desde redes sociais até sistemas de recomendação. Você ficará maravilhado(a) com a capacidade desses bancos de dados em revelar conexões efetivas que, muitas vezes, passam despercebidas nos sistemas e modelos mais tradicionais.

Acompanhe as nossas conversas com especialistas da área, que compartilharão *insights* valiosos sobre as vantagens e os desafios de adotar bancos de dados NoSQL orientados a grafos. Além disso, discutiremos as tendências emergentes e como essa tecnologia está moldando o futuro da gestão de dados. Pronto(a) para desbravar conosco as trilhas fascinantes dos dados interconectados? Então, sintonize no *Desbravando Dados* e embarque nessa jornada exploratória.

VAMOS RECORDAR?

Na era moderna da Tecnologia da Informação (TI), o NoSQL é um conceito que emergiu como uma alternativa vital para lidar com volumes massivos de dados e atender a requisitos específicos de flexibilidade e escalabilidade. Diferentemente dos bancos de dados relacionais, os sistemas NoSQL, ou "Not Only SQL", adotam modelos de dados mais flexíveis e não exigem uma estrutura de esquema fixa. Essa característica torna esses bancos ideais para ambientes dinâmicos, em que a estrutura dos dados pode evoluir rapidamente.

O NoSQL pode ser categorizado em diversos modelos, incluindo chave-valor, orientado a documentos, orientado a colunas e orientado a grafos. Cada modelo tem características específicas, oferecendo vantagens para diferentes tipos de aplicativos e cenários de uso. O modelo chave-valor, por exemplo, é eficaz para operações simples de leitura e gravação, enquanto o orientado a documentos é adequado para armazenar e consultar dados semelhantes a documentos.

Além disso, os bancos de dados NoSQL são frequentemente associados a tecnologias que suportam operações distribuídas, permitindo a escalabilidade horizontal para lidar com grandes volumes de dados e tráfego intenso, com uma quantidade enorme de acessos simultâneos em um sistema. Compreender esses conceitos fundamentais com mais profundidade é crucial para os estudantes de Tecnologia da Informação (TI), pois eles proporcionarão uma base sólida para explorar e implementar soluções eficientes em diversos contextos de desenvolvimento de sistemas de software e no gerenciamento de dados.

<https://www.youtube.com/watch?v=XXYooZfglls>

DESENVOLVA SEU POTENCIAL

ESTRUTURA DE ARMAZENAMENTO PERMANENTE DE DADOS

O armazenamento permanente de dados em um sistema de banco de dados é um elemento fundamental para garantir a durabilidade e a disponibilidade das informações ao longo do tempo. Essa prática diz respeito à capacidade de manter

os dados de maneira persistente, mesmo após o encerramento do sistema, proporcionando uma base sólida para a continuidade e a integridade das operações.

A implementação do armazenamento permanente envolve o uso de dispositivos de armazenamento físicos, como discos rígidos ou SSDs, nos quais os dados são gravados de forma duradoura. Cada entidade no banco de dados, como tabelas, registros e relacionamentos, é organizada e estruturada para otimizar a eficiência na recuperação, quando necessário. Estratégias, como *logs* de transações, são empregadas para registrar alterações nos dados, facilitando a recuperação em caso de falhas ou interrupções inesperadas.

A confiabilidade do armazenamento permanente é crucial para manter a consistência e a segurança dos dados. Mecanismos de backup e recuperação são, segundo Silberschatz (2020), implementados para proteger contra perdas acidentais, garantindo a resiliência do sistema. A evolução contínua dessas práticas busca aprimorar o desempenho, a escalabilidade e a segurança, assegurando que as informações permaneçam acessíveis e íntegras ao longo do tempo, independentemente das mudanças no ambiente ou na aplicação.

A integridade do armazenamento permanente é essencial para o sucesso de diversas aplicações, desde sistemas empresariais até soluções de armazenamento em nuvem.

Essa prática, segundo Elmasri e Navathe (2018), não apenas fornece uma base robusta para a continuidade operacional, mas também atua como alicerce para a confiabilidade e a eficácia dos sistemas de informação em um cenário tecnológico em constante evolução.

ENTENDENDO O CONCEITO DE GRAFOS

Os grafos, no contexto matemático, são estruturas que representam relações entre objetos distintos. Esses objetos, chamados de vértices, são conectados por arestas que denotam a relação entre eles. A teoria dos grafos, iniciada por Euler no século XVIII, fornece uma linguagem abstrata e poderosa para modelar uma variedade de problemas complexos em diversas disciplinas.

Os grafos são caracterizados pela estrutura simples e, ao mesmo tempo, versátil. Vértices podem representar entidades, como pessoas ou locais, enquanto as arestas simbolizam as conexões ou as relações entre essas entidades. A direção das arestas, quando presente, indica a natureza da relação (direcionada ou não direcionada), conferindo flexibilidade ao modelo.

Na área de TI, os grafos, segundo Basso (2020), desempenham um papel fundamental, sendo amplamente utilizados para representar redes sociais, sistemas de transporte, relações de dependência em código-fonte, entre outros. A aplicação de algoritmos de grafos permite resolver problemas complexos, como encontrar o caminho mais curto em uma rede ou identificar *clusters* em conjuntos de dados massivos.

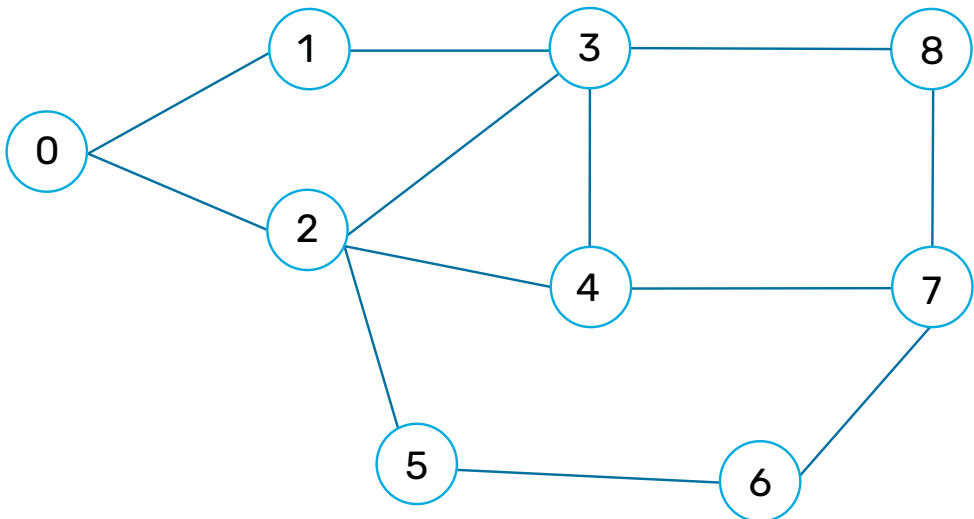


Figura 1 - Estrutura que representa a lógica de grafos / Fonte: o autor.

Descrição da Imagem: a figura apresenta nove nós interligados entre si em uma estrutura de grafos. Fim da descrição.

Diversos algoritmos foram desenvolvidos para explorar e manipular grafos, como o algoritmo de Dijkstra para caminho mais curto e o algoritmo de PageRank, usado pelo Google para classificar páginas da web. A estrutura gráfica permite solucionar problemas que, de outra forma, seriam intratáveis, contribuindo para avanços significativos em algoritmos e Inteligência Artificial (IA).

À medida que a complexidade dos sistemas de informação aumenta, a importância dos grafos na TI só cresce. Bancos de dados orientados a grafos e ferramentas de análise exploratória de dados baseadas em grafos se tornam cada vez mais populares. Essa abordagem oferece uma visão holística e intuitiva dos dados, tornando-se uma peça-chave na resolução de problemas e na extração de *insights* valiosos. O uso inovador dos grafos promete continuar moldando o futuro da Tecnologia da Informação (TI) de maneira empolgante e impactante.

Um exemplo prático de aplicação de grafos na área de Tecnologia da Informação utilizando redes sociais: considere uma plataforma social em que os usuários são representados por vértices e as conexões de amizade entre eles são representadas por arestas. Cada usuário é um nó e a amizade entre dois usuários é uma aresta que conecta esses nós. Nesse contexto, suponha que Maria, João, e Ana sejam usuários da plataforma. Assim, é possível representar essa rede social por meio de um grafo não direcionado, em que cada usuário é um vértice e a amizade é uma aresta. Temos, portanto, três vértices (Maria, João, Ana) conectados por arestas que representam as amizades entre eles. Se Maria é amiga de João e João é amigo de Ana, é possível visualizar essa relação por um grafo em que temos três vértices interconectados. Essa representação gráfica oferece uma visão clara das relações sociais na plataforma.

A utilidade prática vai além da visualização. Algoritmos de grafos podem ser aplicados para identificar comunidades dentro da rede social, sugerir novas amizades com base em amigos em comum ou até mesmo avaliar a influência dos usuários na propagação de informações na rede.

Assim, esse exemplo ilustra como a teoria dos grafos é aplicada na prática, proporcionando *insights* valiosos para otimizar experiências em redes sociais e diversas outras aplicações na Tecnologia da Informação (TI).

PARADIGMA DA ORIENTAÇÃO À GRAFOS

Esse paradigma representa uma abordagem inovadora na modelagem e na manipulação de dados. Diferentemente dos modelos tradicionais baseados em tabelas, o enfoque em grafos explora as relações complexas entre entidades por meio de estruturas de nós e arestas, oferecendo uma representação flexível e eficiente.

**ZOOM NO CONHECIMENTO**

No centro desse paradigma, temos os “nós”, que representam entidades no sistema. Esses nós, segundo Basso (2020), podem ser comparados a registros ou objetos em outros modelos de banco de dados. Contudo, a principal diferença está na forma como eles se conectam. As “arestas” são as ligações entre esses nós e encapsulam as relações, fornecendo um método dinâmico para expressar conexões e hierarquias.

A flexibilidade do modelo de grafos é evidente na capacidade de lidar com estruturas de dados complexas, como redes sociais, sistemas de recomendação e cadeias de suprimentos. Cada nó e aresta pode conter propriedades adicionais, permitindo uma representação rica e detalhada das informações.

Os bancos de dados NoSQL orientados a grafos, como o Neo4j, são amplamente utilizados nesse contexto. Eles oferecem consultas eficientes para traçar relações, identificar padrões e realizar análises complexas. A linguagem de consulta Cypher, por exemplo, permite expressar de forma elegante as interações entre nós e arestas.

Além disso, a escalabilidade é uma característica marcante desses sistemas. À medida que o número de entidades e relações cresce, os bancos de dados orientados a grafos mantêm um desempenho consistente, proporcionando uma solução robusta para aplicações que demandam análises profundas de relacionamentos.

Nesse contexto, o paradigma de orientação a grafos em bancos de dados NoSQL oferece, segundo Silva *et al.* (2021), uma abordagem inovadora e poderosa para modelar e explorar dados, destacando-se em cenários que exigem a representação eficiente de relações complexas entre entidades.

A escalabilidade é uma característica marcante desses sistemas

O SGBD Neo4j

É um Sistema de Gerenciamento de Banco de Dados (SGBD) orientado a grafos projetado para fornecer uma estrutura flexível e eficiente para modelar e consultar dados baseados em relacionamentos complexos.

A base fundamental do Neo4j é o conceito de “nós”. Cada nó representa uma entidade no sistema e pode armazenar propriedades que descrevem características específicas dessa entidade.

Por exemplo, em uma rede social, um nó poderia representar um usuário com propriedades, como nome, idade e cidade. Os relacionamentos entre os “nós” são representados por “arestas”. As arestas conectam os nós e podem ter propriedades próprias. Essa estrutura permite modelar relações entre entidades de maneira intuitiva, proporcionando uma visão clara das conexões em um conjunto de dados.

O Neo4j utiliza uma linguagem de consulta chamada Cypher. Desenvolvida especificamente para trabalhar com grafos, essa linguagem permite expressar consultas de maneira declarativa e elegante. Comandos, como MATCH, CREATE e RETURN, são utilizados para buscar padrões, criar novos nós e arestas e retornar os resultados desejados.

Um dos pontos fortes do Neo4j, segundo Sadalage e Fowler (2013), é a capacidade de escalar horizontalmente, suportando, por exemplo, a distribuição dos dados em *clusters* para atender a demandas crescentes de armazenamento e consulta, garantindo que a performance seja mantida mesmo diante do aumento da complexidade do grafo.

Além disso, o SGBD oferece uma interface gráfica amigável chamada Neo4j Browser, que permite aos usuários explorar visualmente o grafo, executar consultas interativas e compreender melhor a estrutura subjacente dos dados.

Em síntese, a estrutura do Neo4j é construída em torno do poderoso paradigma de grafos, proporcionando uma abordagem única e eficaz para a gestão de dados. Seja modelando redes sociais, sistemas de recomendação ou qualquer cenário que envolva relações complexas, o software, segundo Sadalage e Fowler (2013), destaca-se como uma ferramenta versátil e robusta.

Vamos explorar um exemplo prático de criação e manipulação de um banco de dados no Neo4j utilizando o cenário de uma rede social simples.

- **Instalação e configuração:** antes de tudo, é necessário instalar o Neo4j e iniciar o servidor. Após a instalação, utilizando o browser, use a interface gráfica para interação.
- **Criação de nós:** para representar usuários em uma rede social, é possível criar nós.

Para isso, será utilizada a seguinte *query* Cypher (CQL) para gerar três nós diferentes que representarão os usuários do sistema:

```
CREATE (: Usuario { nome: 'Alice', idade: 25 }),
      (: Usuario { nome: 'Bob', idade: 30 }),
      (: Usuario { nome: 'Carol', idade: 28 });
```

Figura 2 - Script Cypher para criar os usuários utilizando a instrução CREATE / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com três linhas de código referentes aos dados de usuários a serem inseridos no banco de dados. Os dados são referentes a: NOME e IDADE. Fim da descrição.

Essa *query* cria três nós do tipo “Usuario” com propriedades, como nome e idade.

- **Criação de relacionamentos:** agora, criaremos relacionamentos entre esses usuários.

Por exemplo, podemos indicar que Alice e Bob são amigos:

```
MATCH (alice:Usuario { nome: 'Alice' }), (bob:Usuario { nome: 'Bob' })
CREATE (alice) - [:AMIGO_DE] -> (bob);
```

Figura 3 - Script Cypher para associar os usuários utilizando a instrução MATCH e criar o relacionamento com a instrução CREATE / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com duas linhas de código referentes aos usuários a serem relacionados entre si no banco de dados. Os usuários são ALICE e BOB, criando o relacionamento AMIGO_DE. Fim da descrição.

A *query* citada criará um relacionamento entre os usuários que será chamado de “AMIGO_DE” entre Alice e Bob.

- **Consulta:** neste contexto, é possível realizar consultas para explorar as relações.

A *query* a seguir retornará todos os amigos de Alice:

```
MATCH (alice:Usuario { nome: 'Alice' }) - [:AMIGO_DE] -> (amigo)
RETURN amigo.nome;
```

Figura 4 - Script Cypher para criar o relacionamento entre os usuários utilizando a instrução MATCH e retornando os nomes dos amigos nesse relacionamento / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com duas linhas de código referentes aos usuários a serem relacionados entre si no banco de dados. O usuário ALICE se dá mediante o relacionamento AMIGO_DE. São retornados os nomes desses amigos. Fim da descrição.

O resultado seria “Bob”, indicando que ele é um dos amigos de Alice que está cadastrado no banco e que foi inserido no relacionamento AMIGO_DE.

- **Atualização e exclusão:** também é possível atualizar ou excluir dados na estrutura de grafos.

Por exemplo, para atualizar a idade do usuário Carol:

```
MATCH (carol:Usuario { nome: 'Carol' })
SET carol.idade = 29;
```

Figura 5 - Script Cypher para, mediante a instrução MATCH, selecionar o usuário, e SET para efetuar a atualização de dados / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com duas linhas de código referentes aos usuários a serem relacionados entre si no banco de dados. Na primeira linha, encontra-se a instrução MATCH para identificar o usuário e, na segunda, o SET, para, efetivamente, atualizar a idade do usuário selecionado. Fim da descrição.

Para excluir o relacionamento de amizade entre Alice e Bob:

```
MATCH (alice:Usuario { nome: 'Alice' }) - [rel:AMIGO_DE] -> (bob:Usuario { nome: 'Bob' })
DELETE rel;
```

Figura 6 - Script Cypher para, mediante a instrução MATCH, selecionar o usuário, e DELETE para efetuar a exclusão de dados / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com duas linhas de código referentes aos usuários a serem relacionados entre si no banco de dados. Na primeira linha, encontra-se a instrução MATCH para identificar o usuário e, na segunda, o DELETE, para, efetivamente, realizar a exclusão do relacionamento dos usuários selecionados. Fim da descrição.

Essas operações mostram a flexibilidade na manipulação dos dados.

Esse exemplo ilustra a criação, a manipulação e a consulta de dados em um banco de dados Neo4j utilizando o poder do modelo de grafos para representar e explorar relações complexas de maneira eficaz.

CYPHER QUERY LANGUAGE (CQL)

CQL, ou simplesmente Cypher, é uma linguagem de consulta projetada especificamente para trabalhar com bancos de dados orientados a grafos. Ela é amplamente utilizada no SGBD Neo4j.

Essa linguagem oferece uma abordagem intuitiva e expressiva para criar, modificar e consultar grafos de maneira eficaz e robusta.

Sintaxe simbólica: a principal característica do Cypher é, segundo Sadalage e Fowler (2013), a sintaxe simbólica, que se assemelha a padrões de grafos. Os nós são representados por parênteses, os relacionamentos por traços (ou setas) e as propriedades dos nós estão entre as chaves. Isso facilita a leitura e a escrita das *queries*, tornando-as mais acessíveis mesmo para iniciantes.

Padrões de grafos: o Cypher permite a especificação de padrões de grafos nas próprias *queries*. Por exemplo, se for necessário encontrar usuários que são amigos de outros usuários, é possível expressar isso de forma clara na linguagem. Essa abordagem baseada em padrões simplifica a representação de relações complexas entre os dados.

Consulta e correspondência: as *queries* Cypher são orientadas a padrões, permitindo, assim, uma correspondência eficiente e uma filtragem de dados com bastante facilidade. A linguagem é otimizada para consultas de relacionamentos, tornando-a particularmente eficiente em cenários em que os vínculos entre os dados são cruciais para o sistema.

Operações e modificações: além de consultas, Cypher suporta operações de modificação de dados. Podemos criar, atualizar ou excluir nós e relacionamentos conforme necessário. Essas operações são realizadas de maneira intuitiva e segura, mantendo a consistência do grafo.

Compreensão visual e navegação: Cypher não apenas permite a execução de consultas complexas, mas também oferece uma compreensão visual imediata. As *queries*, muitas vezes, refletem a estrutura visual de um grafo, facilitando a interpretação e a navegação por relações intrincadas.

Assim, é possível afirmar que o Cypher Query Language é uma ferramenta poderosa para explorar e manipular dados em bancos de dados orientados a grafos, proporcionando para os administradores uma abordagem intuitiva e expressiva para a modelagem e a consulta de informações conectadas.

Considere um cenário em que é necessário modelar uma rede social e se busca encontrar usuários e suas interações. Suponha que o objeto seja encontrar todos os usuários cadastrados no sistema que sejam amigos de outro usuário chamado João.

Para isso, será utilizada a sintaxe do Cypher para expressar essa consulta:

```
MATCH (joao:Usuario { nome: 'João' })-[:AMIGO]-(amigo)
RETURN amigo.nome AS NomeDoAmigo
```

Figura 7 - Script de um arquivo JSON referente aos dados do livro a ser inserido no banco / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script JSON com 10 linhas de código referentes aos dados dos livros a serem inseridos no banco de dados. Os dados são referentes a: ID, TITULO, AUTOR, ANO_PUBLICACAO, GENERO, QUANTIDADE_EXEMPLARES, EDITORA (NOME e LOCALIZACAO). Fim da descrição.
#IMAGEM DE CONTEÚDO

Neste exemplo:

MATCH: indica a correspondência de padrões.

(joao:Usuario {nome: 'João'}): representa um nó de usuário com a propriedade nome igual a 'João'.

[:AMIGO]: indica um relacionamento 'AMIGO' entre João e os amigos.

-(amigo): define o nó de amigo que está relacionado com João.

RETURN amigo.nome AS NomeDoAmigo: retorna o nome do amigo como resultado da consulta.

Essa *query* encontra todos os amigos de João e retorna os nomes desses amigos. O resultado é uma lista dos amigos conectados a João na rede social. Essa é apenas uma das muitas consultas que podem ser realizadas com o Cypher para explorar eficientemente dados de grafos.

As diferenças entre o CQL e SQL

A Cypher Query Language (CQL) e a Structure Query Language (SQL) são duas linguagens de consulta distintas utilizadas com o objetivo de interagir com diferentes tipos de bancos de dados. Enquanto o SQL é amplamente utilizado em bancos de dados relacionais, a Cypher é específica para aqueles que são orientados a grafos, como o Neo4j.

A seguir, são exibidas algumas das principais diferenças entre essas linguagens:

MODELO DE DADOS

SQL: projetada para bancos de dados relacionais baseados em tabelas, com estrutura rígida e esquema predefinido.

Cypher: especificamente projetada para bancos de dados de grafos, que representam dados, como nós e relacionamentos, proporcionando uma estrutura mais flexível e dinâmica.

SINTAXE

SQL: utiliza uma sintaxe declarativa para consultas, onde você descreve o que deseja obter.

Cypher: possui uma sintaxe declarativa e visual, otimizada para expressar padrões em grafos, facilitando a navegação e a pesquisa de relacionamentos.

OPERAÇÕES DE RELACIONAMENTO

SQL: para realizar operações em relacionamentos, muitas vezes, requer joins complexos para unir tabelas.

Cypher: facilita a representação e a manipulação de padrões de relacionamento diretamente, simplificando a navegação em grafos.

FLEXIBILIDADE NA MODELAGEM

SQL: mais bem adaptada para dados estruturados com relações simples.

Cypher: ideal para representar e consultar dados altamente conectados, como redes sociais, hierarquias e sistemas complexos.

ORIENTAÇÃO PARA GRAFOS:

SQL: não é otimizada para lidar com dados de grafo, muitas vezes, exigindo consultas complexas para representar e recuperar relações.

Cypher: projetada desde o início para operar em dados de grafos, tornando as consultas mais intuitivas e eficientes em ambientes de grafos.

Essas diferenças refletem a natureza dos bancos de dados subjacentes e as necessidades específicas que cada linguagem de consulta visa atender. Cada uma tem seu lugar no cenário de gerenciamento de dados, dependendo das características e das complexidades dos dados a serem manipulados.

Em termos de aplicabilidade, a CQL brilha em cenários em que os dados são naturalmente representados como grafos, com muitas relações e conexões, permitindo expressar consultas de forma mais intuitiva, facilitando a navegação por grafos complexos e a busca de padrões específicos de relacionamentos.

Por outro lado, a SQL é amplamente utilizada em sistemas que adotam o modelo relacional, ideal para estruturar dados em tabelas e realizar operações, como seleção, projeção e junção de conjuntos. É eficaz em ambientes em que as relações são predominantemente representadas de forma tabular.

A escolha entre CQL e SQL dependerá, portanto, da natureza dos dados e do modelo de banco de dados adotado. Projetos que, segundo Basso (2020), envolvem dados altamente conectados e relacionados podem se beneficiar da CQL, enquanto aqueles que seguem uma abordagem relacional tradicional podem preferir a SQL.

Utilizando CQL e SQL simultaneamente

A integração simultânea da Cypher Query Language (CQL) e da Structure Query Language (SQL) em um sistema de software representa uma abordagem avançada e versátil para a manipulação de dados. Ambas as linguagens atendem a propósitos distintos: a primeira é especializada em consultas utilizando o conceito de grafos em bancos de dados NoSQL, enquanto a segunda é tradicionalmente usada para operações em modelos relacionais.

Por exemplo, imagine um sistema de comércio eletrônico que armazena dados transacionais em um banco de dados relacional, e as relações entre clientes, produtos e preferências em um banco de dados de grafos. Com a utilização simultânea das duas linguagens, é possível realizar operações complexas e robustas com uma amplitude maior de possibilidades no mesmo sistema. Mediante instruções SQL, é possível extrair informações provenientes das vendas, tais como o total de receitas em determinado período de tempo. Por outro lado, as instruções CQL permitem explorar os padrões de recomendação entre produtos com base nas interações realizadas pelos clientes.

Essa abordagem é particularmente valiosa em sistemas complexos, em que diferentes paradigmas de dados são necessários para representar de maneira eficiente as diversas relações existentes. Vale ressaltar que as demandas atuais exigem a utilização de diferentes tipos de bancos de dados, com o intuito de atender às necessidades. No entanto, a integração deve ser cuidadosamente planejada para garantir consistência e desempenho.

Segundo Silva *et al.* (2021), *frameworks* e ferramentas específicas podem ser adotadas para gerenciar essa dualidade, permitindo que desenvolvedores extraiam o melhor de ambos os mundos. É importante destacar que, embora a simultaneidade de CQL e SQL ofereça flexibilidade, os desenvolvedores devem considerar as implicações de desempenho e manutenção. A complexidade adicional pode demandar um esforço significativo para garantir que as transições entre as duas linguagens sejam suaves e eficientes.

A utilização conjunta de ambas proporciona uma solução poderosa para a manipulação de dados diversificados em sistemas modernos. Entretanto, requer uma abordagem cuidadosa e compreensão profunda dos paradigmas envolvidos.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Entender bancos de dados NoSQL orientados a grafos é fundamental para profissionais de tecnologia, pois representa uma abordagem inovadora na gestão de dados. Essa categoria de bancos de dados se destaca pela capacidade dela de modelar relações complexas entre entidades utilizando a estrutura de grafos para representar nós e arestas. Esse conhecimento oferece uma alternativa poderosa aos modelos tradicionais, proporcionando flexibilidade na manipulação de dados interconectados.

Profissionais que dominam o conceito desse tipo de banco de dados estão em alta, já que existe, atualmente, uma grande demanda no mercado. Com o crescente reconhecimento da eficácia desse banco na modelagem e na consulta de dados interconectados, empresas de diversos setores buscam especialistas capazes de aproveitar essas tecnologias. Esses profissionais são essenciais para projetos que envolvem redes sociais, análise de fraudes, recomendações personalizadas e sistemas que lidam com

dados altamente inter-relacionados. O conhecimento sólido nessa área não apenas abre portas para oportunidades de emprego desafiadoras, mas também coloca os profissionais em uma posição estratégica para enfrentar os desafios crescentes relacionados à manipulação eficiente de dados complexos e interconectados.

Ao criar conexões com os benefícios desses bancos de dados, o profissional entenderá como eles se destacam em situações que envolvem dados altamente inter-relacionados. A flexibilidade na modelagem, a eficiência nas consultas de relações complexas e a escalabilidade horizontal são características que fazem desses bancos uma escolha valiosa em cenários, como redes sociais, análise de redes, sistemas de recomendação e logística.

Entretanto, também é crucial compreender os desafios associados ao uso de bancos de dados orientados a grafos. O design de um modelo eficiente, o gerenciamento de grandes volumes de dados e a escolha adequada em relação às características específicas do projeto são aspectos que demandam atenção. Um profissional bem-informado será capaz de equilibrar as vantagens e os desafios para aplicar essa tecnologia de forma eficaz em diferentes contextos.

Ao integrar esse conhecimento ao ambiente profissional, você estará preparado(a) para explorar oportunidades em empresas que demandam soluções eficientes para o tratamento de dados inter-relacionados. Além disso, a compreensão das nuances desses bancos de dados contribuirá para o desenvolvimento de habilidades valorizadas no mercado, colocando você em uma posição estratégica em um cenário cada vez mais orientado a dados e interconexões.

1. O processo de armazenamento de dados em bancos de dados é essencial para organizar e preservar informações de maneira eficiente. Os dados são estruturados e organizados em tabelas, documentos ou outros formatos, dependendo do tipo de banco de dados utilizado. Durante esse processo, as informações passam por fases, como a inserção, a atualização e a recuperação, garantindo a integridade e a consistência dos dados. Além disso, sistemas de gerenciamento de banco de dados (SGBD) desempenham um papel crucial, facilitando a manipulação e a consulta dos dados de maneira rápida e segura. Esse procedimento é fundamental para a tomada de decisões, as análises e as diversas operações em aplicações que dependem de dados estruturados.

Qual é a principal finalidade do armazenamento permanente de dados em um sistema de banco de dados?

- a) Armazenar os dados.
 - b) Garantir a durabilidade e a disponibilidade das informações ao longo do tempo.
 - c) Reduzir o espaço ocupado pelos dados.
 - d) Acelerar o processo de escrita no banco de dados.
 - e) Facilitar a compressão de dados para a economia de armazenamento.
-
2. Grafos são estruturas matemáticas que representam relações entre entidades por meio de conjuntos de nós (vértices) interligados por arestas. Cada aresta indica uma conexão entre dois vértices e pode ser direcionada ou não direcionada, dependendo do tipo de relação que representa. Os grafos são utilizados para modelar uma variedade de situações do mundo real, desde redes sociais e sistemas de transporte até representações abstratas de relações entre elementos. A versatilidade deles os torna fundamentais em teoria dos grafos, facilitando a análise e a compreensão de complexas interconexões em diversas disciplinas, como ciência da computação, matemática e engenharia.

AUTOATIVIDADE

Considere as seguintes afirmativas sobre a Cypher Query Language, uma linguagem de consulta utilizada em bancos de dados de grafos:

- I - Cypher é uma linguagem específica para consulta e manipulação de dados em bancos de dados relacionais.
- II - A sintaxe Cypher é projetada para ser intuitiva, ao representar e navegar por dados em forma de grafo.
- III - A Cypher é amplamente adotada em ambientes que utilizam o modelo de banco de dados relacional.
- IV - A cláusula MATCH é usada para especificar padrões de dados a serem encontrados no banco de dados de grafos.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

AUTOATIVIDADE

3. Uma linguagem de manipulação de dados em um banco de dados é um conjunto de comandos e instruções que permite a interação eficiente com as informações armazenadas. Essa categoria de linguagens engloba tanto a Linguagem de Consulta Estruturada (SQL) quanto as outras linguagens específicas com a Cypher Query Language (CQL), que facilitam a inserção, a atualização, a recuperação e a exclusão de dados dentro de um sistema de gerenciamento de banco de dados. Por meio dessas linguagens, os usuários podem realizar diversas operações, desde consultas simples até complexas transações, garantindo a flexibilidade necessária para interagir com as informações de forma precisa e eficaz.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

- I - A Cypher Query Language é adequada para consultas simples e suporta operações complexas em bancos de dados de grafos.

PORQUE

- II - A cláusula RETURN em Cypher é utilizada para especificar os padrões de dados a serem encontrados em uma consulta.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

BASSO, D. E. **Big Data**. Curitiba: Contentus, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

SADALAGE, P. J.; FOWLER, M. **Nosql Essencial**. São Paulo: Novatec, 2013.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

SILVA, L. F. C. *et al.* **Banco de dados não relacional**. Porto Alegre: SAGAH, 2021.

GABARITO

1. Opção B.

O armazenamento permanente visa manter os dados de forma persistente, assegurando a durabilidade e a disponibilidade, independentemente de reinicializações do sistema. As demais opções são incorretas, pois não refletem a principal finalidade do armazenamento permanente, que é preservar a integridade dos dados ao longo do tempo.

2. Opção B.

A Cypher foi projetada especificamente para consulta e manipulação de dados em bancos de dados de grafos, proporcionando uma sintaxe intuitiva para representar e navegar por dados em formato de grafo. A cláusula MATCH é usada em Cypher para especificar os padrões de dados que se deseja encontrar no banco de dados de grafos. Cypher não é uma linguagem específica para consulta em bancos de dados relacionais, mas em bancos de dados de grafos. A Cypher não é amplamente adotada em ambientes que utilizam o modelo de banco de dados relacional: o foco dela está nos bancos de dados de grafos.

3. Opção C.

A Cypher Query Language é adequada tanto para consultas simples quanto para operações mais complexas em bancos de dados de grafos. Ela oferece uma sintaxe intuitiva para representar e navegar por dados em forma de grafo, suportando operações diversas. A cláusula RETURN em Cypher não é utilizada para especificar padrões de dados, mas para especificar o que deve ser retornado como resultado da consulta. A cláusula MATCH é utilizada para especificar padrões de dados a serem encontrados em uma consulta.



TEMA DE APRENDIZAGEM 9

FLEXIBILIDADE DE ARMAZENAMENTO EM BANCOS DE DADOS ORIENTADO A COLUNAS

MINHAS METAS

- Desenvolver uma compreensão aprofundada do paradigma NoSQL.
- Focar no entendimento detalhado dos bancos de dados orientados a colunas.
- Realizar comparações entre bancos de dados orientados a colunas e outros modelos NoSQL.
- Aplicar os conhecimentos adquiridos em casos práticos.
- Desenvolver habilidades para a otimização de consultas em bancos de dados orientados a colunas.
- Explorar a integração de bancos de dados orientados a colunas com diversas ferramentas e frameworks.
- Reconhecer os desafios e as limitações inerentes aos bancos de dados orientados a colunas.

INICIE SUA JORNADA

Bem-vindo(a) à emocionante jornada de aplicação prática dos conceitos que você adquiriu sobre a flexibilidade de armazenamento em bancos de dados orientados a colunas. Como estudante de Tecnologia da Informação (TI), você está prestes a mergulhar em um campo inovador, em que a teoria encontra a implementação real e os paradigmas tradicionais de armazenamento de dados são desafiados. Essa é a sua oportunidade de ir além das salas de aula e laboratórios, colocando em prática as habilidades que te capacitarão a enfrentar os desafios da gestão eficiente de grandes conjuntos de dados.

Nessa jornada, você se deparará com a oportunidade de compreender como a flexibilidade intrínseca dos bancos de dados orientados a colunas pode impactar positivamente a eficiência no manuseio de informações. Explore casos de uso reais, mergulhe em estudos de caso envolventes e enfrente os desafios que surgem ao lidar com estruturas de dados inovadoras. Este é o momento de transformar o conhecimento teórico em experiência prática, solidificando o seu entendimento e te preparando para os cenários dinâmicos de TI.

Ao iniciar esta jornada, mantenha a mente aberta para as possibilidades que se desdobram diante de você. A flexibilidade desse tipo de banco de dados não é apenas um conceito: é uma revolução que oferece a oportunidade de vivenciar em primeira mão. Esteja preparado(a) para desbravar terrenos desconhecidos, enfrentar desafios com determinação e colher os frutos do conhecimento transformado em competência prática.

Lembre-se: esta jornada não é apenas sobre o destino final, mas também sobre a evolução contínua ao longo do caminho. Sintonize-se na empolgação dessa jornada única enquanto aplica os conceitos aprendidos, amplia a sua experiência e se prepara para se destacar no vasto campo da Tecnologia da Informação.

A aventura está apenas começando: inicie a sua jornada agora!

**PLAY NO CONHECIMENTO**

Dê um "play" no conhecimento com o podcast Desbravando Dados, em que mergulhamos nas profundezas da tecnologia para desvendar os segredos da flexibilidade de armazenamento em bancos de dados orientados a colunas. Neste episódio envolvente, exploraremos como essa abordagem revolucionária vai além das fronteiras dos modelos tradicionais, permitindo a eficiência no armazenamento e na recuperação de grandes volumes de dados.

Ao longo do programa, desvendaremos as características únicas que tornam os bancos de dados orientados a colunas uma escolha poderosa para cenários específicos, abrindo caminho para uma compreensão profunda de como essas estruturas flexíveis moldam o mundo do gerenciamento de dados. Assim, especialistas e profissionais da área compartilham insights valiosos, proporcionando uma experiência enriquecedora para estudantes, entusiastas e profissionais de Tecnologia da Informação.

Prepare-se para uma viagem fascinante pelo universo dos dados, em que a flexibilidade não é apenas uma característica, mas uma revolução. Está na hora de desbravar as possibilidades infinitas dos bancos de dados orientados a colunas. Sintonize no Desbravando Dados e expanda o seu conhecimento com um toque inovador na tecnologia de armazenamento de dados.

VOCÊ SABE RESPONDER?

Os dados não estruturados são caracterizados pela ausência de um formato predefinido, representando uma faceta desafiadora e rica em variedade, incluindo documentos, vídeos, imagens e mensagens. Essa diversidade exige abordagens inovadoras e é nesse cenário que o banco de dados NoSQL entra em ação, oferecendo flexibilidade e escalabilidade para lidar eficientemente com essas informações.

Ao recordar o cenário dos dados não estruturados, é essencial destacar a adaptabilidade proporcionada pelo NoSQL, que se destaca ao abraçar modelos, como documentos, grafos e famílias de colunas, adequando-se melhor à natureza fluida do problema. Essa flexibilidade é uma vantagem valiosa em ambientes dinâmicos, permitindo a gestão eficaz de grandes volumes de dados heterogêneos.

Nesse contexto, destacam-se diferentes tipos de abordagens: cada uma oferece um conjunto específico de características, alinhando-se às demandas particulares. Assim, você fortalecerá a sua compreensão sobre a arquitetura e as funcionalidades desses bancos de dados, preparando-se para enfrentar os desafios contemporâneos da gestão na era da informação. Recordar esses conhecimentos não apenas reafirma a sua base teórica, mas também te capacita a aplicar os princípios de forma estratégica em ambientes práticos da TI.

<https://www.youtube.com/watch?v=INTvbsyHwhc>

DESENVOLVA SEU POTENCIAL

BANCO DE DADOS NOSQL ORIENTADO A COLUNAS

Os bancos de dados NoSQL orientados a colunas representam uma categoria inovadora de sistemas de gerenciamento de dados que se destacam pela flexibilidade e eficiência em determinados cenários.

Diferentemente dos bancos de dados relacionais tradicionais, que, segundo Elmasri e Nanathe (2018), organizam dados em linhas e colunas, o NoSQL orientado a colunas utiliza uma abordagem que agrupa os dados por colunas, proporcionando uma série de vantagens distintas.

A estrutura desse tipo de banco de dados é particularmente adequada para situações em que as operações de leitura são predominantes. A organização dos dados por colunas permite a recuperação seletiva de informações, o que resulta em maior eficiência em comparação com as abordagens orientadas à linha. Essa característica torna esses bancos de dados ideais para cenários nos quais a análise e a consulta de grandes volumes de dados são comuns.

Outro destaque do NoSQL nesse contexto é a capacidade de escalabilidade horizontal dele, permitindo adicionar novos nós ou servidores ao sistema de maneira mais eficiente, possibilitando lidar com um aumento significativo no volume de dados e tráfego sem sacrificar o desempenho. Isso é essencial em ambientes modernos, em que a demanda por armazenamento e processamento de dados está em constante crescimento. A flexibilidade de esquema é uma característica marcante desses bancos de dados.

Diferentemente dos modelos relacionais, segundo Silberschatz (2020), que requerem esquemas predefinidos e rígidos, o NoSQL permite adicionar ou modificar colunas sem a necessidade de alterações globais na estrutura. Essa flexibilidade, segundo Sadalage e Fowler (2013), facilita a adaptação a mudanças nos requisitos e acomoda a natureza dinâmica dos dados modernos.

Por fim, é relevante mencionar que alguns dos bancos de dados NoSQL orientados a colunas mais conhecidos atualmente incluem Apache Cassandra e HBase. Essas plataformas implementam de maneira eficiente os princípios desse modelo, oferecendo aos desenvolvedores e DBAs uma alternativa robusta e altamente especializada para cenários específicos.

Esse tipo de banco de dados representa uma abordagem inovadora no gerenciamento de dados, proporcionando eficiência em leituras, escalabilidade horizontal, flexibilidade de esquema e adequação para ambientes em que a análise de grandes volumes de informações é fundamental. Essa arquitetura continua a desempenhar um papel crucial na evolução do panorama de gerenciamento de dados, sendo uma escolha valiosa em diversas aplicações modernas.

O SGBD Apache Cassandra

Trata-se de um robusto SGBD NoSQL projetado para administrar grandes volumes de dados distribuídos de maneira altamente escalável e resiliente.

Desenvolvido pelo Facebook e, posteriormente, concebido como um projeto de código aberto, o Cassandra foi projetado para superar os desafios associados ao armazenamento e à recuperação eficiente em escala global, tornando-se uma escolha popular para empresas que buscam flexibilidade e desempenho.

Uma característica central é a arquitetura distribuída e descentralizada, operando em um modelo *peer-to-peer*, em que todos os nós em um *cluster* são iguais, eliminando pontos únicos de falha e permitindo uma escalabilidade horizontal fácil.

A tolerância a falhas é uma característica essencial desse SGBD .

Essa arquitetura é, segundo Silva *et al.* (2021), particularmente eficaz em ambientes em que a expansão do hardware é uma necessidade constante, proporcionando uma solução flexível para grandes conjuntos de dados.

Outro destaque do Cassandra é a modelagem de dados baseada em colunas. Os dados são organizados em famílias de colunas. Essa estrutura oferece uma flexibilidade significativa em comparação com o modelo relacional. Além disso, suporta consultas baseadas em indexação secundária, o que simplifica a recuperação de dados mesmo em conjuntos de dados vastos e distribuídos.

A tolerância a falhas é uma característica essencial desse SGBD. O sistema é projetado para operar continuamente mesmo quando alguns nós do *cluster* falham. Conceitos, como replicação de dados entre os nós, e estratégias de consistência configuráveis permitem que o Cassandra mantenha a integridade mesmo em ambientes instáveis.

A linguagem de consulta CQL (Cassandra Query Language) simplifica bastante a interação com o banco de dados, tornando a sintaxe semelhante ao SQL e facilitando a transição para desenvolvedores familiarizados com o modelo relacional. Isso torna o Cassandra acessível para uma ampla gama de profissionais de TI, facilitando a adoção e a integração em diversos ambientes de desenvolvimento.

Assim, o SGBD continua a ser uma escolha proeminente em cenários em que a escalabilidade e a resiliência são críticas. A adoção dele é evidente em empresas líderes em diversos setores, como serviços financeiros, telecomunicações e IoT, destacando o papel significativo dele no cenário de bancos de dados NoSQL.

Com uma comunidade ativa de desenvolvedores e atualizações regulares, o Cassandra permanece, segundo Basso (2020), na vanguarda da inovação, consolidando o lugar dele como uma solução confiável e poderosa para as demandas dinâmicas do gerenciamento de dados em larga escala.

Cassandra Query Language (CQL)

Trata-se de um componente fundamental do ambiente do Apache Cassandra projetada para simplificar a comunicação entre os desenvolvedores e o Cassandra. A CQL combina a familiaridade da SQL com a flexibilidade característica dos bancos de dados NoSQL, tornando a transição para esse ambiente acessível e intuitiva.

A CQL compartilha muitas semelhanças sintáticas com a SQL convencional, o que facilita a adoção por profissionais que já estão familiarizados com consultas relacionais. No entanto, segundo Silva *et al.* (2021), é importante observar que, embora compartilhe semelhanças superficiais, ela foi projetada para se alinhar às características específicas do modelo orientado a colunas do Cassandra.

Essa adaptação permite uma transição suave para os desenvolvedores que desejam explorar os benefícios de um banco de dados distribuído e altamente escalável.

Uma característica essencial dessa linguagem é a capacidade dela de modelar dados em famílias de colunas, proporcionando uma estrutura flexível para armazenar e recuperar informações. As operações de CRUD (Create, Read, Update, Delete) são facilmente realizadas, e a CQL suporta consultas complexas, permitindo a recuperação seletiva de dados em grandes conjuntos distribuídos. Essa flexibilidade é crucial em ambientes dinâmicos, em que os requisitos de modelagem de dados podem evoluir rapidamente.

Assim, a CQL também oferece suporte a índices secundários, facilitando consultas eficientes em colunas não primárias. Isso, segundo Basso (2020), amplia a ca-

pacidade de recuperação e aprimora o desempenho em ambientes complexos, em que as consultas precisam abranger diversos aspectos dos dados armazenados. A indexação secundária é uma ferramenta valiosa para otimizar o acesso a informações específicas sem sacrificar a escalabilidade do sistema.

Essa linguagem foi projetada para suportar transações atômicas, garantindo a consistência em operações simultâneas. Isso é crucial em ambientes em que a integridade dos dados é uma prioridade, e as operações devem ser executadas de forma confiável e isolada.

Por fim, a CQL desempenha um papel central na experiência de desenvolvimento com o Cassandra, oferecendo uma interface poderosa e intuitiva para interagir com o banco de dados. A semelhança dela com a SQL tradicional, combinada com recursos específicos do Cassandra, a torna uma linguagem versátil para explorar as capacidades do NoSQL. Nesse contexto, é possível afirmar que a CQL é uma ferramenta crucial no kit de desenvolvimento para aqueles que buscam aproveitar ao máximo o potencial do SGBD em ambientes distribuídos e de grande escala.

Em um exemplo prático, considere uma aplicação que precisa gerenciar informações de usuários em um ambiente distribuído que utiliza o Cassandra. Para isso, há a necessidade de criar uma tabela chamada “USUARIOS” que contém as seguintes colunas: “id_usuario”, “nome”, “email” e “idade”.

**A torna uma
linguagem versátil
para explorar as
capacidades do
NoSQL**

- **Criação da tabela:** para criar a tabela, a linguagem CQL será utilizada da seguinte forma:

```
CREATE TABLE usuarios (  
    id_usuario UUID PRIMARY KEY,  
    nome TEXT,  
    email TEXT,  
    idade INT  
);
```

Figura 1 - Instrução CQL para criar uma tabela chamada USUARIOS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com cinco linhas de código referentes aos dados dos usuários a serem inseridos no banco de dados. Os dados são referentes a: ID_USUARIO, NOME, EMAIL e IDADE. Fim da descrição.

Após a criação da tabela, é necessário que ela seja devidamente preenchida com os dados referentes aos usuários.

O termo UUID se refere à Universally Unique Identifier ou, em português, Identificadores Únicos para Garantir Unicidade. Trata-se de um tipo de dado utilizado para atribuir identificadores únicos a registros armazenados em um banco de dados distribuído. Esses identificadores são cruciais para garantir a unicidade dos registros em ambientes distribuídos, em que a geração de chaves primárias tradicionais pode se tornar um desafio. O UUID é composto por 32 caracteres hexadecimais divididos em cinco grupos, proporcionando uma ampla variedade de combinações e, assim, minimizando as chances de colisões de identificadores.

Essa abordagem é particularmente valiosa em cenários, como sistemas de rastreamento, registros de eventos e qualquer aplicação em que a garantia de unicidade seja essencial, independentemente da escala ou do número de nós no cluster do Cassandra.

- **Inserção de dados:** agora, deve-se inserir os dados na tabela USUARIOS.

Neste exemplo, inseriremos dois novos usuários chamados João e Maria:

```
INSERT INTO usuarios (id_usuario, nome, email, idade)
VALUES (uuid(), 'João Silva', 'joao@email.com', 25)

INSERT INTO usuarios (id_usuario, nome, email, idade)
VALUES (uuid(), 'Maria Oliveira', 'maria@email.com', 30)
```

Figura 2 - Instrução CQL para inserir dados na tabela USUARIOS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo o script CQL com quatro linhas de código para inserir novos registros na tabela USUARIOS. Os dados são referentes aos novos usuários chamados João Silva e Maria Oliveira. Fim da descrição.

- **Consulta de dados:** podemos recuperar informações específicas utilizando a CQL, buscando todos os usuários com idade superior a 25 anos:

```
SELECT * FROM usuarios WHERE idade > 25;
```

Figura 3 - Instrução CQL para fazer busca de dados na tabela USUARIOS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com 1 linha de código para consultar dados na tabela USUARIOS. Nessa consulta, há uma filtragem de dados para somente os usuários com idade superior a 25 anos. Fim da descrição.

- **Atualização de dados:** caso seja preciso atualizar as informações de um usuário, a CQL nos permite fazer isso de maneira eficiente.

Considere o exemplo a seguir, atualizando a idade do usuário especificado:

```
UPDATE usuarios SET idade = 26 WHERE id_usuario = <id_do_usuario>;
```

Figura 4 - Instrução CQL para atualizar dados na tabela USUARIOS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com 1 linha de código para atualizar dados na tabela USUARIOS. Nessa atualização, há uma filtragem de dados pelo nome do usuário e alteração da idade para 26 anos. Fim da descrição.

- **Remoção de dados:** por fim, assim como no SQL, é possível remover um usuário da tabela utilizando a CQL.

No exemplo a seguir, é excluído o registro do usuário com o ID correspondente:

```
DELETE FROM usuario WHERE id_usuario = <id_do_usuario>;
```

Figura 5 - Instrução CQL para excluir registros na tabela USUARIOS / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script CQL com 1 linha de código para excluir registros na tabela USUARIOS. Nessa exclusão, há uma filtragem de dados pelo nome do usuário, removendo, assim, da tabela, o registro correspondente. Fim da descrição.

O exemplo citado ilustra como a CQL pode ser utilizada para realizar operações CRUD (Create, Read, Update, Delete) em um banco de dados Cassandra. A sintaxe familiar com o SQL facilita o desenvolvimento e a manutenção de aplicações em ambientes distribuídos, proporcionando eficiência e escalabilidade.

**APROFUNDANDO**

CRUD, um acrônimo para *Create* (Criar), *Read* (Ler), *Update* (Atualizar) e *Delete* (Excluir), é um conjunto fundamental de operações utilizado em SGBDs e aplicações para a manipulação eficiente de dados.

A partir do CRUD, é possível criar novos registros, ler informações existentes, atualizar dados existentes e excluir o que for desnecessário. Essas operações formam a base essencial para a interação com sistemas de informação, proporcionando uma estrutura universal para a gestão completa em diversas aplicações.

Ele é amplamente empregado em desenvolvimento de softwares, facilitando a implementação de operações básicas de manipulação em sistemas de todos os tamanhos e complexidades.

Segurança da Informação no Apache Cassandra

O Apache Cassandra, sendo um SGBD distribuído, coloca a segurança da informação como uma prioridade fundamental para proteger dados sensíveis e garantir a integridade do sistema computacional.

Uma das principais considerações nesse quesito é, segundo Silva *et al.* (2021), o controle da acessibilidade de dados, oferecendo robustos mecanismos de autenticação e autorização, permitindo a definição precisa de quem pode acessar e manipular dados específicos dentro do *cluster* distribuído.

A autenticação no Cassandra é, geralmente, realizada por meio de uma variedade de provedores, como o LDAP (Lightweight Directory Access Protocol), e a autenticação baseada em certificados. Isso garante que apenas usuários autorizados tenham permissão para interagir com o banco de dados. Além disso, o Cassandra suporta papéis e permissões, possibilitando a configuração fina das autorizações para garantir que cada usuário tenha acesso apenas às operações permitidas.

**APROFUNDANDO**

O LDAP (Lightweight Directory Access Protocol) é uma tecnologia que fornece um protocolo de comunicação eficiente para acessar e gerenciar informações em diretórios de maneira leve e ágil. Projetado para ser simples e escalável, facilita a consulta e a atualização de dados armazenados em diretórios distribuídos, como informações de usuários em sistemas de autenticação centralizada.

Amplamente utilizado em ambientes corporativos, esse protocolo oferece uma estrutura padronizada para organizar e recuperar dados de diretórios, tornando-se uma peça fundamental em sistemas que requerem gerenciamento eficaz de informações distribuídas.

Outro aspecto crítico de segurança gerenciado pelo Cassandra é a criptografia de dados em trânsito e em repouso. A comunicação entre os nós do *cluster* pode ser protegida utilizando SSL/TLS (Secure Sockets Layer/Transport Layer Security), enquanto essa criptografia em repouso é essencial para proteger informações sensíveis armazenadas no disco. Essas medidas garantem a confidencialidade das informações durante a transferência e o armazenamento.

**ZOOM NO CONHECIMENTO**

SSL (Secure Sockets Layer) e TLS (Transport Layer Security) são protocolos de segurança essenciais que fornecem camadas adicionais de proteção para as comunicações na internet. Projetados para garantir a confidencialidade e a integridade transmitidos entre um cliente e um servidor, esses protocolos utilizam criptografia para proteger as informações sensíveis contra interceptação por terceiros.

O SSL foi a primeira versão desse conjunto de protocolos, mas foi sucedido pelo TLS. Ambos são amplamente empregados em transações on-line, *login* em páginas da web, e-mails e outras atividades que exigem a troca segura de dados na vastidão da internet, proporcionando uma camada crítica de segurança para proteger a privacidade e a integridade das informações transmitidas.

Além disso, o Cassandra oferece funcionalidades de auditoria, permitindo o monitoramento contínuo das atividades do banco de dados. Isso inclui o rastreamento de autenticações bem-sucedidas e malsucedidas, consultas executadas e alterações na estrutura do esquema. Esses registros são vitais para a detecção de atividades suspeitas e auxiliam nas investigações em caso de incidentes de segurança.

A segurança física do ambiente em que os nós do *cluster* do Cassandra estão hospedados também seria uma preocupação. Garantir que apenas o pessoal autorizado tenha acesso aos servidores e que os centros de dados estejam protegidos contra ameaças físicas contribui para uma camada adicional de proteção.

A segurança da informação é, segundo Sadalage e Fowler (2013), uma abordagem abrangente que engloba desde a autenticação de usuários até a criptografia e as auditorias detalhadas. Essas medidas visam garantir a confidencialidade, a integridade e a disponibilidade dos dados em ambientes distribuídos, tornando o Cassandra uma escolha confiável para organizações que priorizam a segurança na gestão de grandes volumes de informações críticas.

APACHE SPARK

É um poderoso *framework* de processamento de dados em código aberto que revolucionou a maneira como é necessário trabalhar com grandes volumes de informações simultaneamente. Projetado para oferecer velocidade, facilidade de uso e versatilidade, o Spark simplifica tarefas complexas de processamento distribuído, tornando-o uma escolha popular em ambientes de Big Data. Uma das características distintivas dele é o conceito de Resilient Distributed Datasets (RDD), uma abstração de dados imutáveis distribuídos que permite a execução eficiente de operações paralelas.

A arquitetura dele é composta por diversos módulos, sendo o Spark Core o núcleo do sistema, fornecendo as funcionalidades básicas, incluindo manipulação de RDDs e operações fundamentais de transformação e ação. Além disso, o Spark SQL permite a integração de consultas SQL tradicionais ao processamento de dados distribuídos, simplificando a interação com usuários familiarizados com linguagens de banco de dados.

**Projetado para
oferecer velocidade,
facilidade de uso e
versatilidade**

**ZOOM NO CONHECIMENTO**

Dentre outros recursos, podemos destacar o **Spark Streaming**, que estende as capacidades do *framework* para processamento em tempo real, permitindo a análise contínua de dados em fluxo. Enquanto isso, o **Spark MLlib** oferece uma biblioteca de aprendizado de máquina distribuído, simplificando o desenvolvimento de modelos em ambientes de grande escala. Já o **Spark GraphX** foca em operações de processamento de gráficos, tornando possível análises complexas de redes e relacionamentos.

A eficiência do Spark é impulsionada pelo mecanismo de execução em memória, que minimiza a necessidade de operações de I/O em disco, resultando em processamento mais rápido. A capacidade dele de se integrar com diversos sistemas de armazenamento, como Hadoop Distributed File System (HDFS) e Apache HBase, facilita a incorporação em ambientes já existentes.

Nesse contexto, é possível afirmar que o Apache Spark se destaca como uma ferramenta abrangente para o processamento de dados em larga escala, oferecendo uma ampla gama de funcionalidades que vão desde o processamento em lote até análises em tempo real e aprendizado de máquina. A abordagem flexível, a eficiência e a integração dele com ecossistemas de Big Data consolidam a posição dele como uma escolha crucial para organizações que buscam extrair *insights* valiosos a partir de grandes conjuntos de dados.

Em um exemplo prático, suponha que uma empresa de comércio eletrônico com uma imensa quantidade de dados de vendas deseja realizar uma análise abrangente para obter *insights* valiosos. Nesse cenário, o Apache Spark pode ser utilizado para processar e analisar esses dados em larga escala.

- **Carregamento dos dados:** primeiramente, as informações de vendas são carregadas no ambiente do Spark a partir de fontes, como arquivos CSV ou outras bases de dados.

```
sales_data = spark.read.csv("dados_de_vendas.csv", header=True, inferSchema=True)
```

Figura 6 - Script para busca de dados existentes em um arquivo externo e carregados na aplicação / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script a ser executado no Apache Spark com 1 linha de código referente à busca de dados em um arquivo do tipo CSV do Excel sobre as vendas realizadas pela loja. Fim da descrição.

- **Transformação de dados usando RDDs:** o Spark permite a criação de RDDs para representar e manipular os dados.

Por exemplo, é possível utilizar transformações para filtrar apenas as vendas de um determinado produto.

```
filtered_sales_rdd = sales_data.rdd.filter(lambda venda: venda["produto"] == "smartphone");
```

Figura 7 - Script para criar um RDD para filtrar os dados carregados na aplicação / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script a ser executado no Apache Spark com 1 linha de código referente à criação de um RDD e à filtragem de dados sobre as vendas realizadas pela loja. Fim da descrição.

- **Análise com Spark SQL:** é possível executar consultas SQL diretamente nos dados.

Nesse caso, calcularemos o total de vendas por região.

```
sales_data.createOrReplaceTempView("vendas")
resultado_analise = spark.sql("SELECT regioao, SUM(valor) as total_vendas FROM vendas");
```

Figura 8 - Script para executar uma instrução SQL calculando o total de vendas de uma determinada região / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script a ser executado no Apache Spark com 1 linha de código referente à busca do valor total de vendas realizadas em uma determinada região. Fim da descrição.

- **Aplicação de Machine Learning com MLlib:** para prever as vendas futuras, é possível utilizar o módulo de Machine Learning do Spark chamado MLlib.

Neste exemplo, implementaremos um modelo de regressão linear simples.

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression

assembler = VectorAssembler(inputCols=["quantidade", "preco"], outputCol="features");
data_assembled = assembler.transform(sales_data)

model = LinearRegression(featuresCol="features", labelCol="valor")
trained_model = model.fit(data_assembled)
```

Figura 9 - *Script* para treinar o Machine Learning MLlib do Spark utilizando o conceito de busca Regressão Linear / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script a ser executado no Apache Spark com seis linhas de código referentes à execução de uma instrução de regressão linear para treinar a Machine Learning MLlib para situações específicas. Fim da descrição.

- **Streaming em tempo real com Spark Streaming:** se a empresa deseja monitorar vendas em tempo real, o Spark Streaming pode ser empregado para processar e analisar dados conforme são recebidos.

```
streaming_sales_data = spark.readStream.format("csv").load("dados_em_tempo_real");
```

Figura 10 - *Script* para fazer monitoramento de vendas em tempo real / Fonte: o autor.

Descrição da Imagem: é exibida uma tela preta contendo um script a ser executado no Apache Spark com 1 linha de código referente ao monitoramento em tempo real das vendas on-line realizadas pela empresa. Fim da descrição.

- **Armazenamento dos resultados:** por fim, os resultados das análises podem ser armazenados em um banco de dados ou exportados para visualizações interativas, proporcionando *insights* valiosos para a tomada de decisões estratégicas.

O exemplo citado ilustra como o Apache Spark pode ser aplicado em diferentes etapas de análise de dados em larga escala, desde o processamento local no servidor até a execução de algoritmos de Machine Learning e análise em tempo real, demonstrando a versatilidade e a eficácia dele em ambientes de Big Data.

Integração entre o Spark e o Cassandra

A integração do Spark e do Cassandra, ambos desenvolvidos pela empresa Apache, representa uma aliança poderosa no universo do processamento de dados distribuídos. O Spark, com a capacidade de processamento em memória e flexibilidade, é ideal para análises avançadas em grandes conjuntos de dados. Por outro lado, o Cassandra, um banco de dados NoSQL altamente escalável e distribuído, é projetado para armazenar grandes volumes de dados com alta disponibilidade.

Juntos, esses dois sistemas formam uma combinação sinérgica que aborda desafios em ambientes de Big Data.

Uma das principais vantagens da integração é a capacidade do Spark de utilizar o Cassandra como um repositório de dados, facilitando a leitura e a gravação por meio do conceito de DataFrames. Isso permite, segundo Sadalage e Fowler (2013), que as operações de transformação e análise sejam realizadas de maneira transparente, aproveitando as capacidades distribuídas de ambos os sistemas.

Além disso, o **Spark Connector for Cassandra** é uma ferramenta essencial que simplifica a interação entre essas duas tecnologias, oferecendo funcionalidades, como o paralelismo automático de consultas, otimizando a distribuição de tarefas entre os nós das duas estruturas. Isso resulta em um processamento mais eficiente, aproveitando as características de escalabilidade horizontal do Cassandra.

A integração também permite o uso de ambos em conjunto para análises em tempo real, uma vez que o **Spark Streaming** pode ser aplicado aos dados armazenados no Cassandra. Isso é particularmente valioso para cenários em que a capacidade de responder a eventos em tempo real é crucial, como monitoramento de IoT ou análise de dados de transmissões em tempo real.

Por fim, essa integração oferece uma solução abrangente para ambientes de Big Data, combinando a capacidade analítica poderosa do Spark com a alta escalabilidade e a flexibilidade de armazenamento do Cassandra. Essa parceria robusta continua a desempenhar um papel significativo em projetos que exigem processamento e análise eficientes em larga escala, proporcionando aos desenvolvedores e cientistas de dados ferramentas poderosas para explorar e extrair *insights* valiosos de grandes volumes de dados distribuídos.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Ao atingir este ponto em sua jornada acadêmica em Tecnologia da Informação (TI), é crucial compreender a importância de criar conexões entre o conhecimento adquirido até agora e a sua futura trajetória no ambiente profissional. Cada conteúdo, desde a programação até o *design* de sistemas, construiu uma base sólida, fornecendo habilidades e perspectivas essenciais. Agora, é o momento de integrar esses conhecimentos para formar um entendimento holístico, permitindo uma abordagem abrangente para os desafios do mundo profissional. Ao criar conexões, é fundamental identificar as interseções entre as disciplinas, reconhecendo como a programação pode impactar o *design* de sistemas, como a administração de bancos de dados pode influenciar o desenvolvimento de aplicativos, e assim por diante.

Essa visão integrada é inestimável, pois reflete a complexidade do ambiente profissional de TI, em que a colaboração interdisciplinar é frequentemente crucial para o sucesso dos projetos. Além disso, é importante antecipar as futuras tendências e avanços tecnológicos. A rápida evolução no campo da TI demanda uma mentalidade adaptativa e a busca constante por atualização. Esteja ciente das inovações emergentes, participe de comunidades profissionais e se mantenha informado(a) sobre as últimas tendências. Isso não apenas enriquecerá a sua compreensão, mas também demonstrará aos futuros empregadores a sua disposição para se manter relevante em um ambiente dinâmico. O estágio profissional oferece a oportunidade de aplicar as suas habilidades de maneira prática. Ao ingressar em estágios ou em projetos reais, você terá a chance de colocar em prática o seu conhecimento consolidado, enfrentar desafios do mundo real e aprimorar as suas habilidades profissionais. Isso não apenas solidifica o aprendizado, mas também fornece uma experiência valiosa que será um diferencial no mercado de trabalho.

Por fim, não subestime o poder da rede. Construir conexões com colegas, professores, profissionais da indústria e comunidades on-line expande as suas oportunidades. A troca de conhecimentos, experiências e perspectivas não apenas enriquece a sua compreensão, mas também abre portas para oportunidades profissionais, desde colaborações em projetos até possíveis ofertas de emprego. Ao usar o conhecimento acumulado até agora, você está construindo uma base sólida para uma carreira de sucesso em Tecnologia da Informação. Este é o início de uma jornada contínua de aprendizado e crescimento, em que a adaptabilidade e a proatividade são tão essenciais quanto a base técnica. Está na hora de dar o próximo passo com confiança, sabendo que o seu preparo acadêmico é a fundação robusta para os desafios que o futuro profissional reserva.

AUTOATIVIDADE

1. Dados não estruturados se referem a informações que não seguem um formato organizado ou padrão predefinido. Ao contrário dos dados estruturados, como tabelas em bancos de dados relacionais, os dados não estruturados não têm um esquema fixo, permitindo uma variedade de formatos, como texto livre, áudios, vídeos e imagens. Essa diversidade torna os dados não estruturados desafiadores de serem gerenciados por sistemas tradicionais, mas também oferece uma riqueza de informações valiosas. Comumente encontrados na web, redes sociais e fontes de mídia, esses dados exigem abordagens flexíveis e ferramentas especializadas para extração e análise eficazes, destacando a importância de lidar com a complexidade inerente à heterogeneidade dos dados não estruturados.

Qual é uma característica comum dos bancos de dados NoSQL?

- a) Utilizam exclusivamente o modelo relacional para organizar os dados.
 - b) São projetados apenas para ambientes de pequena escala.
 - c) Possuem um esquema de dados flexível, permitindo a adição dinâmica de campos.
 - d) Adotam transações ACID de forma estrita.
 - e) Requerem uma estrutura de dados rigidamente definida antes da inserção de registros.
2. O armazenamento de dados não estruturados em estruturas NoSQL representa uma abordagem flexível e escalável para lidar com informações que não seguem um formato fixo. Diferentemente do modelo relacional, projetados para dados tabulares e estruturados, os bancos de dados NoSQL permitem armazenar dados em diversos formatos, como documentos, grafos, pares chave-valor e colunas. Essa flexibilidade é especialmente vantajosa para lidar com a diversidade de dados provenientes de fontes, como redes sociais, sensores IoT e conteúdo multimídia. Esse tipo de armazenamento capacita as aplicações a evoluírem dinamicamente, adaptando-se a diferentes tipos de dados e facilitando o processamento eficiente de informações não estruturadas em ambientes de grande escala.

Qual das seguintes alternativas sobre o Apache Cassandra está correta?

- a) O Apache Cassandra é um banco de dados relacional que segue estritamente o modelo ACID.
- b) O Apache Cassandra é um sistema de gerenciamento de banco de dados orientado a documentos.
- c) O Apache Cassandra utiliza um modelo de consistência forte para garantir a sincronização entre réplicas.
- d) O Apache Cassandra é projetado para ambientes mononodais e não suporta escalabilidade horizontal.
- e) O Apache Cassandra é um banco de dados NoSQL orientado a colunas, altamente escalável e distribuído.

3. Considere as seguintes afirmativas sobre o Apache Spark:

- I - O Spark suporta apenas a linguagem de programação Java.
- II - O Apache Spark é um *framework* de processamento de dados distribuído e de código aberto.
- III - O Apache Spark é projetado exclusivamente para processamento em lote, não suportando operações em tempo real.
- IV - O Spark inclui módulos para processamento de dados em SQL, Machine Learning e *streaming*.

É correto o que se afirma em:

- a) I e IV, apenas.
- b) II e III, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) II, III e IV, apenas.

REFERÊNCIAS

BASSO, D. E. **Big Data**. Curitiba: Contentus, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. São Paulo: Pearson Education do Brasil, 2018.

SADALAGE, P. J.; FOWLER, M. **Nosql Essencial**. São Paulo: Novatec, 2013.

SILBERSCHATZ, A. **Sistema de banco de dados**. Rio de Janeiro: GEN; LTC, 2020.

SILVA, L. F. C. *et al.* **Banco de dados não relacional**. Porto Alegre: SAGAH, 2021.

GABARITO

1. Opção C.

Os bancos de dados NoSQL são conhecidos pela flexibilidade em relação ao esquema de dados. Ao contrário dos bancos de dados relacionais, eles permitem a adição dinâmica de campos e não exigem uma estrutura de dados rigidamente definida. Essa característica é especialmente útil em cenários em que os requisitos de dados podem evoluir ao longo do tempo, proporcionando agilidade e adaptabilidade. As demais opções estão incorretas, pois os bancos de dados NoSQL não estão restritos ao modelo relacional. Eles podem ser dimensionados para ambientes de qualquer escala, frequentemente, não seguem transações ACID de forma estrita e não requerem uma estrutura de dados fixa.

2. Opção E.

O Apache Cassandra é, de fato, um banco de dados NoSQL orientado a colunas que se destaca pela escalabilidade horizontal e distribuição eficiente em ambientes distribuídos. Ele não segue estritamente o modelo ACID, utiliza um modelo de consistência eventual e é particularmente eficaz para lidar com grandes volumes de dados. As demais opções estão incorretas, pois o Cassandra não é relacional, não utiliza modelo de consistência forte, suporta escalabilidade horizontal e não é orientado a documentos.

3. Opção C.

A afirmativa II está correta, pois o Apache Spark é, de fato, um framework de processamento de dados distribuído e de código aberto. A afirmativa III está correta, pois o Spark suporta tanto o processamento em lote quanto em tempo real, tornando-o versátil para diversas aplicações. A afirmativa IV também está correta, pois o Spark oferece módulos para processamento de dados em SQL (Spark SQL), Machine Learning (MLlib) e streaming (Spark Streaming). A afirmativa I está incorreta, pois o Spark suporta várias linguagens de programação, incluindo Java, Scala, Python e R.